

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA DE
TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

DESARROLLO DE UNA PLATAFORMA DE
COMUNICACIÓN CON PERIFÉRICOS
SOBRE MPSOC DE ALTAS PRESTACIONES
PARA ORDENADORES DE A BORDO EN
SISTEMAS ESPACIALES

JORGE ALEJANDRO ESTEFANÍA HIDALGO

MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

Título: Desarrollo de una plataforma de comunicación con periféricos sobre MPSoC de altas prestaciones para Ordenadores de a Bordo en sistemas espaciales.

Autor: D. Jorge Alejandro Estefanía Hidalgo

Tutor: D. Daniel Sánchez García

Ponente: D. Alvaro Araujo Pinto

Departamento: Departamento de Ingeniería Electrónica

MIEMBROS DEL TRIBUNAL

Presidente: D.

Vocal: D.

Secretario: D.

Suplente: D.

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:
.....

Madrid, a de de 20...

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA DE
TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

**Desarrollo de una plataforma de
comunicación con periféricos sobre
MPSoC de altas prestaciones para
Ordenadores de a Bordo en sistemas
espaciales**

Jorge Alejandro Estefanía Hidalgo

RESUMEN

Este Trabajo de Fin de Máster presenta el desarrollo de una plataforma de comunicación con periféricos sobre el MPSoC Zynq UltraScale+ ZCU102, en el marco del proyecto LINCE, una iniciativa del PERTE Aeroespacial liderada por Indra para el desarrollo de microsátélites de órbita baja.

El trabajo abarca cinco áreas principales.

En primer lugar, el diseño e implementación en VHDL de un transceptor serie altamente configurable para la lógica programable del MPSoC.

En segundo lugar, el desarrollo de un driver serie en C sobre el sistema operativo en tiempo real RTEMS en el sistema de procesamiento del MPSoC, que utiliza el transceptor serie desarrollado y ofrece una API pública que abstrae completamente del hardware.

En tercer lugar, el diseño y fabricación de tres placas de circuito impreso de ampliación de la ZCU102. Las placas se conectan a ella para ampliar sus capacidades y el abanico de interfaces que se pueden desarrollar y probar sobre ella. La primera se diseñó para ejercitar específicamente los enlaces serie RS422 y RS485. Las otras dos, la placa CDHS y la placa AOCS, responden a las necesidades del ordenador de a bordo del satélite. Entre ambas cubren interfaces RS422/RS485, CAN, PWM, señales analógicas y SpaceWire.

En cuarto lugar, el estudio del transporte de datos entre el sistema de procesamiento y la lógica programable, el factor determinante del rendimiento del conjunto. Se implementaron y midieron tres arquitecturas de transporte sobre el mismo transceptor: la primera basada en AXI GPIO, la segunda con un controlador de acceso directo a memoria (DMA) por transceptor y la tercera con un único controlador multicanal (MCDMA) gobernado por un puente diseñado en VHDL. La comparativa mostró que la arquitectura MCDMA ofrece el mejor rendimiento de las tres.

En quinto lugar, la validación de la plataforma mediante el desarrollo de herramientas de software y hardware adicionales, que permitieron verificar el correcto funcionamiento del driver serie y de las distintas funcionalidades de las placas de ampliación.

El conjunto del trabajo constituye una plataforma funcional y validada que el equipo de Sener (la empresa de ingeniería con la que se ha desarrollado el trabajo), Indra y el laboratorio B105 pueden utilizar como base para el subsistema de comunicaciones del ordenador de a bordo del satélite LINCE en el futuro.

SUMMARY

This Master's Thesis presents the development of a peripheral communication platform on the Zynq UltraScale+ ZCU102 MPSoC, within the framework of the LINCE project, an initiative of the Spanish Aerospace PERTE led by Indra for the development of low Earth orbit microsatellites.

The work covers five main areas.

First, the design and implementation in VHDL of a highly configurable serial transceiver for the programmable logic of the MPSoC.

Second, the development of a serial driver in C on the RTEMS real-time operating system running on the processing system of the MPSoC, which uses the serial transceiver developed and provides a public API that fully abstracts the hardware.

Third, the design and manufacture of three expansion printed circuit boards for the ZCU102. The boards connect to it in order to extend its capabilities and broaden the range of interfaces that can be developed and tested on it. The first board was designed to exercise the RS422 and RS485 serial links specifically. The other two, the CDHS board and the AOCS board, respond to the needs of the satellite's on-board computer. Together they cover RS422/RS485, CAN, PWM, analog and SpaceWire interfaces.

Fourth, the study of data transport between the processing system and the programmable logic, the determining factor in the overall performance of the system. Three transport architectures were implemented and measured on the same transceiver: the first one based on AXI GPIO, the second one with a direct memory access (DMA) controller per transceiver, and the third one with a single multichannel controller (MCDMA) governed by a bridge designed in VHDL. The comparison showed that the MCDMA architecture offers the best performance of the three.

Fifth, the validation of the platform through the development of additional software and hardware tools, which made it possible to verify the correct operation of the serial driver and of the different features of the expansion boards.

Taken as a whole, the work constitutes a functional and validated platform that the teams at Sener (the engineering company that partnered on this work), Indra and the B105 laboratory can use in the future as a basis for the communications subsystem of the on-board computer of the LINCE satellite.

PALABRAS CLAVE

MPSoC, FPGA, VHDL, RTEMS, RS422, RS485, CAN, SpaceWire, driver serie, PCB, AXI, DMA, MCDMA, AXI-Stream, sistemas empotrados, tiempo real, satélite, ordenador de a bordo.

KEYWORDS

MPSoC, FPGA, VHDL, RTEMS, RS422, RS485, CAN, SpaceWire, serial driver, PCB, AXI, DMA, MCDMA, AXI-Stream, embedded systems, real time, satellite, on-board computer.

Agradecimientos

A Dani, mi tutor del TFM, por su actitud activa y ayuda constante. Para mí ha sido una suerte tenerte como tutor. Gracias por confiar en mí y por animarme siempre.

A mi compañero de trabajo Diego, ha sido un placer compartir la ZCU102 contigo.

Al equipo de Sener, especialmente a Alba, por su apoyo durante el proyecto y sus respuestas rápidas.

A Alvaro por darme la oportunidad de participar en el proyecto LINCE, por las cervezas viernes si viernes también y por sus valiosos consejos.

Al resto de compañeros del laboratorio, por hacer de él algo más que un lugar de trabajo. Formar parte del B105 ha sido excepcional, he podido disfrutar del buen rollo, empaparme del conocimiento sobre todos vuestros trabajos y contar con vuestra ayuda y consejo en todo momento, sobre el trabajo y sobre temas personales. Vuestro apoyo ha marcado la diferencia y me llevo un gran recuerdo de haber compartido tantos momentos en el labo y de las innumerables veces que me ha tocado ser mopa oficial vistiendo de blanco.

A mis compañeros de clase durante la carrera. La de exámenes que hemos salvado en el último momento gracias a las explicaciones en pizarra de Eugenio. Los croquis de Iratxe sobre el chisme del pueblo. Las largas horas de academia con Nico. La organización de los trabajos de Gonzalo. La pasión por la electrónica, los coches y la música compartida con Carlos. El saber disfrutar de la buena vida con Marcos. Las largas horas de estudio con Javier, los exámenes sacados en un día, las conversaciones profundas y el apoyo que nos hemos dado.

Con este TFM cierro una etapa de mi vida, con la suerte de haber podido trabajar en un satélite de verdad, algo que siempre quise hacer de pequeño. Recordaré con cariño mis años en la universidad, los cuales han sido duros pero a la vez me han convertido en quien soy hoy en día. Gracias a todos los que han participado en este desarrollo y aventura que ha sido pasar por la UPM.

Índice

Resumen y Palabras Clave	ii
Agradecimientos	iv
Lista de acrónimos y glosario	x
1. Introducción y objetivos	1
1.1. Introducción y motivación	1
1.2. Objetivos	2
1.3. Metodología	2
1.3.1. Planificación temporal	3
1.4. Estructura de la memoria y documentación	4
2. Marco teórico	6
2.1. Proyecto LINCE	6
2.2. MPSoC Zynq UltraScale+ ZCU102	6
2.3. Vivado y Vitis (AMD): PS y PL	7
2.3.1. Vivado Design Suite	7
2.3.2. Vitis Unified Software Platform	9
2.4. RTEMS	9
2.5. Comunicaciones serie	9
2.5.1. RS485	10
2.5.2. RS422	10
2.5.3. SpaceWire	11
2.5.4. CAN	12
2.6. Transporte de datos entre el PS y la PL	13
2.6.1. Bus AXI	13
2.6.2. Acceso directo a memoria (DMA)	14
2.6.3. Coste de la interrupción	16
3. Desarrollo	17
3.1. Preparación del entorno de desarrollo	17
3.1.1. Instalación de Vivado y Vitis	17
3.1.2. Instalación de RTEMS	17
3.1.3. Validación del flujo de desarrollo de extremo a extremo	17
3.2. Diseño del transceptor serie configurable (PL)	19
3.2.1. Oscilador controlado numéricamente (NCO)	20
3.2.2. Transmisor	21
3.2.3. Receptor	24
3.2.4. Normalización de la palabra recibida y almacenamiento	26
3.2.5. Bloque de canal	27
3.2.6. Bloque <i>top</i> del transceptor	27
3.2.7. Verificación en simulación del canal completo	30

3.2.8.	Bloque Zynq UltraScale+ MPSoC	31
3.3.	Herramientas de apoyo al desarrollo del transceptor	31
3.3.1.	Generación de transceptores con TCL	31
3.3.2.	Generación de proyecto con TCL	31
3.4.	Diseño del driver serie en RTEMS (PS)	31
3.4.1.	API pública	32
3.4.2.	Interfaz con el hardware	32
3.4.3.	Gestión interna de la transmisión	34
3.4.4.	Gestión interna de la recepción	34
3.5.	Diseño hardware	35
3.5.1.	Selección de componentes bajo la restricción de 1,8 V	35
3.5.2.	Reglas de diseño y proceso de fabricación	36
3.5.3.	Diseño de la placa de comunicación serie	36
3.5.4.	Diseño de la placa CDHS	39
3.5.5.	Diseño de la placa AOCS	44
3.5.6.	Fabricación	48
3.6.	Desarrollo de herramientas de testing	48
3.6.1.	Aplicación de testing del driver serie en RTEMS	51
3.6.2.	Aplicación de testing de las placas CDHS y AOCS	51
3.6.3.	Arneses y equipamiento de validación	52
3.7.	Arquitecturas de transporte PS-PL	52
3.7.1.	Variante A — AXI GPIO (referencia)	53
3.7.2.	Variante B — 14× AXI DMA	54
3.7.3.	Variante C — AXI MCDMA con puente VHDL	55
3.7.4.	Comparación y elección de la arquitectura final	59
3.7.5.	Verificación y depuración iterativa del driver	60
3.7.6.	Banco de pruebas de <i>loopback</i> en la PL	62
4.	Resultados	64
4.1.	Validación CDHS	64
4.1.1.	Montaje	64
4.1.2.	Comunicaciones serie RS422/RS485	64
4.1.3.	Bus CAN	66
4.1.4.	ADC SPI (termistores)	67
4.1.5.	PWM (calentadores)	67
4.2.	Validación AOCS	68
4.2.1.	Montaje	68
4.2.2.	Comunicaciones serie RS422/RS485	68
4.2.3.	PWM (puentes en H)	70
4.3.	Validación de la placa de comunicación serie	70
4.3.1.	Montaje y topología de buses	71
4.3.2.	Batería de pruebas de caracterización	71
4.3.3.	Efecto del limitador de <i>slew rate</i> sobre la velocidad máxima	72
4.3.4.	Comportamiento del bus multipunto	73
4.3.5.	Integridad, aislamiento y estabilidad	74
4.3.6.	Limitaciones del método de medida	75
4.4.	Comparativa de las arquitecturas de transporte PS-PL	75
4.4.1.	Condiciones de la campaña	75
4.4.2.	Coste de hardware	76

4.4.3. Coste de software	77
4.4.4. Coste en interrupciones	77
4.4.5. Latencia	78
4.4.6. <i>Throughput</i>	79
4.4.7. Barrido de velocidad	80
4.4.8. Robustez	81
4.4.9. Selección de la arquitectura de transporte	81
5. Conclusiones y líneas futuras	83
5.1. Conclusiones	83
5.2. Líneas futuras	85
Bibliografía	87
A. Aspectos éticos, económicos, sociales y ambientales	89
B. Presupuesto económico	93
C. Mapas de señales, tabla del NCO, volumen de código y detalle RTL	95

Índice de figuras

1.1. Ciclo de <i>sprint</i> seguido en el trabajo.	3
2.1. Tarjeta de evaluación ZCU102 [1].	7
2.2. Reparto PS-PL del MPSoC y entorno de la ZCU102.	8
2.3. Topologías RS485 y RS422.	11
2.4. Codificación <i>Data-Strobe</i> y recuperación del reloj.	12
2.5. Canales de una transacción AXI4-Lite.	13
2.6. Canales de una transacción AXI4 <i>full</i>	14
2.7. Único canal de un enlace AXI4-Stream.	14
2.8. Transferencia por acceso directo a memoria.	15
2.9. Transferencia multicanal con AXI MCDMA.	16
3.1. Cronograma de una trama completa en configuración 8E1, LSB <i>first</i>	26
3.2. Diagrama de bloques de un canal del transceptor serie configurable.	28
3.3. Camino de un byte entre la lógica programable y la aplicación.	34
3.4. Hoja de nivel superior del esquemático de la placa de comunicación serie.	38
3.5. Render 3D de la placa de comunicación serie.	39
3.6. Diagrama de bloques de la placa CDHS.	40
3.7. Render 3D de la placa CDHS.	40
3.8. Subsistema CAN de la placa CDHS.	42
3.9. Canal RS de la placa CDHS con el THVD1424RGTR.	43
3.10. Hoja de nivel superior del esquemático de la placa AOCS.	45
3.11. Render 3D de la revisión V2 de la placa AOCS.	45
3.12. Subsistema SpaceWire de la placa AOCS.	47
3.13. Placa CDHS tras el proceso de soldadura.	49
3.14. Placa AOCS tras el proceso de soldadura.	50
3.15. Placa de comunicación serie tras el proceso de soldadura.	50
3.16. Arnés de <i>loopback</i> sobre conector D-Sub-9.	52
3.17. Arquitectura de la variante MCDMA.	56
3.18. Multiplexado de los catorce canales por TDEST.	58
4.1. Sistema completo montado: ZCU102 con la placa CDHS.	65
4.2. Flancos de la línea RS485 de la placa CDHS con y sin control de <i>slew rate</i>	66
4.3. Señal diferencial CAN con y sin resistencias de terminación.	67
4.4. Señal PWM en el conector J5 de la placa CDHS (1 kHz, 50 %).	68
4.5. Placas AOCS con componentes montados.	69
4.6. Señales PWM de control de motor en la placa AOCS.	70
4.7. PCB de comunicación serie en la campaña de caracterización.	71
4.8. Velocidad máxima sin errores en cada bus según el valor del bit SLO.	73
4.9. Tasa de error del bus multipunto frente a la velocidad, con y sin limitador de <i>slew</i>	73
4.10. Ocupación de la FPGA por variante.	77
4.11. Interrupciones por kilobyte transferido, por variante.	78

4.12. Latencia media de entrega de un <i>frame</i> de 11 bytes.	79
4.13. <i>Throughput</i> de un canal frente al techo físico de la línea.	79
4.14. <i>Throughput</i> frente a la velocidad de línea, en escala logarítmica.	80
C.1. <i>Block Design</i> del ejemplo de la puerta AND.	104
C.2. Generador de imágenes de arranque de Vitis.	106

Índice de tablas

3.1. Puertos del transmisor TX_CONFIGURABLE_SERIAL.	22
3.2. Puertos del receptor RX_CONFIGURABLE_SERIAL.	24
3.3. Reparto de bits de los vectores de interfaz	29
3.4. Mapa de bits de los dos canales del AXI GPIO de una instancia.	33
3.5. Control complementario DE/RE en el THVD1424.	43
3.6. Las tres arquitecturas de transporte implementadas y comparadas.	53
3.7. Mapa de registros del bloque TX_ISR_EOF_HANDLER (base 0xA0001000).	57
3.8. Mapa de memoria de la variante MCDMA.	58
3.9. Estructura del descriptor de <i>buffer</i> (BD) del MCDMA.	59
3.10. Coste de las tres variantes de transporte.	59
3.11. Topología de buses reproducida por el módulo de <i>loopback</i> de la PL.	62
4.1. Topología de buses de la PCB de comunicación serie.	71
4.2. Resultado de los nueve tests de caracterización de la PCB.	72
4.3. Integridad del bus multipunto (T3) frente a la velocidad, con SLO = 0.	74
4.4. Tasa de error en régimen permanente (T9), 10 s de tráfico continuo por bus.	75
4.5. Ocupación de la lógica programable de las tres variantes sobre el XCZU9EG.	76
4.6. Huella de software del driver de cada variante.	77
4.7. Interrupciones por kilobyte transferido.	78
4.8. Latencia de entrega de un <i>frame</i> de 11 bytes a 115 200 8N1.	78
4.9. <i>Throughput</i> en B/s por variante.	79
4.10. <i>Throughput</i> (B/s) frente a la velocidad de línea (baudios).	80
4.11. Rechazo de <i>frames</i> corruptos (T7) y desbordamiento del <i>ring</i> (T5).	81
4.12. Síntesis de la comparativa.	81
B.1. Presupuesto económico.	94
C.1. Mapa de señales entre la placa LINCE Comunicación Serial y el conector FMC HPC0 (P2) de la ZCU102.	97
C.2. Mapa de señales entre la placa CDHS y el conector FMC HPC0 de la ZCU102.	97
C.3. Mapa de señales entre la revisión V2 de la placa AOCS y el conector FMC HPC0 de la ZCU102.	98
C.4. Velocidades del NCO verificadas en simulación.	100
C.5. Distribución de las líneas de código desarrolladas en el trabajo.	101
C.6. Posiciones del selector de modo de arranque SW6 de la ZCU102 [1].	107

Lista de acrónimos y glosario

Los términos ingleses sin traducción asentada en español se escriben en cursiva; los ya incorporados al uso técnico habitual (*hardware*, *software*, *driver*, *firmware*) se dejan en redonda. Algunas entradas de esta lista no son siglas, sino términos que conviene precisar; se recogen aquí por comodidad de consulta.

AOCS *Attitude and Orbit Control Subsystem* — Subsistema de control de actitud y órbita.

ADC *Analog-to-Digital Converter* — Convertidor analógico-digital.

AMBA *Advanced Microcontroller Bus Architecture* — Familia de estándares de interconexión de ARM.

API *Application Programming Interface* — Interfaz de programación que un módulo expone a quien lo usa.

ARM *Advanced RISC Machines* — Arquitectura de procesador de los núcleos de aplicación del MPSoC.

AXI *Advanced eXtensible Interface* — Bus de interconexión de alto rendimiento del estándar ARM AMBA.

BD *Buffer Descriptor* — Descriptor de *buffer*: estructura en memoria que define una transferencia de DMA en modo *scatter-gather*.

BER *Bit Error Rate* — Tasa de error de bit de un enlace.

BIF *Boot Image Format* — Fichero de descripción con el que Vitis empaqueta los componentes de la imagen de arranque.

BRAM *Block RAM* — Bloque de memoria RAM integrado en la FPGA.

BSP *Board Support Package* — Paquete de soporte de placa.

BOM *Bill of Materials* — Lista de materiales de una PCB.

CAN *Controller Area Network* — Bus de comunicaciones serie multipunto.

CDHS *Command and Data Handling System* — Subsistema de mando y gestión de datos.

CMRR *Common Mode Rejection Ratio* — Razón de rechazo al modo común.

COTS *Commercial Off-The-Shelf* — Componente comercial de catálogo, no cualificado específicamente para espacio.

CRC *Cyclic Redundancy Check* — Código de redundancia cíclica para la detección de errores de transmisión.

CPU *Central Processing Unit* — Unidad central de proceso.

DDR *Double Data Rate* — Memoria dinámica principal del sistema de procesamiento.

DMA *Direct Memory Access* — Acceso directo a memoria sin intervención de la CPU.

DSP *Digital Signal Processor* — Bloque de procesamiento digital de señal.

DTB *Device Tree Binary* — Árbol de dispositivos en formato binario para sistemas Linux/U-Boot.

EMC *Electromagnetic Compatibility* — Compatibilidad electromagnética.

EMI *Electromagnetic Interference* — Interferencia electromagnética.

ESD *Electrostatic Discharge* — Descarga electrostática.

ESA *European Space Agency* — Agencia Espacial Europea.

ETSIT Escuela Técnica Superior de Ingenieros de Telecomunicación.

FIFO *First In, First Out* — Estructura de datos de cola.

FMC *FPGA Mezzanine Card* — Tarjeta de expansión para FPGA según estándar VITA 57.1.

FPGA *Field Programmable Gate Array* — Matriz lógica programable en campo.

FSM *Finite State Machine* — Máquina de estados finitos.

FSBL *First Stage Boot Loader* — Cargador de arranque de primera etapa.

GIC *Generic Interrupt Controller* — Controlador de interrupciones del procesador ARM.

- GPIO** *General Purpose Input/Output* — Puerto de entrada/salida de propósito general.
- GUI** *Graphical User Interface* — Interfaz gráfica de usuario.
- HDL** *Hardware Description Language* — Lenguaje de descripción de hardware.
- HP** *High Performance* — Puertos AXI de alto rendimiento entre el sistema de procesamiento y la lógica programable del MPSoC.
- HPC** *High Pin Count* — Variante del conector FMC con el mayor número de pines.
- IDE** *Integrated Development Environment* — Entorno integrado de desarrollo.
- INTC** *Interrupt Controller* — Controlador de interrupciones (IP de la lógica programable).
- IRQ** *Interrupt Request* — Petición de interrupción.
- ISR** *Interrupt Service Routine* — Rutina de servicio de interrupción.
- LDO** *Low-Dropout Regulator* — Regulador lineal de baja caída de tensión.
- LEO** *Low Earth Orbit* — Órbita terrestre baja.
- LINCE** *Línea de INdustrialización de Cargas de pago y plataformas Espaciales*.
- LVDS** *Low-Voltage Differential Signaling* — Señalización diferencial de bajo voltaje.
- LUT** *Look-Up Table* — Tabla de búsqueda, bloque básico de una FPGA.
- LSB** *Least Significant Bit* — Bit menos significativo; en el transceptor, uno de los dos órdenes de transmisión.
- MCDMA** *Multichannel Direct Memory Access* — Controlador de DMA multicanal de AMD (IP `axi_mcdma`, PG288).
- MMIO** *Memory-Mapped I/O* — Entrada/salida con mapeado en memoria.
- MM2S** *Memory-Mapped to Stream* — Canal de un DMA que lee de memoria y produce un flujo AXI-Stream (camino de transmisión).
- MMU** *Memory Management Unit* — Unidad de gestión de memoria; define, entre otras cosas, qué regiones son cacheables.
- MPSoC** *Multiprocessor System-on-Chip* — Sistema multiprocesador en chip.

MSB *Most Significant Bit* — Bit más significativo.

NCO *Numerically Controlled Oscillator* — Oscilador controlado numéricamente.

OBC *On-Board Computer* — Ordenador de a bordo.

PCB *Printed Circuit Board* — Placa de circuito impreso.

PERTE Proyecto Estratégico para la Recuperación y Transformación Económica — Instrumento de financiación pública del Gobierno de España.

PL *Programmable Logic* — Lógica programable (FPGA del MPSoC).

PMUFW *Power Management Unit Firmware* — Firmware de la unidad de gestión de potencia.

PS *Processing System* — Sistema de procesamiento (ARM del MPSoC).

PWM *Pulse Width Modulation* — Modulación por ancho de pulso.

PYMES Pequeñas y medianas empresas.

QEMU *Quick Emulator* — Emulador de máquina que ejecuta software sin el hardware físico.

RSB *RTEMS Source Builder* — Herramienta que compila el *toolchain* y los BSP de RTEMS desde código fuente.

RTEMS *Real-Time Executive for Multiprocessor Systems* — RTOS de código abierto para sistemas empuetrados.

RTL *Register Transfer Level* — Nivel de descripción del hardware mediante transferencias entre registros.

RTOS *Real-Time Operating System* — Sistema operativo de tiempo real.

S2MM *Stream to Memory-Mapped* — Canal de un DMA que consume un flujo AXI-Stream y lo escribe en memoria (camino de recepción).

SAR *Successive Approximation Register* — Arquitectura de convertidor analógico-digital por aproximaciones sucesivas.

SEU *Single Event Upset* — Cambio de estado de un biestable o de una celda de memoria provocado por el impacto de una partícula.

SG *Scatter-Gather* — Modo de operación de un DMA en el que las transferencias se describen mediante una lista enlazada de descriptores en memoria.

SLO Pin de los transceptores LTC2865 que habilita el limitador de velocidad de flanco, activo a nivel bajo.

SMD *Surface Mount Device* — Componente de montaje superficial.

SMP *Symmetric Multiprocessing* — Multiprocesamiento simétrico sobre varios núcleos.

SPI *Serial Peripheral Interface* — Interfaz serie para periféricos.

SPW Abreviatura de SpaceWire empleada en los mapas de señales y los ficheros de restricciones.

TCL *Tool Command Language* — Lenguaje de scripting utilizado en Vivado.

TDEST *Transfer Destination* — Señal del canal AXI-Stream que identifica el canal de destino de cada paquete.

TFM Trabajo Fin de Máster.

TLAST *Transfer Last* — Señal del canal AXI-Stream que marca el último byte de un paquete.

TMR *Triple Modular Redundancy* — Triplicación de la lógica con votadores para tolerar los fallos provocados por la radiación.

TREADY *Transfer Ready* — Señal del canal AXI-Stream con la que el receptor indica que puede aceptar un dato.

TVALID *Transfer Valid* — Señal del canal AXI-Stream con la que el emisor indica que el dato presentado es válido.

TVS *Transient Voltage Suppressor* — Supresor de sobretensiones transitorias.

UART *Universal Asynchronous Receiver-Transmitter* — Transceptor serie asíncrono universal.

UPM Universidad Politécnica de Madrid.

USB *Universal Serial Bus* — Bus serie empleado aquí para la consola de la ZCU102.

VADJ Carril de alimentación ajustable con el que la ZCU102 alimenta los bancos de entrada/salida del conector FMC; en este trabajo, a 1,8 V.

VHDL *Very High Speed Integrated Circuit Hardware Description Language* — Lenguaje de descripción de hardware.

WNS *Worst Negative Slack* — Peor margen de temporización de un diseño implementado; positivo indica que se cumplen las restricciones.

XSA *Xilinx Support Archive* — Archivo de especificación hardware exportado por Vivado para Vitis.

1. Introducción y objetivos

1.1. Introducción y motivación

Este trabajo se enmarca en el proyecto **LINCE** (*Línea de INdustrialización de Cargas de pago y plataformas Espaciales*), una iniciativa del PERTE Aeroespacial liderada por Indra. El proyecto impulsa la producción en serie de plataformas de microsátélites (100–200 kg) y de sus cargas de pago para órbita baja (LEO), con el objetivo de reducir costes y plazos y de consolidar la autonomía estratégica española en este segmento. Su constelación demostradora, STARTICAL, validará en vuelo tecnologías críticas de comunicaciones y vigilancia [2].

Dentro del consorcio, Sener participa como socio tecnológico: desarrolla y valida el firmware del OBC, y traslada a los demás participantes los requisitos técnicos que recibe de Indra, líder del consorcio. La Universidad Politécnica de Madrid participa a través del grupo B105 Electronic Systems Lab, responsable del subsistema de comunicaciones del ordenador de a bordo (OBC); este trabajo se desarrolla dentro de esa responsabilidad. El proyecto sigue el enfoque **NewSpace**: uso de componentes comerciales (COTS, *commercial off-the-shelf*) frente al uso tradicional de componentes resistentes a la radiación, que son mucho más caros. Los efectos de la radiación se mitigan mediante estrategias como la redundancia.

El OBC concentra el control del satélite y se comunica con los periféricos de la plataforma (propulsores, cámaras, seguidores de estrellas, sensores térmicos) mediante enlaces serie. Los requisitos de LINCE fijan catorce enlaces RS422/RS485 operando de forma simultánea, cada uno con sus propios parámetros de línea. Esa cifra obliga a estudiar el transporte de datos desde cada transceptor hasta la aplicación de usuario. El reparto de recursos debe minimizar el área ocupada en la lógica programable (PL) y el uso de CPU del sistema de procesamiento (PS), sin penalizar por ello el rendimiento del conjunto. La ZCU102, plataforma de desarrollo basada en el MPSoC Zynq UltraScale+ escogida para el prototipado, no dispone ni de tantos transceptores físicos serie ni de los conectores de los periféricos del satélite, de modo que el propio banco de ensayo forma parte del problema a resolver.

El autor se incorporó al laboratorio B105 Electronic Systems Lab como becario para trabajar en el proyecto LINCE debido a su interés en proyectos espaciales y satelitales. La aportación inicial se centró en el desarrollo de firmware en VHDL y C para las comunicaciones serie RS422 y RS485 sobre la ZCU102. La necesidad de ejercitar ese firmware en condiciones eléctricas reales amplió después el alcance del trabajo al diseño y fabricación del hardware que completa el banco de ensayo.

1.2. Objetivos

El objetivo principal de este trabajo es desarrollar una plataforma funcional y validada de comunicación con periféricos serie sobre el MPSoC Zynq UltraScale+ ZCU102, que sirva como base para el OBC del proyecto LINCE. Para alcanzarlo, se plantearon los siguientes objetivos específicos:

1. **Diseñar un módulo VHDL de transceptor serie configurable** para la lógica programable del MPSoC, capaz de operar sobre los estándares RS422 y RS485, con parámetros configurables en tiempo de ejecución: baudrate, paridad, número de bits de datos, bits de parada, orden de bit y limitación del *slew rate* del transceptor físico. El número total de transceptores serie que deben operar a la vez es de 14, por los requisitos del proyecto LINCE.
2. **Diseñar la arquitectura de transporte de datos entre la lógica programable y el sistema de procesamiento.** El transceptor resuelve la capa física de un enlace, pero llevar los flujos de los catorce enlaces instanciados hasta el procesador es un problema distinto, y su solución condiciona el rendimiento del conjunto. Se plantea investigar las alternativas disponibles, desarrollarlas, compararlas y seleccionar la que mejor escale en rendimiento, en ocupación de la lógica programable y en consumo de recursos de CPU.
3. **Desarrollar un driver de software completo en C para RTEMS 7** que abstraiga el acceso al hardware y permita a la aplicación gestionar los transceptores serie, incluyendo la configuración de todos los parámetros exponiendo una API pública sencilla de utilizar.
4. **Diseñar y fabricar una placa de comunicación serie** que permita ejercitar los catorce transceptores de forma simultánea y sobre topologías de bus realistas, como puede ser punto a punto o multipunto compartido.
5. **Diseñar y fabricar dos placas de circuito impreso de expansión de la ZCU102** que permitan la conexión con diversos periféricos, bajo las especificaciones de los subsistemas CDHS y AOCS del satélite, incluyendo interfaces RS422/RS485, SpaceWire, CAN, PWM y medida de señales analógicas.
6. **Integrar y validar el sistema completo**, verificando la comunicación entre la ZCU102 y las tres placas, y comprobando que cada placa cumple con su función de proporcionar una capa física con buses compartidos o de exponer las conexiones a los distintos periféricos sin errores.

1.3. Metodología

Durante el desarrollo del trabajo se ha tenido un contacto estrecho con los compañeros de Sener. Se han seguido **metodologías ágiles** mediante la plataforma *Jira* [3], que permite organizar las tareas de forma visual utilizando un **tablero tipo Kanban** [4]. La metodología ágil utilizada consiste en establecer las tareas que se van a realizar durante un tiempo establecido, denominado *sprint*. Al final de cada *sprint* se celebra una reunión estructurada en cuatro fases: revisión del *sprint* cerrado, preparación del siguiente, análisis de posibles mejoras sobre la metodología y preguntas finales. Los *sprints* han tenido una duración de dos semanas y se han realizado 20 en total.

La figura 1.1 recoge el ciclo, desde la selección de tareas del *backlog* hasta la reunión que lo cierra y realimenta el siguiente.

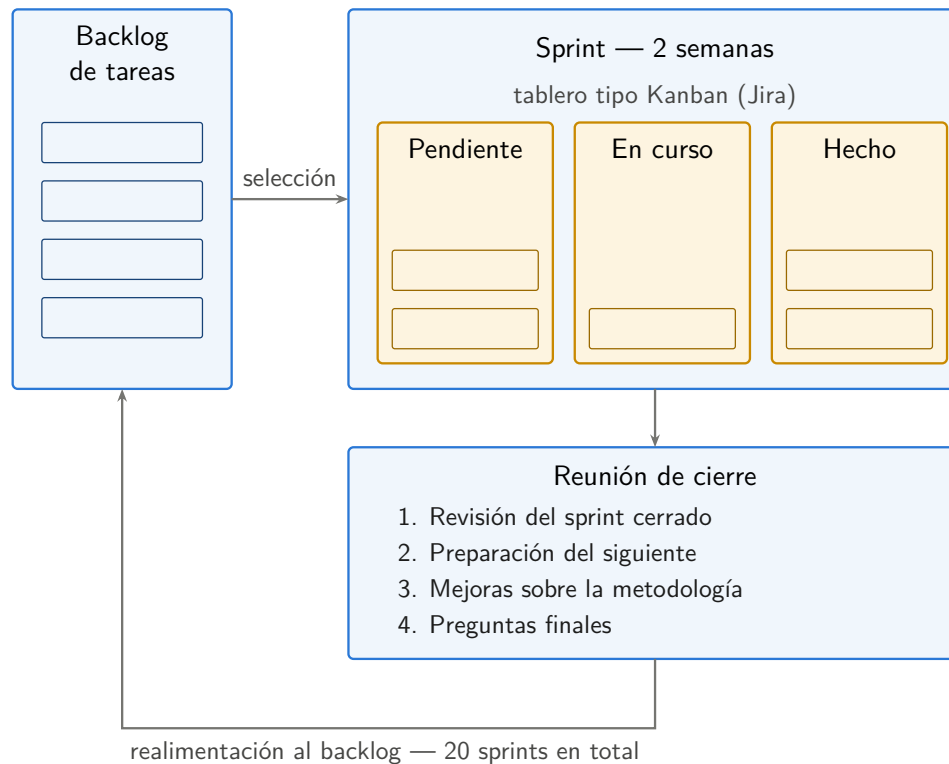


Figura 1.1: Ciclo de *sprint* seguido en el trabajo.

Además de las reuniones de *sprint*, se ha mantenido una comunicación constante con el equipo de Sener para resolver todo tipo de dudas acerca de los requerimientos del proyecto y de cómo orientar el desarrollo.

El flujo de trabajo se apoyó en automatizaciones propias en forma de scripts de shell, Python y TCL, que redujeron el tiempo dedicado a tareas repetitivas y eliminaron los errores manuales.

1.3.1. Planificación temporal

El trabajo se estructuró en tres fases principales:

Fase 1: Familiarización y establecimiento del entorno (octubre – enero): Los primeros meses se dedicaron a establecer las bases del entorno de desarrollo: instalación y configuración de Vivado y Vitis, compilación de RTEMS 7 desde código fuente para la arquitectura AArch64 mediante el RTEMS Source Builder, y validación del flujo completo (síntesis en Vivado → empaquetado en Vitis → ejecución en RTEMS sobre la ZCU102) con un ejemplo funcional de extremo a extremo. Esta fase fue fundamental para comprender la arquitectura PS-PL del MPSoC y las particularidades del entorno de compilación cruzada para sistemas empujados.

Fase 2: Desarrollo e integración (febrero – junio): Con el entorno establecido, se abordó el desarrollo principal: el transceptor serie en VHDL, el driver para RTEMS, el diseño de las PCB de comunicación serie, CDHS y AOCS, y las aplicaciones de prueba.

Se siguió una metodología iterativa: cada módulo se implementaba, se validaba de forma aislada y se integraba en el sistema global antes de avanzar al siguiente.

En el firmware (VHDL y C), cada bloque se comprobaba mediante simulaciones de *test-bench* antes de integrarlo en el sistema completo. Además, se realizaron simulaciones del sistema completo, que permitieron encontrar y corregir varios errores antes de llevar el diseño a la placa.

En el hardware se asumió desde el principio la detección de errores de diseño como parte del proceso: cada placa se caracterizaba en el laboratorio y, cuando el calendario lo permitió, se corrigió y se volvió a fabricar. Quedó fuera del alcance la validación de la comunicación SpaceWire presente en la PCB AOCS, por su alta complejidad técnica.

Fase 3: Estudio del transporte y caracterización (junio – julio): Se caracterizó el firmware desarrollado (VHDL y C) y el hardware (PCBs) mediante un conjunto de pruebas, las cuales revelaron que la aproximación inicial del sistema de transporte de datos no era eficiente. Se desarrollaron dos alternativas nuevas: la primera consistió en un sistema de transporte basado en un controlador AXI DMA independiente por canal, y la segunda en un sistema de transporte basado en un único controlador AXI MCDMA con un puente VHDL que multiplexa todos los canales. Ambas alternativas fueron implementadas y comparadas.

1.4. Estructura de la memoria y documentación

Todo el trabajo desarrollado (código VHDL, aplicaciones y drivers de RTEMS, bancos de pruebas, diseños de las placas y scripts de automatización) se ha versionado en un único repositorio Git, disponible públicamente en GitHub [5]. Bajo el directorio `tfm/` se organiza por bloques temáticos: `01_ip_serie` (transceptor serie en VHDL), `02_transporte` (las tres arquitecturas de transporte), `03_bench_loopback` y `04_pcb_caracterizacion` (bancos de prueba y caracterización), `05_aplicaciones_cdhs_aocs` (aplicaciones sobre las placas CDHS y AOCS), `06_firmware` (componentes y scripts de arranque), `07_tools` (herramientas de desarrollo comunes) y `00_docs` (documentación, esquemáticos y BOM). A lo largo de la memoria se hace referencia a rutas concretas dentro de este repositorio, siempre relativas a su raíz.

En cuanto a la tipografía, los términos ingleses sin traducción asentada en español se escriben en cursiva (*buffer*, *stream*, *frame*, *slew rate*); los ya incorporados al uso técnico habitual (hardware, software, driver, firmware) se dejan en redonda.

El resto del documento se organiza en cuatro capítulos y tres anexos.

El **capítulo 2** recoge el marco teórico: el proyecto LINCE, el MPSoC Zynq UltraScale+ y la plataforma ZCU102, el entorno de herramientas Vivado y Vitis, el sistema operativo de tiempo real RTEMS, los estándares de comunicación empleados (RS422, RS485, SpaceWire y CAN) y los mecanismos de transporte de datos entre el sistema de procesamiento y la lógica programable, con el bus AXI y el acceso directo a memoria.

El **capítulo 3** describe el desarrollo: la preparación del entorno y del flujo de compilación, el diseño del transceptor serie configurable en VHDL, el driver para RTEMS 7, las tres arquitecturas de transporte implementadas, el diseño y fabricación de las tres placas de circuito impreso y las herramientas de prueba desarrolladas.

El **capítulo 4** presenta los resultados: la validación de la placa de comunicación serie con los catorce transceptores en funcionamiento simultáneo, la comparativa cuantitativa entre las tres arquitecturas de transporte, la caracterización eléctrica de la placa de comunicación serie y la validación de las placas CDHS y AOCS sobre cada una de sus interfaces.

El **capítulo 5** reúne las conclusiones y las líneas futuras de trabajo. El **anexo A** recoge el análisis de aspectos éticos, económicos, sociales y ambientales; el **anexo B**, el presupuesto económico del trabajo; y el **anexo C**, los mapas de señales de las tres placas, la tabla del oscilador controlado numéricamente, el recuento del código desarrollado y el detalle a nivel de transferencias entre registros (RTL, *Register Transfer Level*) de los bloques auxiliares del transceptor.

2. Marco teórico

2.1. Proyecto LINCE

El trabajo que recoge esta memoria se enmarca aquí: en el subsistema de comunicaciones del ordenador de a bordo (OBC), del que la UPM es responsable dentro del consorcio.

El Proyecto LINCE (*Línea de INdustrialización de Cargas de pago y plataformas Espaciales*) es una iniciativa estratégica enmarcada en el PERTE Aeroespacial, cuyo propósito es consolidar la capacidad industrial de España en el segmento de los microsátélites (100–200 kg) destinados a órbitas bajas (LEO). Liderado por Indra, el consorcio busca desarrollar plataformas satelitales modulares, fiables y de altas prestaciones, optimizando costes y plazos mediante procesos de producción en serie. Este ecosistema integra a grandes empresas, PYMES y centros de investigación, como la Universidad Politécnica de Madrid (UPM), para habilitar servicios avanzados de telecomunicaciones, vigilancia y observación, garantizando así la autonomía estratégica de España y Europa en el sector espacial [2].

2.2. MPSoC Zynq UltraScale+ ZCU102

El OBC del satélite se construye sobre este MPSoC, y la ZCU102 es la plataforma sobre la que se ha desarrollado y validado este trabajo. Es una tarjeta de evaluación de propósito general de Xilinx, hoy AMD (en adelante, AMD), para el prototipado de sistemas empotrados sobre el MPSoC (*Multiprocessor System-on-Chip*) Zynq UltraScale+. Este MPSoC integra en un único encapsulado un sistema de procesamiento con cuatro núcleos ARM (*Advanced RISC Machines*) y una matriz de lógica programable, unidos por buses internos de alto rendimiento. La arquitectura separa el sistema en dos dominios complementarios: el Sistema de Procesamiento (PS, *Processing System*) y la lógica programable (PL, *Programmable Logic*).

El **PS** es el dominio del software. Son núcleos ARM de arquitectura fija que ejecutan instrucciones de forma secuencial, igual que cualquier microprocesador clásico. Sobre ellos corren el sistema operativo, la gestión de la memoria DDR (*Double Data Rate*) y las rutinas de control de alto nivel.

La **PL** es el dominio del hardware a medida: una matriz reconfigurable FPGA (*Field Programmable Gate Array*) que no ejecuta un programa, sino que se configura físicamente. Con un lenguaje de descripción de hardware como VHDL (*Very High Speed Integrated Circuit Hardware Description Language*) se interconectan los recursos del silicio (tablas de búsqueda, biestables, memoria BRAM (*Block RAM*) y bloques DSP (*Digital Signal Processor*)) para formar circuitos digitales concretos. A cambio de un flujo de diseño más

costoso, la PL ofrece determinismo temporal estricto y paralelismo masivo.

La placa expone dos conectores FMC (*FPGA Mezzanine Card*) HPC (*High Pin Count*), conformes al estándar VITA 57.1 [6]. Sobre ellos se montan tarjetas de expansión (*mezzanines*) que llevan pines digitales y pares diferenciales de la FPGA hacia el exterior, que es la vía para conectar periféricos físicos a la PL. En la figura 2.1 están señalados, junto al resto de elementos de la tarjeta.

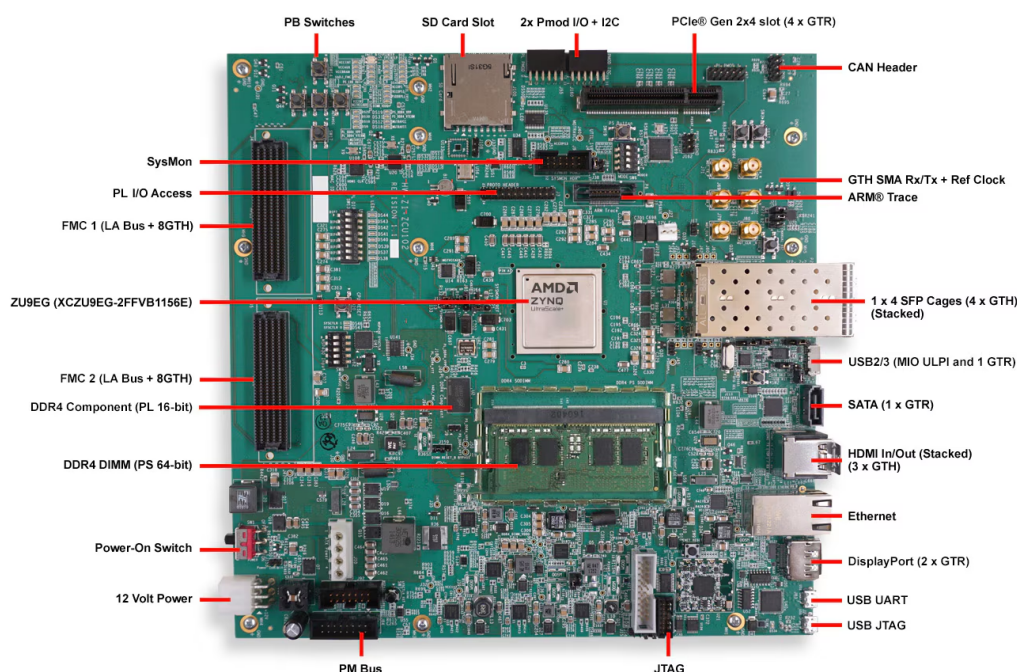


Figura 2.1: Tarjeta de evaluación ZCU102 [1].

La figura 2.2 recoge el reparto, con los puertos AXI entre PS y PL recuadrados: son el camino que recorren los datos de los catorce enlaces serie y el objeto de la comparativa del capítulo 4.

2.3. Vivado y Vitis (AMD): PS y PL

Estas dos herramientas son las que han producido todo lo desarrollado en este trabajo: el VHDL que ocupa la PL y el código C que corre en el PS. AMD distribuye dos entornos para el desarrollo sobre el Zynq UltraScale+: Vivado para la PL y Vitis para el PS.

2.3.1. Vivado Design Suite

En Vivado se diseñó e implementó toda la lógica que ocupa la PL, desde el transceptor serie hasta los puentes de transporte. Vivado es el entorno de diseño, síntesis e implementación de la lógica de la PL. Admite descripción en HDL (*Hardware Description Language*) y diseño por bloques (*Block Design*), donde los periféricos de diseño propio se conectan al PS mediante el bus AXI (*Advanced eXtensible Interface*). El fichero de restricciones (.xdc) asocia las señales del diseño a los pines físicos del encapsulado. El flujo termina con dos

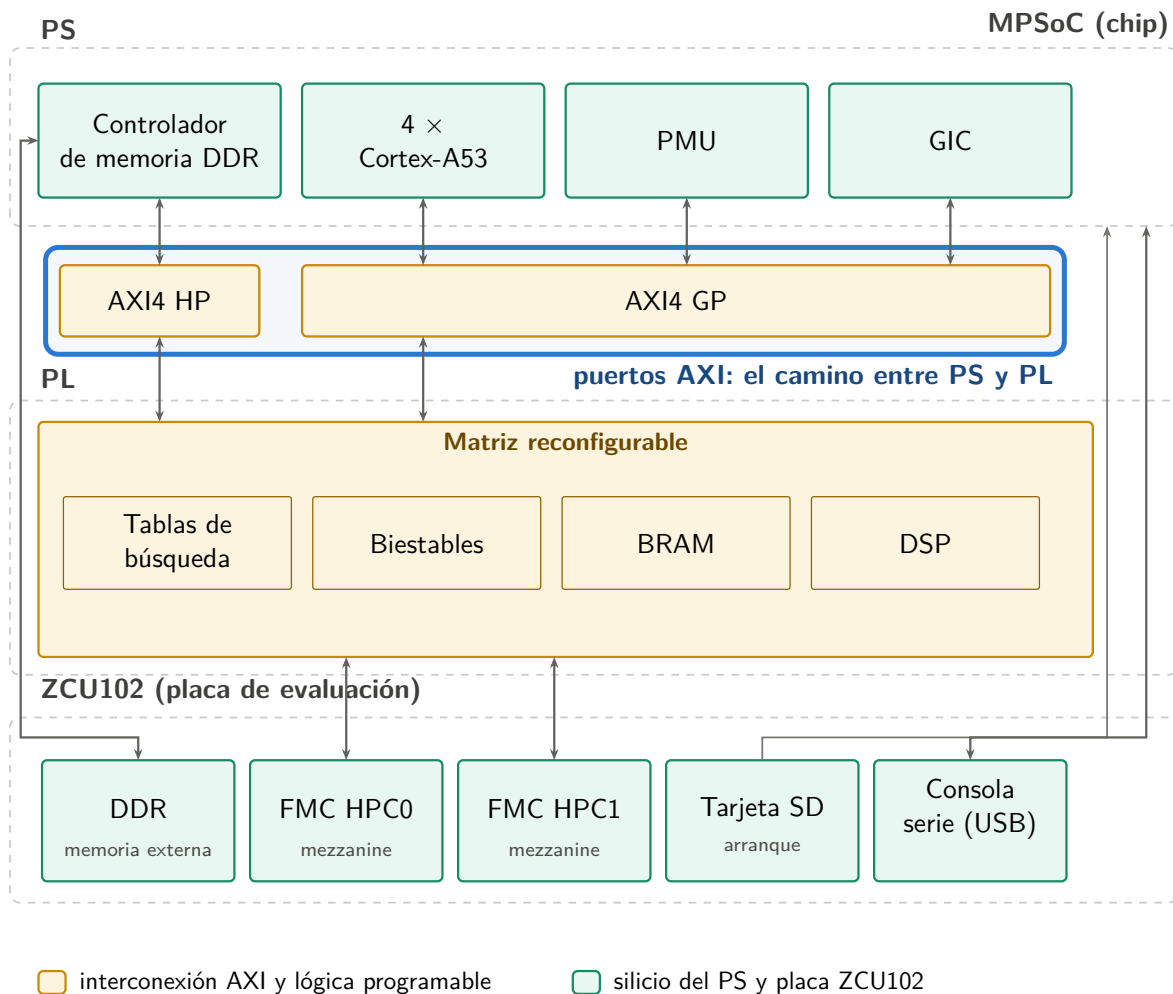


Figura 2.2: Reparto PS–PL del MPSoC y entorno de la ZCU102.

salidas: el *bitstream* que configura la FPGA y un fichero .xsa que describe la arquitectura hardware para las herramientas de software.

Los periféricos de diseño propio se integran con el procesador dentro de Vivado, que además dirige sus señales hasta los conectores FMC.

2.3.2. Vitis Unified Software Platform

En Vitis se construyó el software que corre en el PS, tanto el driver como las aplicaciones de prueba. Vitis es el IDE para el software del PS. Toma el fichero .xsa de Vivado y genera el paquete de soporte de la placa (BSP, *Board Support Package*): el mapa de memoria y los drivers de bajo nivel con los que el código en C accede a los periféricos de la PL. Aporta además el compilador cruzado para los núcleos ARM y el empaquetado del binario de arranque (BOOT.BIN), que reúne el firmware de inicialización (PMUFW, *Power Management Unit Firmware*, y FSBL, *First Stage Boot Loader*), el *bitstream* y el ejecutable en un solo fichero cargable desde una tarjeta SD.

Con Vitis se construyen, por tanto, tanto la capa de acceso a los periféricos como la imagen con la que la placa arranca de forma autónoma.

2.4. RTEMS

RTEMS es el sistema operativo bajo el que se ejecuta el software de vuelo, y por tanto el que tiene que dar servicio al driver desarrollado aquí. RTEMS (*Real-Time Executive for Multiprocessor Systems*) es un sistema operativo de tiempo real de código abierto para sistemas empotrados de misión crítica. No usa memoria virtual: opera en un único espacio de direcciones físico, con un solo proceso y varios hilos. Al no haber cambios de contexto pesados ni traducción de direcciones, la latencia de respuesta a una interrupción es baja y acotada.

Es un estándar de facto en la industria aeroespacial (ESA, *European Space Agency*, y NASA lo emplean en satélites y sondas) por su robustez y su soporte para arquitecturas ARM como la del Zynq UltraScale+. En este tipo de plataforma se ejecuta sobre el PS y se encarga de la lógica de control y del acceso a los registros de los periféricos de la PL. El acceso a un periférico de diseño propio no cuenta con driver previo en RTEMS: da control total sobre la lógica y sobre cada registro, a cambio de una curva de aprendizaje y una complejidad de desarrollo mayores que las de un periférico ya integrado.

2.5. Comunicaciones serie

El OBC de LINCE debe atender enlaces de estos cuatro tipos. Sus características eléctricas y sus formatos de trama son los que fijan lo que el transceptor tiene que soportar. En arquitecturas satelitales, las comunicaciones serie son el medio habitual de intercambio de datos entre el ordenador de a bordo (OBC) y sus periféricos: sensores, actuadores y cargas de pago. Frente al bus paralelo, el enlace serie reduce drásticamente el número de líneas físicas, y con ello el peso y el volumen del arnés. La señalización diferencial, común a la mayoría de estos estándares, cancela en el receptor el ruido electromagnético inducido por igual en los dos hilos.

Los cuatro estándares que se describen a continuación (RS485, RS422, SpaceWire y CAN) cubren ese papel con topologías y prestaciones distintas.

2.5.1. RS485

Es el estándar que hace posible el bus compartido de siete nodos de la placa de comunicación serie. El estándar TIA/EIA-485 (RS485) define las características eléctricas de transmisores y receptores para enlaces serie diferenciales. Su rasgo distintivo es la topología multipunto: admite hasta 32 transceptores estándar sobre el mismo par de hilos, más si son de carga fraccional¹.

La información viaja como diferencia de tensión entre los hilos A y B, lo que da un alto rechazo al modo común. Se suele usar en *half-duplex* sobre dos hilos, con transmisión y recepción compartiendo el medio; esto obliga a controlar por software el sentido del enlace (pines de *driver enable*) para evitar colisiones. Alcanza unos 1200 m a baja velocidad o hasta decenas de Mbps en distancias cortas, y necesita resistencias de terminación de $120\ \Omega$ en los extremos del bus para evitar reflexiones² [7, 8].

2.5.2. RS422

Sobre él se montan los dos buses *master/slave* de esa misma placa. El estándar TIA/EIA-422 (RS422) comparte con el RS485 la señalización diferencial, pero no la topología: está pensado para enlaces punto a punto o *multi-drop*³, con un único transmisor y hasta 10 receptores por par.

Los transceptores RS422 no ceden el control del bus, así que la comunicación bidireccional continua (*full-duplex*) exige cuatro hilos: un par para transmisión y otro para recepción. Al separar físicamente los dos sentidos, no hace falta arbitrar el bus por software, lo que reduce la carga de proceso y hace el enlace idóneo para el envío continuo de telemetría de sensores [9, 10].

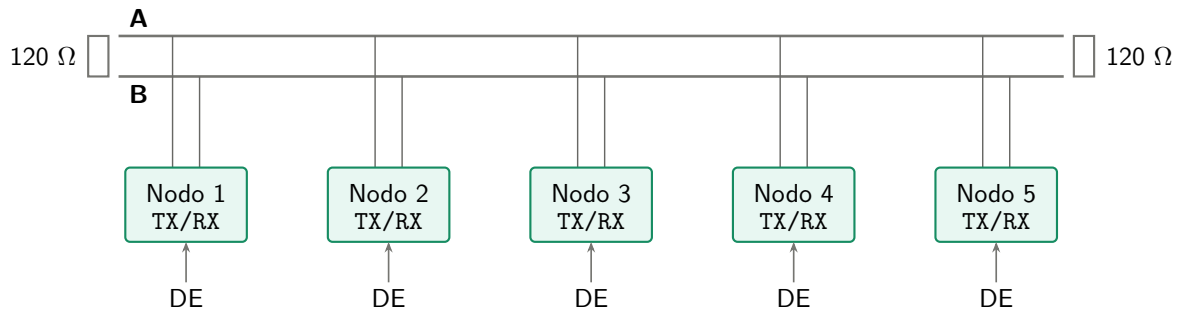
La figura 2.3 contrapone las dos topologías. En RS485 los nodos cuelgan del mismo par y solo uno transmite a la vez, de modo que hace falta controlar el acceso al bus y cederlo tras cada transmisión. En RS422 cada par tiene un único transmisor, y por eso el enlace no necesita arbitraje.

¹Receptores que cargan la línea con una fracción de la unidad estándar, lo que permite superar los 32 nodos.

²Ecos de la señal en un extremo mal adaptado de la línea, que se superponen a los bits siguientes.

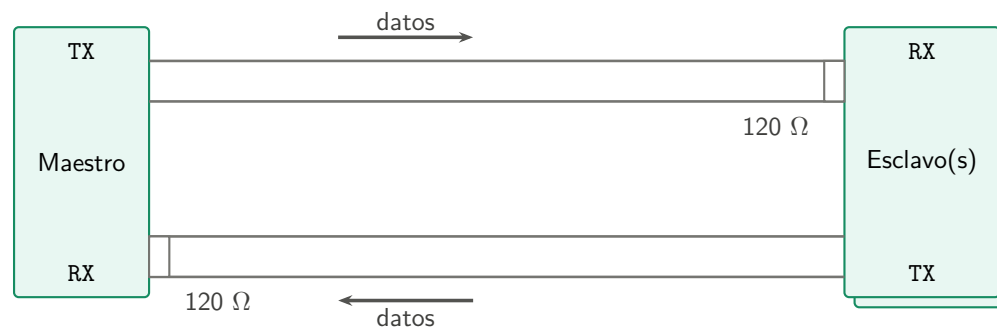
³Un transmisor y varios receptores sobre el mismo par; los receptores no transmiten por esa línea.

RS485 — bus multipunto compartido



Solo un nodo transmite a la vez; el resto permanecen a la escucha.

RS422 — punto a punto full-duplex



Un único transmisor por par: no hace falta compartir el medio.

Figura 2.3: Topologías RS485 y RS422.

2.5.3. SpaceWire

La placa AOCS expone dos enlaces SpaceWire, la única interfaz de las tres placas que quedó sin validar. SpaceWire es un estándar de red a bordo de alta velocidad coordinado por la ESA, usado para interconectar nodos de alto rendimiento como memorias masivas, procesadores de instrumento y enlaces de telemetría.

En el nivel físico usa señalización LVDS⁴ y una codificación *Data-Strobe*. Enviar el reloj por un par propio obliga a que ese par y el de datos lleguen alineados dentro de una fracción del tiempo de bit. A 200 Mbps el tiempo de bit es de 5 ns, de modo que un desfase de pocos nanosegundos entre ambos pares, acumulado por la diferencia de longitud de las pistas y por la dispersión del cable, basta para muestrear el bit equivocado.

La codificación *Data-Strobe* (DS) supera esta limitación empleando dos líneas diferenciales: *Data* transmite directamente el flujo de datos, mientras que *Strobe* conmuta únicamente cuando el bit a transmitir es idéntico al anterior (permaneciendo constante si el dato cambia). De este modo, se garantiza exactamente una transición por período de bit

⁴Low-Voltage Differential Signaling: par diferencial de unos 350 mV de excursión, para alta velocidad y bajo consumo.

entre ambas líneas. En recepción, el reloj se recupera de forma inmediata aplicando una operación XOR a ambas señales y el dato se muestrea directamente de la línea *Data*, lo que elimina la necesidad de un alineamiento estricto de reloj y aporta una alta tolerancia al desfase (*skew*) entre pares. La figura 2.4 recoge el mecanismo sobre un flujo de diez bits.

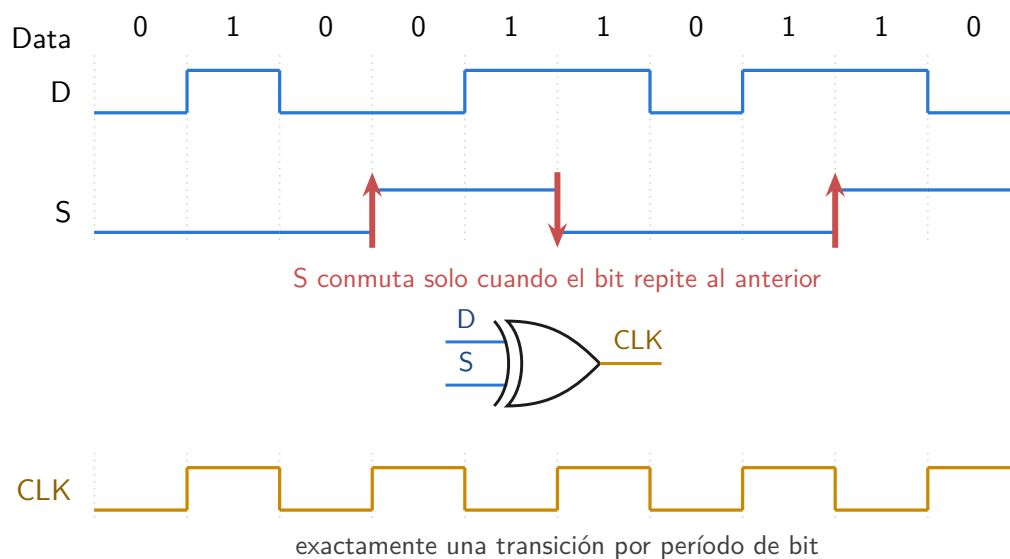


Figura 2.4: Codificación *Data-Strobe* y recuperación del reloj.

El estándar define enlaces punto a punto *full-duplex* asíncronos de hasta 400 Mbps, con una pila que incluye una capa de red para encaminar paquetes a través de *routers* SpaceWire [11, 12].

2.5.4. CAN

La placa CDHS lleva un bus CAN redundante, y su validación es uno de los resultados del capítulo 4. El bus CAN (*Controller Area Network*) nació en automoción y se usa como estándar de facto (*CAN for Aerospace*) en las redes internas de telemetría y telecomando (TM/TC) de satélites pequeños y comerciales.

Es multipunto y multimaestro, con acceso por escucha de portadora, es decir, cada nodo espera a que el bus esté en reposo antes de empezar a transmitir, y con arbitraje por prioridad de mensaje. Los nodos no tienen dirección: cada trama lleva un identificador que fija su contenido y su prioridad. Si dos nodos transmiten a la vez, la colisión se resuelve bit a bit y sin destruir datos: gana el identificador de mayor prioridad y el otro nodo reintenta. La capa de enlace añade detección de errores por hardware (CRC, *bit stuffing*⁵, comprobación de trama) y aislamiento automático de nodos defectuosos, de modo que un periférico averiado no bloquea el bus [13, 14].

⁵Inserción de un bit de polaridad contraria tras cinco bits iguales para que el receptor mantenga el sincronismo.

2.6. Transporte de datos entre el PS y la PL

Llevar los catorce enlaces serie hasta la aplicación es el problema que ocupa buena parte del trabajo. El capítulo 3 lo resuelve construyendo y midiendo tres arquitecturas distintas. El PS y la PL necesitan intercambiar flujos de bytes: los que un periférico de la PL recibe y hay que llevar a la memoria del procesador, y los que el procesador quiere transmitir. El MPSoC ofrece varias vías para ese intercambio de información, y la elección entre ellas no es un detalle de implementación: condiciona la carga de CPU (*Central Processing Unit*), la latencia y el área ocupada en la FPGA.

Esta sección describe los mecanismos disponibles: el bus que conecta ambos dominios, el acceso directo a memoria y el coste que impone el modelo de interrupciones.

2.6.1. Bus AXI

Cada una de esas tres arquitecturas se apoya en un perfil distinto del estándar. La comunicación entre el PS y la PL del Zynq UltraScale+ se realiza a través de puertos que implementan el estándar AMBA (*Advanced Microcontroller Bus Architecture*) AXI4 de ARM [15]. El estándar define tres perfiles, y todos se describen aquí sobre la misma rejilla de canales para poder contraponerlos.

AXI4-Lite. Perfil reducido para leer y escribir *registros* individuales. Usa los cinco canales de una transacción *memory-mapped* (dirección y datos de escritura, respuesta de escritura, y dirección y datos de lectura), pero cada canal mueve una sola palabra por transacción, y cada palabra consume una instrucción de carga o almacenamiento del procesador. Es el bus con el que se configuran los periféricos (figura 2.5).

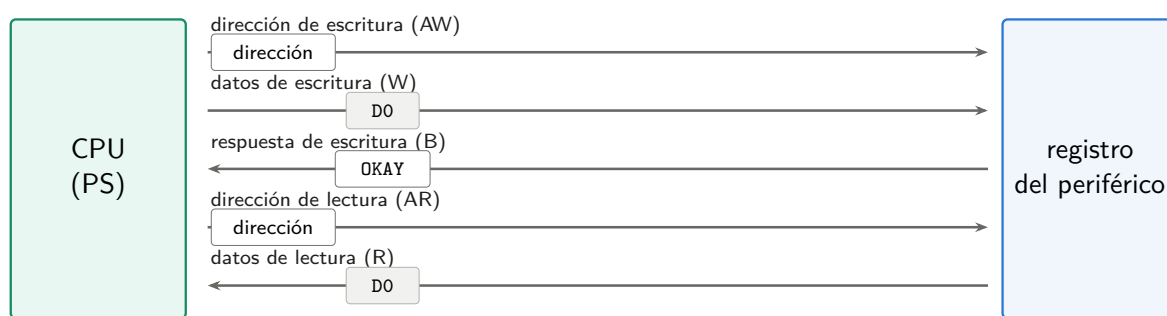


Figura 2.5: Canales de una transacción AXI4-Lite.

AXI4 full (*memory-mapped*). Perfil completo. Conserva esos mismos cinco canales, pero los de datos admiten ráfagas (*bursts*) de hasta 256 transferencias por cada dirección emitida. Es el bus que utilizan los maestros de la PL para acceder a la memoria DDR del PS a través de los puertos de alto rendimiento (HP), sin pasar por la CPU (figura 2.6).

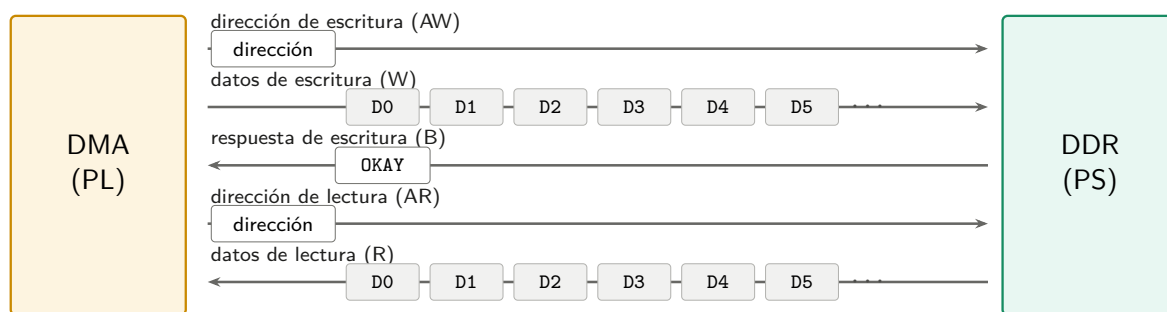


Figura 2.6: Canales de una transacción AXI4 *full*.

AXI4-Stream. Perfil sin direcciones: elimina los canales de dirección y de respuesta y deja un único canal de datos unidireccional. El flujo lo acompañan señales de *handshake* (TVALID/TREADY)⁶, una marca de fin de paquete (TLAST) y, opcionalmente, un identificador de destino (TDEST). Es el perfil idóneo para conectar un periférico que produce o consume bytes en secuencia, como un UART (figura 2.7).

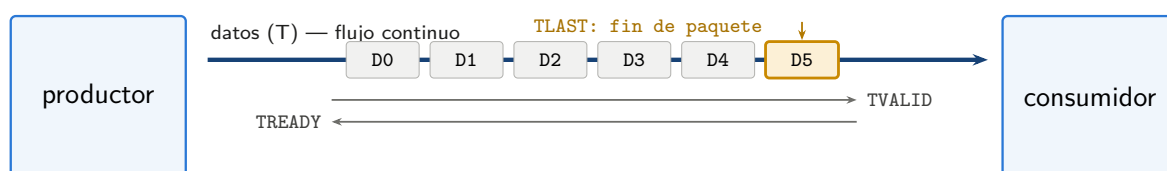


Figura 2.7: Único canal de un enlace AXI4-Stream.

Cuántas direcciones hay que mover por cada dato útil es lo que decide el coste en CPU de cada arquitectura del capítulo 3.

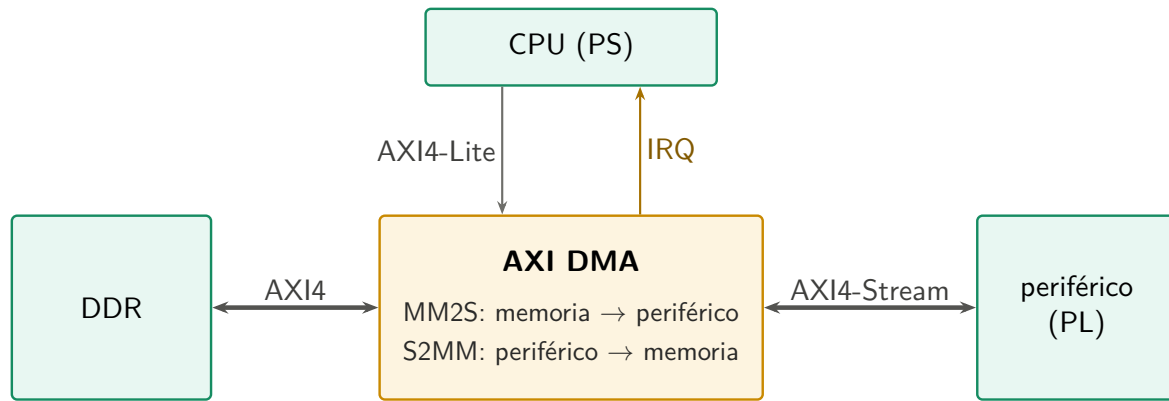
2.6.2. Acceso directo a memoria (DMA)

Las dos arquitecturas que superan a la de partida se construyen sobre este mecanismo, cada una con una IP de DMA distinta. Un controlador DMA es un maestro de bus que transfiere bloques de datos entre la memoria y un periférico **sin intervención de la CPU**. El procesador se limita a programar la transferencia (dirección de origen o destino y longitud) y a recibir una única interrupción cuando el bloque completo ha terminado. La tasa de interrupción deja de depender del número de bytes y pasa a depender del número de *paquetes*, lo que en la práctica supone una reducción de dos a tres órdenes de magnitud.

La figura 2.8 muestra el reparto de papeles. Los datos van de la memoria al periférico sin pasar por la CPU, que solo conserva dos hilos con el controlador: el de configuración y el de la interrupción de fin de bloque.

En el ecosistema de AMD, dos IP cubren este papel.

⁶Par de señales con el que emisor y receptor acuerdan cuándo un dato del flujo es válido y ha sido aceptado.



La CPU no interviene en el camino de datos.

Figura 2.8: Transferencia por acceso directo a memoria.

AXI DMA (PG021)

El IP `axi_dma` [16] traduce entre AXI4-Stream y AXI4 *memory-mapped* en dos canales independientes:

- **MM2S** (*Memory-Mapped to Stream*): lee de la memoria y emite un flujo AXI-Stream hacia el periférico.
- **S2MM** (*Stream to Memory-Mapped*): recibe un flujo del periférico y lo escribe en memoria. El fin de paquete lo marca la señal **TLAST** del *stream*.

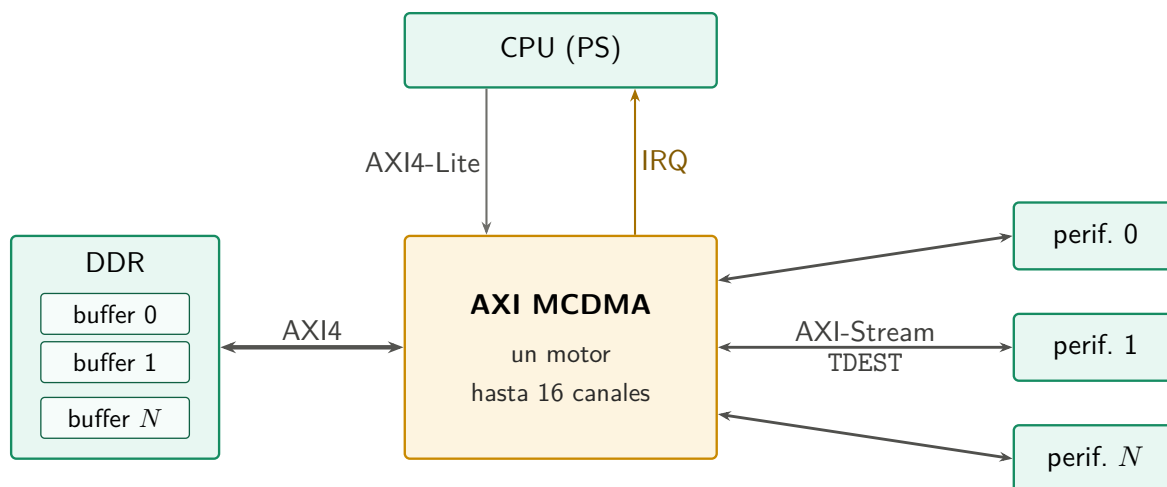
Cada canal admite dos modos de funcionamiento. En modo **directo** (*simple DMA*) el procesador escribe la dirección del *buffer* y la longitud en sendos registros, y el motor ejecuta una sola transferencia. En modo **scatter-gather** (SG) el procesador construye en memoria una lista enlazada de *descriptores de buffer* (BD), cada uno con la dirección, la longitud y los bits de control del bloque, y el DMA la recorre autónomamente. El modo SG permite encadenar transferencias sin que la CPU intervenga entre ellas, a costa de que el propio DMA lea y escriba los descriptores en memoria.

La limitación de este IP es que **sirve a un único stream**. Para N periféricos hacen falta N instancias, con su coste de área y la complejidad de gestionar en software N controladores y N interrupciones.

AXI MCDMA (PG288)

El IP `axi_mcdma` [17] levanta esa limitación: es un controlador **multicanal** (MCDMA, *Multichannel DMA*) que multiplexa hasta 16 canales lógicos sobre un único motor y un único puerto hacia la memoria. Cada canal tiene sus registros, su anillo de descriptores y su *buffer* en la DDR, pero comparten el hardware de transferencia.

El reparto entre canales lo hace la señal **TDEST** del AXI-Stream. En recepción (S2MM), el MCDMA lee el **TDEST** de cada paquete, lo toma como número de canal y escribe la carga en el *buffer* de ese canal, sin que el procesador intervenga en el demultiplexado. En transmisión (MM2S) hace lo inverso: etiqueta cada paquete con el **TDEST** de su canal de origen. Un solo par de interfaces AXI-Stream da servicio así a los N periféricos (figura 2.9).



Un motor y un puerto a memoria compartidos por los N canales.

Figura 2.9: Transferencia multicanal con AXI MCDMA.

2.6.3. Coste de la interrupción

Lo que separa una vía de transporte de otra no es el ancho de banda del bus, que sobra en todos los casos, sino **cuántas veces se interrumpe al procesador**. Atender una interrupción obliga a salvar el contexto de la tarea en curso, saltar al vector, ejecutar la rutina de servicio, reconocer la fuente en el controlador y restaurar el contexto. Ese coste es casi constante, del orden de unos pocos microsegundos en un procesador empujado, y no depende de cuántos bytes traiga la interrupción.

Si un sistema interrumpe una vez por byte, el procesador sigue el ritmo del enlace solo mientras el coste C de la interrupción sea menor que el tiempo T_b que tarda en llegar un byte. Al subir la velocidad de línea, T_b baja y C no, así que hay una velocidad a partir de la cual toda la CPU se va en entrar y salir de la rutina de servicio y ningún byte más llega a la aplicación. Con N enlaces simultáneos el umbral se alcanza N veces antes.

Mogul y Ramakrishnan describieron este régimen para las interfaces de red y lo llamaron *interrupt livelock* [18]. Por encima de una tasa de llegada crítica, el procesador ocupa cada ciclo en atender interrupciones y el rendimiento útil cae, porque no le queda tiempo para ejecutar la aplicación que debía recibir esos datos.

Existe una mitigación parcial: sacar el trabajo del contexto de interrupción hacia una tarea planificable (*deferred interrupt processing*).

La solución de fondo es cambiar la unidad de notificación, pasando de interrumpir por byte a interrumpir por paquete. Esto es lo que aporta el acceso directo a memoria.

3. Desarrollo

3.1. Preparación del entorno de desarrollo

Durante este trabajo se utilizó una máquina virtual con un sistema operativo Linux (Ubuntu/Debian) como entorno de desarrollo principal. Esta elección respondió a dos motivos prácticos. La máquina virtual puede dejarse trabajando de forma indefinida, lo que resulta necesario para las compilaciones prolongadas de la herramienta de síntesis. Además, se le pueden asignar muchos más recursos de memoria y almacenamiento de los que estarían disponibles en un equipo de trabajo convencional.

Se siguieron los siguientes pasos para dejar operativo el entorno de desarrollo.

3.1.1. Instalación de Vivado y Vitis

Para el desarrollo de la lógica programable (PL) y la generación de imágenes de arranque, se utilizaron las herramientas oficiales de AMD: Vivado Design Suite y Vitis Unified Software Platform. Vivado se empleó para el diseño a nivel de hardware y la generación del *bitstream*, mientras que Vitis se utilizó para empaquetar los binarios y generar la imagen de arranque (BOOT.BIN). Ambas estaban instaladas previamente en la máquina virtual, en su **versión 2025.1**, que es la que se mantuvo durante todo el trabajo y con la que son coherentes los componentes de arranque precompilados empleados más adelante.

3.1.2. Instalación de RTEMS

RTEMS (versión 7) se compiló desde código fuente en lugar de partir de binarios precompilados, siguiendo la guía *Quick Start* del manual de usuario oficial del proyecto [19] y empleando la herramienta RTEMS Source Builder (RSB) [20]. Un binario genérico no habría garantizado la combinación exacta que este trabajo necesitaba: un *toolchain* cruzado para la arquitectura AArch64, el BSP `zynqmp_apu` concreto de la ZCU102 y el soporte de multiprocesamiento simétrico (SMP) habilitado explícitamente en la configuración del núcleo. Sobre ese control se apoyó después la integración del driver propio del transceptor.

El entorno recién compilado se validó sobre el emulador QEMU (`qemu-system-aarch64`) con la máquina `xlnx-zcu102`, ejecutando los ejemplos `hello.exe` y `dhrystone.exe` [21] que el propio BSP incluye: confirman que el arranque y la consola serie funcionan y que el *toolchain* genera código funcional.

3.1.3. Validación del flujo de desarrollo de extremo a extremo

Con las herramientas ya instaladas, quedaba por comprobar que encajaban entre sí. Antes de abordar el transceptor serie se recorrió el flujo completo con el ejemplo más pequeño

posible: una **puerta AND**. La puerta se describió en VHDL y se sintetizó en la **PL**, y en el **PS** se escribió un programa de RTEMS que excitaba todas las combinaciones de sus entradas, leía la salida por AXI e imprimía cada resultado por la consola serie. La dificultad no está en la puerta, sino en el camino que lleva un *bitstream* y un programa en C hasta la ZCU102.

Ese camino se reduce a dejar dos ficheros en la tarjeta SD desde la que arranca la placa. Uno es la **imagen de arranque**, que reúne el *bitstream* de la PL y todo el software que levanta el sistema antes de que exista aplicación alguna; el otro es la **imagen de la aplicación** que después ejecuta el PS. La primera se construye con Vivado y Vitis, la segunda compilando la aplicación de RTEMS. El procedimiento paso a paso — la descripción VHDL, el *Block Design*, los componentes de la imagen de arranque, la posición del conmutador de la placa y el volcado del barrido — se recoge en el apartado C.8 del Anexo C, y el proyecto completo está en `tfm/00_docs/AND_example` dentro del repositorio [5].

El barrido de la tabla de verdad salió por la consola serie con la salida a uno únicamente cuando ambas entradas valían uno. Fue la primera confirmación de que la cadena entera funcionaba: el VHDL sintetizado en la PL, la imagen de arranque, la aplicación de RTEMS y la comunicación PS↔PL por AXI. De esa puesta en marcha salió además un detalle que reaparece más adelante: RTEMS desconoce a priori las direcciones de la PL, así que hay que declarar el rango `0xA0000000-0xA01FFFFFF` como memoria de periférico nada más arrancar la tarea `Init`. Sin ese mapeado, el primer acceso a los GPIO aborta la ejecución. El mismo problema vuelve en el apartado 3.6.1, ya resuelto como `mmu_map_pl_axi_early()`.

Automatización del flujo. Lo que hizo falta corregir no fue el ejemplo, sino el coste de repetirlo. Cada iteración obliga a sintetizar el diseño en Vivado, crear y compilar una plataforma y una aplicación nuevas en Vitis, ensamblar la imagen de arranque, compilar la imagen de RTEMS, copiar los ficheros a la tarjeta y arrancar la placa: **casi una hora de media**. Y cuando la placa no arranca, no da ninguna indicación del motivo. Agrava el problema que Vitis reutiliza artefactos ya compilados: un diseño nuevo no siempre llega a la imagen final, la placa se comporta como antes del cambio en el VHDL y limpiar y recompilar no da un resultado fiable. De ahí la regla que se adoptó para el resto del trabajo, plataforma y aplicación nuevas ante cualquier cambio en la PL, que es más lenta pero garantiza que la imagen corresponde al hardware recién sintetizado.

Con ese tiempo por iteración, automatizar dejó de ser una comodidad. Se escribieron tres herramientas que, entre las tres, cubren el camino entero y están disponibles en el repositorio [5]:

- `make_img.sh` compila la aplicación de RTEMS y la deja empaquetada en el formato de imagen que el gestor de arranque espera encontrar en la tarjeta.
- `automate_ymodem_update.py` ahorra el vaivén de la tarjeta SD. En lugar de sacarla de la placa, copiar el fichero desde el ordenador y volver a montarla, el script manda la imagen a la placa por el mismo cable USB que hace de consola y la deja escrita en la tarjeta. Probar un cambio en la aplicación se quedó en lanzar el script y pulsar el botón que reinicia el procesador sin cortar la alimentación.
- `generate_boot.sh` cubre la mitad que faltaba, la de la imagen de arranque. Encadena de forma desatendida los pasos que antes se hacían navegando por los asistentes gráficos de Vivado y Vitis, donde olvidar un componente o arrastrar un artefacto

antiguo basta para que la placa no arranque sin decir por qué. Admite empezar desde el proyecto de Vivado o desde un diseño ya sintetizado, según lo que haya cambiado.

El tiempo de iteración pasó así de una sesión larga de interacción gráfica a un par de órdenes en el terminal, con la garantía de que lo que arranca en la placa corresponde al hardware recién sintetizado. Este flujo quedó establecido como procedimiento estándar para el resto del trabajo.

3.2. Diseño del transceptor serie configurable (PL)

El transceptor serie se diseñó para ser altamente configurable y adaptable a cualquier dispositivo, debido a que se desconocían los dispositivos finales que iban a estar conectados al OBC.

El transceptor completo consta de varios bloques conectados entre sí. Los bloques fundamentales son:

- **Oscilador controlado numéricamente** (`NC0.vhd`): proporciona la señal de temporización de bit para las 54 velocidades de bus soportadas. El transmisor y el receptor instancian uno cada uno para poder trabajar a cualquiera de ellas.
- **Transmisor** (`TX_CONFIGURABLE_SERIAL.vhd`): serializa y emite las tramas, y gobierna la señal de *Driver Enable* del transceptor físico.
- **Receptor** (`RX_CONFIGURABLE_SERIAL.vhd`): detecta el inicio de trama, muestrea la línea, comprueba paridad y bit de parada y valida la palabra recibida.
- **Registro de desplazamiento** (`ShiftRegister.vhd`): convierte a paralelo los bits recibidos. Incorpora la lógica necesaria para adaptarse a todas las combinaciones de número y orden de los bits.
- **Memoria FIFO**: almacena las palabras recibidas hasta que el PS las lee.
- **Bloque de canal** (`CONFIGURABLE_SERIAL.vhd`): integra los bloques anteriores y añade la lógica de acoplamiento entre ellos.
- **Bloque top** (`CONFIGURABLE_SERIAL_TOP.vhd`): empaqueta el canal completo en los dos vectores que lo conectan al PS.

Los parámetros configurables son:

- Velocidad de bus, o *baudrate*, desde 50 hasta 4 000 000 baudios.
- Número de bits de parada, o *stop bits*, que marcan el final de la trama y devuelven la línea a reposo: 1, 1,5 o 2.
- Modo de paridad: par, impar, marca, espacio o deshabilitada. La paridad es un único bit de comprobación que iguala a par o a impar el número de unos de la palabra.
- Orden de los bits, empezando por el menos significativo (LSB) o por el más significativo (MSB).
- Número de bits de datos, entre 5 y 9.

Los siguientes apartados describen esos bloques uno a uno, siguiendo el camino que recorre un dato. Primero, el NCO, que marca la base de tiempos: el transmisor y el receptor tienen cada uno el suyo. Después, el transmisor y el receptor. Y por último, los bloques que los conectan al PS. El apartado 3.2.5 cierra con el diagrama de conjunto (figura 3.2), cuando ya se han presentado todas sus piezas.

3.2.1. Oscilador controlado numéricamente (NCO)

El transmisor y el receptor tienen cada uno su propia base de tiempos, un NCO que se describe en primer lugar porque su salida, el *tick*, es la referencia con la que cada uno mide el tiempo de bit.

El transceptor debía poder variar su velocidad en tiempo de ejecución sobre un rango muy amplio de baudrates, y un divisor entero del reloj de la FPGA no puede generar con exactitud la mayoría de esas frecuencias. La solución fue un **oscilador controlado numéricamente** (NCO), un tipo de circuito que corrige periódicamente el desfase acumulado por esa imprecisión, y que se diseñó a medida para las necesidades del proyecto: genera la señal de *enable* al baudrate seleccionado de entre 54 valores de velocidad almacenados en una tabla de verdad.

El bloque expone una interfaz mínima: además del reloj, una entrada de reset asíncrono activo a nivel bajo (*rst*), una habilitación (*en*), el selector de velocidad (*baud_sel*, 6 bits), un selector de modo (*half_mode*) y la salida de temporización (*tick*). El número de bits del acumulador es un parámetro *generic*, fijado a 32.

El circuito consiste en un sumador acumulador: para cada frecuencia seleccionada, se tiene calculado un incremento concreto que depende del número de bits del sumador. Cuantos más bits tenga este sumador, mejor precisión se consigue. En esta implementación se han utilizado 32 bits. El incremento se suma y acumula en cada período de reloj. Cuando el sumador se desborda, se utiliza el bit de *carry_out* como *tick* del NCO. Este *tick* se genera a la frecuencia deseada, y el mecanismo de acumulación garantiza que el error promedio sea inferior al de un divisor entero simple.

Los incrementos no se almacenan como literales: los calcula en tiempo de síntesis una función que evalúa $\text{inc} = \lfloor 2^{32} \cdot f_{\text{baud}} / f_{\text{clk}} \rfloor$ para cada entrada de la tabla, de modo que basta cambiar la constante de frecuencia de reloj para reajustar las 54 velocidades. La tabla tiene dos columnas por entrada: el incremento nominal y el de doble frecuencia, que selecciona *half_mode*. Este segundo valor es el que permite medir semiperíodos de bit, algo que necesitan tanto el transmisor, en el campo de parada, como el receptor, en el bit de inicio. Tabularlo en lugar de duplicar el primero evita arrastrar multiplicado por dos el error de truncamiento del incremento nominal.

Las dos entradas de control restantes son las que permiten a la máquina de estados del transmisor y a la del receptor anclar la fase de su temporizador: *rst* pone a cero acumulador y salida, con lo que el *tick* arranca alineado con el inicio de la trama, y *en* congela el acumulador en los estados en los que no se necesita temporización, evitando que la fase derive entre tramas. La salida *tick* está registrada, por lo que aparece un ciclo de reloj después del desbordamiento.

Para validar su funcionamiento se desarrolló un *testbench* dedicado, *tb_NCO.vhd* (en *tfm/01_ip_serie/testbench/* del repositorio del proyecto [5]), que barre las **29 velocidades del rango de uso** (de 9600 a 3,6864 Mbaudios) en sus **dos modos** de

temporización, y mide en cada punto la frecuencia real del *tick* promediando 200 períodos tras descartar los diez primeros. Son 58 medidas independientes. Las entradas de la tabla por debajo de 9 600 baudios y la de 4 Mbaudios no se ejercitan, por quedar la primera fuera del uso previsto y exigir la segunda un tiempo de simulación desproporcionado. La transcripción siguiente recoge el principio y el final de la campaña:

Listado 3.1: Salida de la simulación de `tb_NCO.vhd` (extracto)

```

---- Configuracion: 9600 half=0 baud=9600 ----
CFG=9600 expected=9600.0 Hz measured=9600.002000 Hz err=0.002000 Hz (0.2 ppm)
---- Configuracion: 9600 half=1 baud=9600 ----
CFG=9600 expected=19200.0 Hz measured=19200.004000 Hz err=0.004000 Hz (0.2 ppm)
---- Configuracion: 14400 half=0 baud=14400 ----
CFG=14400 expected=14400.0 Hz measured=14399.988000 Hz err=-0.012000 Hz (-0.8 ppm)
...
---- Configuracion: 115200 half=0 baud=115200 ----
CFG=115200 expected=115200.0 Hz measured=115200.074000 Hz err=0.074000 Hz (0.6 ppm)
---- Configuracion: 115200 half=1 baud=115200 ----
CFG=115200 expected=230400.0 Hz measured=230400.148000 Hz err=0.148000 Hz (0.6 ppm)
...
---- Configuracion: 3686400 half=0 baud=3686400 ----
CFG=3686400 expected=3686400.0 Hz measured=3685956.506000 Hz err=-443.494000 Hz
(-120.3 ppm)
---- Configuracion: 3686400 half=1 baud=3686400 ----
CFG=3686400 expected=7372800.0 Hz measured=7371913.012000 Hz err=-886.988000 Hz
(-120.3 ppm)
Fin del testbench: medidas completadas para todas las frecuencias.

```

De esa campaña procede la tabla C.4 del Anexo C, y su lectura valida el diseño: el peor error de todo el rango es de 120 ppm (a 3,6864 Mbaudios), dos órdenes de magnitud por debajo de la tolerancia habitual del $\pm 2\%$ de un receptor UART, y en la mayoría de las velocidades estándar el error es nulo o inferior a 1 ppm. El NCO queda así acreditado como fuente de temporización fiable para todo el rango de operación del transceptor.

3.2.2. Transmisor

El transmisor recibe una palabra en paralelo y la emite por la línea serie con el formato de trama que fijan sus entradas de configuración. Sus puertos se recogen en la tabla 3.1.

Puerto	Dir.	Función
Clk	in	Reloj del sistema, 100 MHz, activo por flanco de subida.
Reset	in	Reset asíncrono del sistema, activo a nivel bajo.
Start	in	Pulso de petición de transmisión.
Data	in	Palabra a transmitir (9 bits, alineada en el LSB).
baud_sel	in	Índice de velocidad dentro de la tabla del NCO (6 bits).
stop_bit	in	Duración del campo de parada, en semiperíodos de bit (010 = 1, 011 = 1,5, 100 = 2).
parity	in	Modo de paridad: par, impar, <i>mark</i> , <i>space</i> o deshabilitada.
bit_order	in	Orden de emisión: 0 = LSB <i>first</i> , 1 = MSB <i>first</i> .
data_bits	in	Número de bits de datos (000–100 → 5–9 bits).
EOT	out	Transmisor libre: a nivel alto únicamente en reposo.
DE	out	<i>Driver Enable</i> del transceptor físico.
TX	out	Línea de transmisión serie.

Tabla 3.1: Puertos del transmisor TX_CONFIGURABLE_SERIAL.

Dispone además de un parámetro *generic*, DELAY_CYCLES, cuyo uso se detalla más adelante.

Máquina de estados

La temporización del tiempo de bit y del semiperíodo de bit la proporciona el NCO, instanciado dentro del propio transmisor: la FSM lo habilita, lo pone a cero y conmuta su entrada `half_mode` según el estado, y consume su salida `tick` como base de tiempos. El comportamiento se organiza en siete estados:

- **Idle:** estado de reposo. Mantiene la línea en alto, EOT a uno y el NCO detenido. Al llegar **Start** activa DE, pone a cero el contador de retardo y pasa a **PreDE**.
- **PreDE:** mantiene DE activo y la línea en reposo durante DELAY_CYCLES ciclos de reloj, para dar tiempo al transceptor físico a habilitar su etapa de salida antes de que empiece la trama. Al cumplirse la cuenta se borra el contador de bits y se resetea el acumulador del NCO, con lo que la fase del temporizador arranca desde cero justo en el bit de inicio, sin arrastrar el desfase acumulado de la trama anterior. Hecho esto se pasa a **StartBit**.
- **StartBit:** fuerza la línea a nivel bajo durante un tiempo de bit completo. Con el primer *tick* del NCO pasa a **SendData**.
- **SendData:** saca a la línea el bit seleccionado por el multiplexor de serialización y avanza el contador de bits con cada *tick*. Cuando el contador alcanza el número de

bits configurado, pasa a **ParityBit** presentando ya el bit de paridad o, si la paridad está deshabilitada (**parity** = 100), directamente a **StopBit** con la línea en alto.

- **ParityBit**: mantiene el bit de paridad durante un tiempo de bit y pasa a **StopBit**.
- **StopBit**: mantiene la línea en alto y conmuta el NCO a **half_mode**, que lo hace oscilar al doble de la velocidad seleccionada. Contando semiperíodos (dos, tres o cuatro según **stop_bit**) se obtienen los tres campos de parada admitidos, incluido el de 1,5 bits, que no es expresable con una cuenta de bits enteros. Alcanzada la cuenta pasa a **PostDE**.
- **PostDE**: simétrico de **PreDE**. Mantiene **DE** activo y la línea en reposo otros tantos ciclos antes de volver a **Idle**, donde **DE** se libera.

Lógica de apoyo a la máquina de estados

La FSM se limita a secuenciar la trama y a generar señales de control. El trabajo de seleccionar qué bit sale a la línea en cada momento recae en un camino de datos puramente combinacional que la acompaña, y que evita tener que desplazar o rotar la palabra de entrada.

Banco de contadores. Tres contadores síncronos con habilitación y borrado independientes, gobernados por la FSM: el de bits de datos (4 bits), el de semiperíodos del campo de parada (3 bits) y el de retardo de **DE** (14 bits, suficiente para el valor por defecto de **DELAY_CYCLES**). Todos comparten el mismo proceso síncrono y el mismo reset asíncrono.

Decodificador de anchura de palabra. Traduce el código de **data_bits** (000–100) al número real de bits (5–9), que es el valor terminal con el que se compara el contador de datos.

Multiplexor de serialización. Selecciona el bit a emitir a partir de la cuenta actual, en tres etapas. La rama **LSB first** es directa: el índice del contador es el índice del bit. La rama **MSB first** requiere cinco multiplexores distintos, uno por anchura de palabra, porque el primer bit emitido depende del tamaño de la trama: es el bit 4 en una palabra de 5 bits y el bit 8 en una de 9. Una segunda etapa elige entre esas cinco ramas según **data_bits**, y una tercera entre **LSB** y **MSB first** según **bit_order**. El resultado es que la palabra permanece inmóvil en su registro de entrada durante toda la trama.

Generador de paridad. Una red de puertas XOR calcula la paridad de los bits activos, con una expresión por cada anchura de palabra, y un multiplexor final aplica el modo configurado: paridad par, su complemento para la impar, o las constantes uno y cero para los modos *mark* y *space*.

Reset del NCO. La entrada de reset del NCO se combina con el reset global del sistema, de modo que el temporizador se inicializa tanto cuando lo pide la FSM (antes de cada trama) como cuando se resetea el transceptor completo.

Retardo de establecimiento del *Driver Enable*

Los estados **PreDE** y **PostDE** se incorporaron después de la primera versión del transmisor, al aparecer caracteres corruptos de forma ocasional en las primeras recepciones sobre la placa CDHS. La hipótesis de partida era que **DE** conmutaba con demasiada rapidez en el paso de transmisión a recepción y no dejaba establecerse al transceptor físico.

Los dos estados sitúan un margen configurable a ambos lados de la trama: DE se activa DELAY_CYCLES ciclos antes del bit de inicio y se libera otros tantos después del último bit de parada. El valor por defecto es de 10 000 ciclos, esto es, 100 μ s con el reloj de 100 MHz. En simulación se reduce a una decena de ciclos mediante el *generic* para no alargar los *testbench*, sin que ello afecte al valor de síntesis.

La medida no eliminó los caracteres corruptos, cuya causa estaba en el receptor y se aborda en el apartado siguiente, pero los dos estados se conservaron: el margen es coherente con los tiempos de habilitación que especifican los transceptores empleados y su coste, 200 μ s por trama en el peor caso, es despreciable frente al tiempo de trama salvo a las velocidades más altas.

3.2.3. Receptor

El receptor vigila la línea serie, recupera la temporización de bit a partir del flanco de inicio y entrega los bits muestreados al registro de desplazamiento, señalando al final si la trama es válida. Sus puertos se recogen en la tabla 3.2. Las cinco entradas de configuración son las mismas del transmisor y con idéntica codificación, por lo que ambos comparten la palabra de configuración del canal.

Puerto	Dir.	Función
Clk, Reset	in	Reloj de 100 MHz y reset asíncrono activo a nivel bajo.
baud_sel, stop_bit, parity, bit_order, data_bits	in	Configuración del formato de trama, idéntica a la del transmisor.
LineRD_in	in	Línea de recepción, ya registrada por el bloque de canal.
shiftRegister_Q	in	Palabra normalizada que va acumulando el registro de desplazamiento, sobre la que se calcula la paridad esperada.
ERROR_OK	in	Borrado de las banderas de error.
PAR_ERROR	out	Error de paridad.
FRAME_ERROR	out	Error de trama (bit de parada ausente).
Valid_out	out	Pulso de bit válido; habilita el registro de desplazamiento.
Code_out	out	Valor muestreado de la línea, que entra al registro de desplazamiento.
Store_out	out	Pulso de escritura de la palabra completa en la FIFO.

Tabla 3.2: Puertos del receptor RX_CONFIGURABLE_SERIAL.

Máquina de estados

El receptor no dispone de un reloj común con el emisor, de modo que la referencia temporal ha de extraerla de la propia línea. Su FSM consta de cinco estados:

- **Idle:** NCO detenido y línea en observación. Un nivel bajo en la línea se interpreta como posible bit de inicio: se resetea el acumulador del NCO, con lo que la fase del temporizador queda anclada al flanco de bajada. Se borra el contador de bits y se pasa a **StartBit**.
- **StartBit:** el NCO trabaja en **half_mode**, de forma que su primer *tick* llega a mitad del bit de inicio. En ese instante se vuelve a muestrear la línea. Si sigue en bajo, el bit de inicio es legítimo y se pasa a **RcvData**. Si ha vuelto a alto, se descarta y se regresa a **Idle**.
- **RcvData:** cada *tick*, ya a la velocidad nominal, genera un pulso de **Valid_out**, que hace que el registro de desplazamiento capture el valor presente en la línea, e incrementa el contador de bits. Como la fase arrancó a mitad del bit de inicio, todos los *ticks* posteriores caen en el centro de su bit, que es el punto de máxima inmunidad frente al error de velocidad entre los dos extremos del enlace. Alcanzado el número de bits configurado, se pasa a **ParityBit** o, si la paridad está deshabilitada, directamente a **StopBit**.
- **ParityBit:** compara el valor muestreado con la paridad esperada. Si coinciden, pasa a **StopBit**. Si no, activa **PAR_ERROR** y vuelve a **Idle** descartando la trama, sin escribirla en la FIFO.
- **StopBit:** muestrea el centro del primer bit de parada. Si la línea está en alto, emite **Store_out** y la palabra se escribe en la FIFO. Si está en bajo, la trama está desalineada y se activa **FRAME_ERROR**. En ambos casos se regresa a **Idle**.

Solo se comprueba el primer bit de parada: la entrada **stop_bit** no interviene en la FSM del receptor. El resto del campo de parada transcurre con la FSM ya en **Idle**, donde la línea en reposo está en alto y no se dispara ninguna detección. El receptor queda así preparado para el siguiente flanco de inicio, sin necesidad de contar la duración exacta del campo.

Lógica de apoyo a la máquina de estados

Contador de bits y decodificador de anchura. Idénticos a los del transmisor: el contador avanza con cada *tick* en **RcvData** y se compara con la anchura de palabra decodificada a partir de **data_bits**.

Cálculo de la paridad esperada. La red de puertas XOR opera sobre **shiftRegister_Q**, es decir, sobre la palabra que el registro de desplazamiento ya ha normalizado. Gracias a esa normalización el cálculo es el mismo en LSB y en MSB *first*, y reutiliza sin cambios el esquema del transmisor: paridad par, su complemento para la impar y las constantes de los modos *mark* y *space*.

Registros de error. **PAR_ERROR** y **FRAME_ERROR** son biestables que retienen el error a uno mientras **ERROR_OK** esté a cero, y el PS los borra poniendo ese bit a uno. El borrado es síncrono, de modo que, con **ERROR_OK** mantenido a uno, las banderas se limpian en cada ciclo de reloj y el software tiene que bajarlo a cero para poder leerlas. El mecanismo se incorporó como detector de errores para el software, pero acabó teniendo poca relevancia práctica: el driver deja **ERROR_OK** fijo a uno y la calidad del enlace se mide comparando los datos recibidos.

Verificación del bit de inicio a mitad de período

El muestreo a mitad del bit de inicio descrito en **StartBit** cumple una segunda función, además de fijar la fase de muestreo: filtra los falsos inicios de trama. Pulsos de ruido breves en la línea generan flancos descendentes que un receptor sin esta comprobación aceptaría como bits de inicio válidos, capturando a continuación una trama de basura y entregando un carácter corrupto al *buffer*. Con la verificación, un pulso más corto que medio bit deja la línea de nuevo en alto antes del *tick*, y la FSM regresa a **Idle** sin capturar nada. Solo se prosigue con la recepción si la línea sigue en bajo al llegar a la mitad del bit. Es el primero de los instantes de muestreo marcados en la figura 3.1.

Esta técnica es una práctica estándar de robustez en receptores UART, y es la que resolvió los caracteres corruptos observados en las pruebas sobre la placa CDHS: con ella no volvió a aparecer ninguno en las pruebas posteriores.

Cronograma de una trama completa

Con el transmisor y el receptor ya descritos, la figura 3.1 recoge la secuencia completa de una trama sobre la línea, con la relación temporal entre los tres elementos que gobiernan el enlace: la ventana de **DE**, los *ticks* con los que el transmisor delimita cada bit y los instantes en los que el receptor muestrea. La configuración representada es de 8 bits de datos, paridad par, un bit de parada y orden *LSB first*, con el dato **0x53**. El transmisor consume un *tick* por frontera de bit y pasa al doble de frecuencia en el campo de parada, donde cuenta semiperíodos. El receptor, tras alinear su fase medio bit después del flanco de inicio, muestrea en el centro de cada bit. El primer muestreo, en el centro del bit de inicio, es el que descarta los falsos inicios de trama. Las zonas sombreadas **PreDE** y **PostDE** no están dibujadas a escala: duran 100 μ s frente a los 8,7 μ s del tiempo de bit a 115 200 baudios.

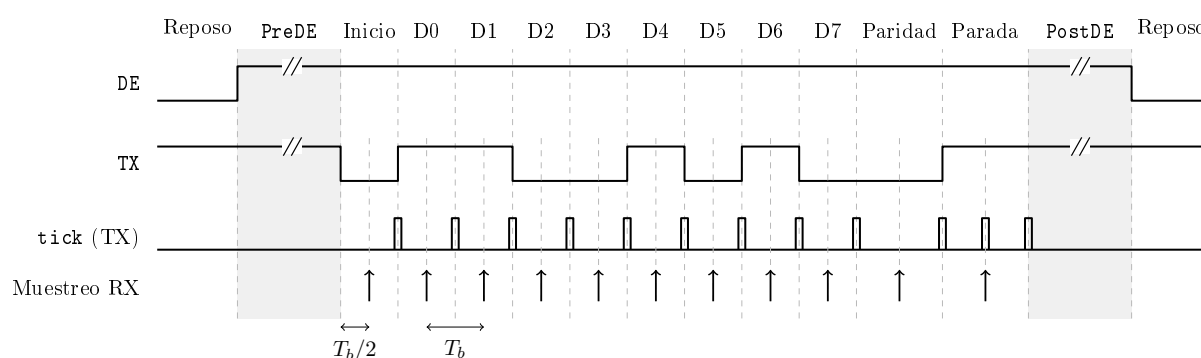


Figura 3.1: Cronograma de una trama completa en configuración 8E1, *LSB first*.

3.2.4. Normalización de la palabra recibida y almacenamiento

Entre el receptor y el PS se sitúan dos bloques auxiliares. El registro de desplazamiento convierte a paralelo los bits que entrega el receptor y **normaliza** la palabra: sea cual sea la anchura y el orden de bits configurados, el primer bit de datos recibido acaba siempre en **Q(0)** y la palabra queda alineada al **LSB**. Sus dos consumidores, la FIFO y el cálculo de paridad esperada, pueden así ignorar la configuración del canal, y el software del PS recibe la palabra en el mismo formato en los cinco anchos y en los dos órdenes posibles.

La FIFO es un bloque IP de 9 bits de dato y 512 de profundidad que almacena los mensajes ya validados hasta que el PS los lee. Su habilitación de lectura se toma **a nivel** y no por flanco, condición necesaria para el transporte por DMA descrito en la sección 3.7: con la lectura a nivel la FIFO avanza exactamente en el ciclo en que el consumidor acepta el dato.

La descripción RTL de ambos bloques, con los caminos de desplazamiento y la construcción de las señales de escritura y lectura, se recoge en el apartado C.6 del Anexo C.

3.2.5. Bloque de canal

CONFIGURABLE_SERIAL.vhd instancia el transmisor, el receptor, el registro de desplazamiento y la FIFO, y añade la lógica de acoplamiento entre ellos y con el PS:

- **Registro de la línea de recepción.** La entrada RD se registra un ciclo antes de entrar al receptor, para no muestrear directamente una señal externa asíncrona respecto del reloj del sistema.
- **Detección de flanco de los mandos del PS.** TX_Send y Data_read llegan del PS como señales de nivel a través del AXI GPIO. Sendos detectores de flanco de dos etapas de registro los convierten en pulsos de un ciclo, que es lo que esperan los bloques internos.
- **Arranque de la transmisión.** El pulso de Start hacia el transmisor solo se emite si además el transmisor está libre (EOT activo), y en ese mismo ciclo la palabra presente en Data_in se captura en un registro propio. De este modo el dato permanece estable durante toda la trama aunque el PS modifique el registro de escritura, y las peticiones que llegan con una transmisión en curso se ignoran en lugar de corromperla.
- **Adaptación del campo de parada.** El PS codifica los bits de parada en dos bits; transmisor y receptor los esperan como una cuenta de semiperíodos de tres bits. Una pequeña lógica combinacional traduce entre ambas representaciones.
- **Vector de depuración.** Una salida de cuatro bits agrupa las señales internas del camino de recepción (FIFO vacía, escritura en FIFO, bit válido y línea registrada), empleada únicamente para el diagnóstico en simulación.

Con el canal ya completo, la figura 3.2 recoge el conjunto: los bloques descritos en los apartados anteriores, la lógica de acoplamiento que acaba de enumerarse y los puertos con los que el canal se presenta al exterior, agrupados por función. Se aprecia en ella que la palabra de configuración es común a los dos sentidos, que el camino de recepción encadena receptor, registro de desplazamiento y FIFO antes de llegar al PS, y que SLO no entra en la lógica: viaja del PS al pin del transceptor físico sin tocar el canal.

3.2.6. Bloque *top* del transceptor

CONFIGURABLE_SERIAL_TOP.vhd no añade funcionalidad: empaqueta el canal completo en dos vectores, uno de entrada y otro de salida, para que pueda conectarse a los dos canales de un AXI GPIO sin lógica adicional. El reparto de bits, que es el contrato entre la PL y el driver de RTEMS descrito en la sección siguiente, se recoge en la tabla 3.3.

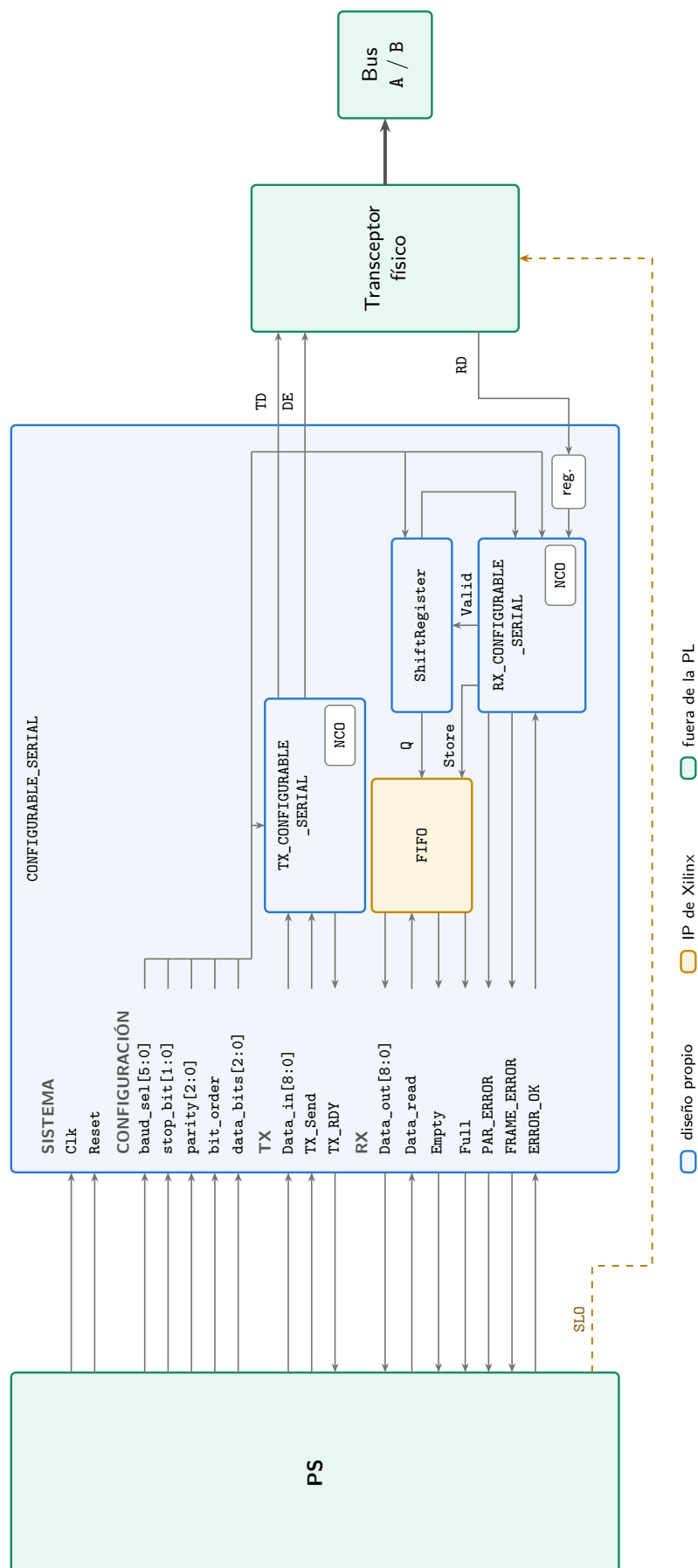


Figura 3.2: Diagrama de bloques de un canal del transceptor serie configurable.

Bits	Campo	Función
<i>Vector de configuración y escritura (PS → PL), 28 bits</i>		
8:0	Data_in	Palabra a transmitir.
9	TX_Send	Petición de transmisión.
10	ERROR_OK	Reconocimiento de las banderas de error.
11	Data_read	Lectura de la FIFO de recepción.
17:12	baud_sel	Índice de velocidad.
19:18	stop_bit	Bits de parada.
22:20	parity	Modo de paridad.
25:23	data_bits	Número de bits de datos.
26	bit_order	Orden de bits.
27	SLO	Control del limitador de <i>slew rate</i> del transceptor físico.
<i>Vector de estado y lectura (PL → PS), 14 bits</i>		
8:0	Data_out	Palabra recibida.
9	Empty	FIFO de recepción vacía.
10	Full	FIFO de recepción llena.
11	PAR_ERROR	Error de paridad.
12	FRAME_ERROR	Error de trama.
13	TX_RDY	Transmisor libre.

Tabla 3.3: Reparto de bits de los vectores de interfaz del bloque CONFIGURABLE_SERIAL_TOP.

Junto a esos dos vectores, el bloque expone las líneas físicas del canal (TD, RD, DE y SLO) y una salida auxiliar de dos bits que duplica TX_RDY y Empty. Esta última es la que se lleva al controlador de interrupciones, para que el PS pueda ser avisado tanto de la llegada de datos como de la disponibilidad del transmisor sin tener que sondear el vector de estado. La salida SLO es una conexión directa desde el bit 27 del vector de configuración hasta el pin del transceptor físico: no interviene en la lógica del transceptor en VHDL.

Envoltorios de nivel superior

El canal así empaquetado es la unidad que reutilizan los tres envoltorios de nivel superior presentes en el repositorio, cada uno asociado a una de las arquitecturas de transporte entre el PS y la PL:

- MULTI_SERIAL_CORE.vhd agrupa hasta catorce instancias del bloque *top* en un único IP, con las interfaces de todos los canales aplanadas en vectores de ancho fijo y un par de genéricos por canal que ocultan los pines DE y SLO que no se cablean; es el bloque de catorce canales de la arquitectura basada en AXI GPIO.
- SERIAL_CHANNEL_IP.vhd envuelve un solo canal como IP independiente, con esos mismos genéricos, para que los canales puedan instanciarse y conectarse uno a uno

en el *Block Design* en lugar de como un bloque monolítico.

- `UART_AXIS_TOP.vhd` sustituye la interfaz paralela por un registro de configuración AXI-Lite y dos canales AXI-Stream (uno de transmisión y otro de recepción, este último con *TDEST* fijado al índice de canal y *TLAST* por byte), que es lo que permite conectar el transceptor directamente a un controlador de DMA.

Ninguno de los tres modifica el comportamiento del transceptor descrito en esta sección: cambian únicamente la forma en que sus dos vectores se presentan al PS. Su papel dentro de cada arquitectura se detalla en la sección 3.7.

3.2.7. Verificación en simulación del canal completo

Cada bloque descrito en los apartados anteriores tiene su propio *testbench*. La comprobación que realmente acredita el transceptor es la del canal completo. Es la única que ejercita a la vez las dos máquinas de estados, la normalización del registro de desplazamiento y la coherencia entre la paridad generada y la esperada. El *testbench* `tb_CONFIGURABLE_SERIAL_TOP.vhd`, en `tfm/01_ip_serie/testbench/` del repositorio del proyecto [5], conecta la salida TX del canal a su propia entrada RD. **Recorre el espacio de configuración completo** a 921 600 baudios: cinco anchuras de palabra (5 a 9 bits) $\times$ dos órdenes de bit $\times$ tres campos de parada (1, 1,5 y 2) $\times$ cinco modos de paridad, es decir **150 configuraciones**, con tres tramas de contenido distinto en cada una. En total, **450 tramas** comparadas dato a dato entre lo enviado y lo recibido:

Listado 3.2: Salida de la simulación de `tb_CONFIGURABLE_SERIAL_TOP.vhd` (extracto)

```
==== TB START: TX->RX loopback tests ====
==== sending byte ====
PASS cfg bits=5 order='0' stop=1 par=0 sent=11 rec=11
==== sending byte ====
PASS cfg bits=5 order='0' stop=1 par=0 sent=12 rec=12
==== sending byte ====
PASS cfg bits=5 order='0' stop=1 par=0 sent=13 rec=13
==== sending byte ====
PASS cfg bits=5 order='0' stop=1 par=1 sent=28 rec=28
...
PASS cfg bits=8 order='1' stop=2 par=4 sent=118 rec=118
...
==== sending byte ====
PASS cfg bits=9 order='1' stop=3 par=4 sent=132 rec=132
==== sending byte ====
PASS cfg bits=9 order='1' stop=3 par=4 sent=133 rec=133
==== sending byte ====
PASS cfg bits=9 order='1' stop=3 par=4 sent=134 rec=134
==== TESTS FINISHED ====
Passed = 450 / Total = 450
```

Las 450 tramas se recuperaron íntegras. Ninguna disparó las banderas `PAR_ERROR` ni `FRAME_ERROR`, que el *testbench* vigilaba trama a trama. El dato relevante no es tanto el número como su cobertura. Al completar el barrido quedó comprobado que el transmisor y el receptor coinciden en todos sus bloques: el multiplexor de serialización, el decodificador de anchura, la red de paridad y los dos caminos de desplazamiento del registro. Esta coincidencia se cumple **en todas** las combinaciones que el PS puede programar, y no solo en la configuración 8N1 habitual.

Por legibilidad, en esta transcripción y en las que siguen se reproducen únicamente las líneas de **report** del *testbench*. Se omiten las marcas de tiempo, iteración e instancia que el simulador antepone a cada una.

3.2.8. Bloque Zynq UltraScale+ MPSoC

Para que el proyecto en Vivado funcione correctamente, es importante aplicar el *preset* al bloque IP Zynq UltraScale+ antes de añadir el resto de elementos. El *preset* aplica configuraciones de relojes y alimentación cruciales para el correcto funcionamiento de la PL.

3.3. Herramientas de apoyo al desarrollo del transceptor

3.3.1. Generación de transceptores con TCL

Se desarrolló un script de TCL, `new_generate_transceivers.tcl`, que permite generar y conectar automáticamente un número deseado de transceptores en el *Block Design* de Vivado. Recibe el número de instancias y los nombres del bloque Zynq y del *SmartConnect* a los que engancharse, y a partir de ahí añade y conecta el resto de los bloques necesarios: un AXI GPIO de doble canal por transceptor, el controlador de interrupciones común, un bloque de solo lectura con los metadatos del sistema que consulta el driver, y la asignación de direcciones en el mapa de memoria del PS. Parte de un proyecto que ya tenga las fuentes .vhd del transceptor, el *Block Design* creado y el bloque IP Zynq UltraScale+ MPSoC configurado. El script está en `tfm/01_ip_serie/scripts/` dentro del repositorio del proyecto [5], y cada proyecto de Vivado conserva su copia en el subdirectorío `scripts/` correspondiente.

3.3.2. Generación de proyecto con TCL

También se desarrolló otro script de TCL, `new_rebuild_all.tcl`, que regenera el proyecto entero de Vivado con un número determinado de transceptores, partiendo desde cero. Crea el proyecto para la ZCU102, añade las fuentes y las restricciones, instancia y configura el bloque Zynq, invoca al generador de transceptores anterior, valida el diseño y genera el *wrapper*. El proyecto queda así listo para lanzar la síntesis y la generación del *bitstream*. Esto permite reproducir exactamente cualquier configuración sin necesidad de guardar el proyecto de Vivado completo en el repositorio.

3.4. Diseño del driver serie en RTEMS (PS)

Completado el transceptor en la PL, se continuó con su contraparte en el PS: los ficheros `transceiver.c` y `transceiver.h`. Es un driver para RTEMS que habla con el hardware a través del AXI GPIO de cada canal y expone a la aplicación una API pública sencilla, la misma para los catorce canales. Las subsecciones siguientes lo describen de fuera hacia dentro. Primero se presenta la API que ve la aplicación. Después, el mapa de registros con el que el driver dialoga con la PL. Por último, se explica cómo gestiona por dentro la transmisión y la recepción sin bloquear al procesador.

Este driver se planteó como prueba de concepto: el objetivo era que funcionase, sin requisitos impuestos y sin optimizar nada. Funcionó, y con eso queda cumplido su propósito.

3.4.1. API pública

Para la aplicación, cada canal serie es un objeto `Transceiver` sobre el que actúan las seis funciones de su API pública. El driver oculta el reparto de memoria, la gestión de interrupciones y la sincronización entre tareas: el código de usuario se limita a abrir el canal, fijar su configuración de protocolo y recibir o transferir bytes. Las funciones, con su tipo de retorno, son:

- `uint32_t Transceiver_Global_INIT(void)`
- `rtems_status_code Transceiver_Init(dev, id, cfg)`
- `void Transceiver_SetRxCallback(dev, cb, arg)`
- `size_t Transceiver_Read(dev, buf, maxlen)`
- `int Transceiver_Send(dev, data, len)`
- `int Transceiver_SendString(dev, s)`

`Transceiver_Global_INIT()` se invoca una sola vez al arranque. Descubre cuántos canales trae el *bitstream*, prepara el controlador de interrupciones, instala la rutina de interrupción común y devuelve el número de canales encontrados.

`Transceiver_Init()` se llama una vez por canal. Recibe una estructura `Transceiver_Config_t` con la tasa de baudios, los bits de datos, la paridad, los bits de parada y el orden de bits, y deja el canal listo para operar.

`Transceiver_SetRxCallback()` registra la función de recepción, el *callback*. La escribe el usuario y su contenido es libre. El driver la invoca cada vez que hay bytes nuevos disponibles. Es la aplicación la que decide qué hacer con ellos: leerlos en el acto, despertar a otra tarea, contabilizar la recepción o cualquier otra cosa. Es solo una notificación: cuando se dispara, el driver ya ha sacado del hardware los bytes y los tiene disponibles.

`Transceiver_Read()` extrae del *buffer* de recepción hasta `maxlen` bytes ya recibidos, devuelve cuántos ha copiado y no bloquea aunque no haya ninguno. Es la función con la que la aplicación retira los datos, normalmente desde el *callback* o desde una tarea propia.

`Transceiver_Send()` encola los bytes a transmitir y retorna de inmediato, sin esperar a que salgan por la línea. Devuelve cuántos aceptó, o un error si el *buffer* de salida estaba lleno. `Transceiver_SendString()` es la misma operación para cadenas terminadas en nulo.

3.4.2. Interfaz con el hardware

El driver y la PL se comunican a través de un único bloque AXI GPIO por canal, configurado como doble canal y mapeado en 4 KB del espacio de direcciones del procesador. Los dos canales tienen sentido fijo. El canal 1, de 28 bits, escribe del PS a la PL. El canal 2, de 14 bits, lee de la PL al PS. Cada uno se maneja con su registro de datos, situado en los desplazamientos `0x00` y `0x08` respecto de la dirección base de la instancia. Toda la interacción con el hardware se reduce así a escribir y leer dos palabras de 32 bits.

El registro de escritura reúne el dato a transmitir, tres bits de control y los campos de configuración de protocolo. El de lectura reúne el byte recibido y las banderas de estado. La tabla 3.4 recoge el reparto de bits.

Escritura (PS → PL)		Lectura (PL → PS)	
Bits	Campo	Bits	Campo
8:0	Dato a transmitir	8:0	Byte recibido
9	SEND	9	RX_EMPTY
10	ERROR_OK	10	RX_FULL
11	DATA_READ	11	PARITY_ERROR
17:12	Tasa de baudios	12	FRAME_ERROR
19:18	Bits de parada	13	TX_RDY
22:20	Paridad		
25:23	Bits de datos		
26	Orden de bits		
27	SLO		

Tabla 3.4: Mapa de bits de los dos canales del AXI GPIO de una instancia.

Los campos de configuración de protocolo se escriben una sola vez, en `Transceiver_Init()`, y permanecen fijos mientras el canal está abierto. El driver guarda esa palabra y la reutiliza como base cada vez que toca los bits inferiores, para no alterar la configuración al transmitir. Los bits de control funcionan como pulsos, no como niveles. Para transmitir se deposita el byte en los bits 8:0 y se lleva `SEND` a uno y de vuelta a cero, lo que ordena al transmisor cargar el dato. Para consumir un byte recibido se lee el canal de entrada y se pulsa `DATA_READ`, que hace avanzar la FIFO de recepción. `ERROR_OK` es el reconocimiento equivalente para las banderas de error. En sentido contrario, `RX_EMPTY` indica si queda algo por leer y `TX_RDY` que el transmisor ha terminado el byte en curso y admite el siguiente.

Descubrimiento del mapa de direcciones. Ninguna dirección de canal está escrita en el código. En la dirección fija `0xA0020000` se sitúa un AXI GPIO adicional, de solo lectura. Sus dos entradas están cableadas en la PL a constantes que el script de generación calcula al construir el *bitstream*. Ese bloque de información publica tres valores:

- **Número de transceptores** instanciados, N , en los bits 15:0 del canal 1.
- **Stride**, en los bits 31:16 del canal 1: la distancia en bytes entre la dirección base de una instancia y la de la siguiente, 4 KB en este diseño.
- **Dirección base** del primer transceptor, en el canal 2.

Las instancias se asignan de forma contigua a partir de esa base, y el AXI INTC compartido por todas ellas queda inmediatamente después de la última. Con los tres valores, leídos al arrancar en `Transceiver_Global_Init()`, el driver deduce el resto del mapa con dos cuentas:

$$\text{base}_i = \text{base} + i \cdot \text{stride}, \quad i = 0, \dots, N - 1 \quad \text{base}_{\text{INTC}} = \text{base} + N \cdot \text{stride}$$

De este modo el mismo binario RTEMS funciona sobre cualquier *bitstream*, desde un único canal hasta los catorce, sin recompilar. Basta con que el hardware declare cuántas instancias trae y dónde empiezan.

Bit SL0. El bit SL0 del registro de escritura no actúa sobre la lógica en VHDL: se propaga hasta el pin SL0 del chip transceptor de la PCB. El driver se limita a exponerlo. Su efecto eléctrico y la cuantificación de su impacto sobre la velocidad máxima del enlace se tratan en la caracterización de la placa (sección 4.3.3).

3.4.3. Gestión interna de la transmisión

La figura 3.3 resume el recorrido interno de un byte en cada sentido. Esta subsección se ocupa de la transmisión, y la siguiente, de la recepción.

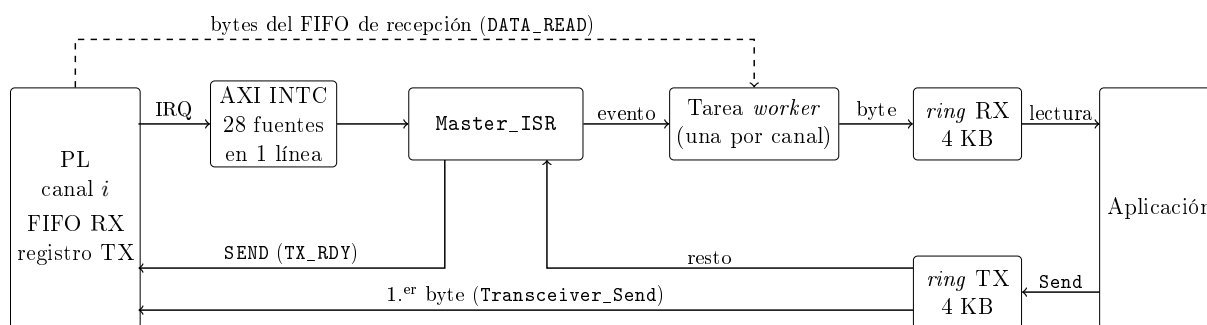


Figura 3.3: Camino de un byte entre la lógica programable y la aplicación.

Cada canal dispone de un *buffer* circular de salida de 4 KB, tamaño que da margen para ráfagas a las tasas más altas soportadas, hasta 4 Mbaudios. `Transceiver_Send()` copia ahí los bytes de la aplicación. Si el transmisor estaba en reposo, escribe el primero directamente en el registro de escritura para poner en marcha el hardware. La tarea que llama no se bloquea.

El resto de la ráfaga lo va sacando la rutina de interrupción. Los catorce canales comparten un único controlador AXI INTC, que concentra sus veintiocho fuentes de interrupción (la de recepción y la de transmisión de cada canal) en una sola línea hacia el PS. Una única rutina, `Master_ISR`, atiende esa línea. Cuando el transmisor de un canal termina de emitir un byte, activa su señal `TX_RDY`. La `Master_ISR` lee el registro de interrupciones pendientes y localiza el canal en la tabla de instancias `g_instances[]` mediante un índice directo, sin listas enlazadas ni búsquedas en contexto de interrupción. Toma entonces el siguiente byte de su cola y lo escribe. Cuando la cola queda vacía, marca el canal como libre.

En transmisión, por tanto, la rutina de interrupción sí mueve el dato, pero es la escritura de un único byte por interrupción. Su coste está acotado y no compromete la latencia del sistema. Un semáforo binario serializa las llamadas concurrentes de varias tareas a `Transceiver_Send()` sobre el mismo canal. El acceso al *buffer* frente a la `Master_ISR` se protege inhibiendo un instante las interrupciones durante el encolado.

3.4.4. Gestión interna de la recepción

La recepción sigue el patrón de procesamiento diferido recomendado en RTEMS: la rutina de interrupción hace lo imprescindible y delega el grueso del trabajo en una tarea ordinaria. Esa tarea, la *worker*, es una tarea de RTEMS: un hilo con su propia pila y prioridad que el planificador del sistema operativo de tiempo real gestiona junto con las demás. Se ejecuta cuando le corresponde, cede la CPU mientras espera y no carga al núcleo más que con un cambio de contexto. Hay una `Rx_Worker_Task` por canal, creada en `Transceiver_Init()` y bloqueada a la espera de trabajo.

Cuando llega un byte por la línea, el receptor del canal genera una interrupción. La `Master_ISR` la reconoce, inhibe temporalmente esa fuente en el `INTC` y envía un evento `RTEMS` a la `Rx_Worker_Task` del canal. Ahí acaba el trabajo en contexto de interrupción. La tarea despierta y vacía el FIFO del hardware byte a byte hacia el *buffer* circular de recepción, también de 4 KB, confirmando cada extracción con el bit `DATA_READ`. Al agotar el FIFO invoca el *callback* que la aplicación haya registrado, vuelve a habilitar la interrupción y se bloquea de nuevo. La aplicación recoge los bytes cuando puede con `Transceiver_Read()`. Un semáforo binario por canal protege el *buffer* de recepción frente al acceso simultáneo de la *worker* y la aplicación.

Esa copia, de duración variable, corre ya como tarea sujeta al planificador y no en la rutina de interrupción. Por ello, la latencia de interrupción del sistema no depende del volumen de datos que llegue por las líneas serie.

3.5. Diseño hardware

Una vez completado el firmware del transceptor serie, se desarrolló el hardware de prueba necesario para validar el sistema en condiciones reales. El plan inicial era diseñar una única placa de comunicación serie para ejercitar múltiples líneas a la vez. Posteriormente, el proyecto LINCE requirió dos tarjetas adicionales con especificaciones impuestas por Indra: la placa CDHS y la placa AOCS. Estas placas permitieron a Indra validar su firmware sobre la ZCU102. Los esquemáticos (PDF) y las BOM de las tres placas están disponibles en el repositorio del proyecto [5] bajo `tfm/00_docs/pcbs/`.

3.5.1. Selección de componentes bajo la restricción de 1,8 V

La selección de componentes de las tres placas está condicionada por un límite eléctrico de la plataforma.

Los bancos de entrada/salida del conector FMC HPC empleados por estas tarjetas operan a **1,8 V**: es el nivel del carril `VADJ` con el que la ZCU102 alimenta la interfaz. Todo componente que se conecte directamente a esos pines debe, por tanto, admitir una *VIO* (voltaje de bus) de 1,8 V. La *VIO* es la tensión con la que se alimenta el banco de entrada/salida digital de un circuito integrado, independiente de la de su núcleo. Fija los niveles lógicos que la pieza acepta y entrega en sus pines. La alternativa, interponer adaptadores de nivel entre el MPSoC y cada transceptor, añade componentes activos en el camino de señal. Introduce además retardo de propagación en líneas que conmutan a varios megabaudios, y multiplica los puntos de fallo. Se descartó salvo donde fue inevitable, y se buscaron deliberadamente componentes con *VIO* nativo a 1,8 V.

El criterio de partida no fue la cualificación formal para espacio. El componente cualificado es la pieza *rad-hard*, con un coste unitario y unos plazos de suministro que no encajan en un programa NewSpace como LINCE. La práctica habitual en este ámbito es otra: emplear componentes de catálogo, no cualificados oficialmente, que acreditan **herencia de vuelo** por haberse utilizado ya en misiones anteriores.

La búsqueda se orientó por tanto a piezas de catálogo con herencia de vuelo, y no dio resultado. Los transceptores de bus que la acreditan pertenecen a una generación tecnológica anterior y su interfaz digital opera a 3,3 V o 5 V. No se encontró ninguno que admitiese una *VIO* de 1,8 V, de modo que la herencia de vuelo y el nivel de la plataforma

resultaron incompatibles.

Al no encontrarse componentes con herencia de vuelo, se buscó el **grado automotriz** (cualificación AEC-Q100¹). Esta cualificación impone rangos de temperatura extendidos y niveles de robustez muy superiores a los de grado comercial. Es el escalón al que se recurre en misiones de bajo coste cuando no hay una pieza de catálogo con herencia de vuelo que cumpla los requisitos eléctricos. Los transceptores TCAN1044AVDRQ1 del bus CAN y el conversor analógico-digital ADS7950QDBTRQ1 de la placa CDHS responden a este criterio. Lo mismo ocurre con el transceptor THVD1424RGTR de los canales serie de las placas CDHS y AOCS, elegido por soportar *VIO* de 1,8 V de forma nativa.

A estas tarjetas se les exige la **validación funcional** del subsistema de comunicaciones, no ser un **modelo de vuelo**. El grado automotriz permitió ejercitar el subsistema en condiciones eléctricas realistas dentro del coste y del plazo del trabajo.

La búsqueda deja además un dato aprovechable para el modelo de vuelo. Ningún transceptor de catálogo con herencia de vuelo opera su interfaz digital a 1,8 V. Ese modelo tendrá que renunciar al nivel nativo de la plataforma y adaptar la interfaz a piezas de 3,3 V o 5 V. Para ello, deberá cambiar la tensión del carril VADJ o interponer adaptadores de nivel.

3.5.2. Reglas de diseño y proceso de fabricación

Las tres placas se diseñaron en **Altium Designer**. En los tres proyectos el primer paso, antes de colocar ningún componente, fue definir las **reglas de diseño**. Altium las comprueba de forma continua mientras se rutan las pistas. Definirlas al principio hace que una restricción de fabricación incumplida salte como error en ese momento, y no como un rechazo del fabricante con el diseño ya cerrado.

El criterio adoptado fue ceñirse a las **capacidades del proceso de fabricación del fabricante seleccionado**, PCBWay [22], en lugar de definir un conjunto de reglas propio. Los límites de su proceso estándar (anchuras y separaciones mínimas de pista, diámetro mínimo de taladro, anillo de vía y distancia mínima al borde de placa) son los que acotan lo que es fabricable a coste ordinario. Apurarlos por debajo obliga a recurrir a procesos de precisión más caros, y mantenerse muy por encima de ellos no aporta ninguna ventaja. Diseñar directamente contra esas capacidades garantiza que las tres placas fueran fabricables en la primera tirada, como en efecto ocurrió.

Sobre esas reglas generales se impusieron dos restricciones adicionales propias de las señales que transportan estas placas. La primera es el **rutado en par diferencial** con impedancia controlada para las líneas SPW, RS422/RS485 y CAN. La segunda es la **longitud acotada** en las líneas de conmutación rápida procedentes del FMC, de manera que las señales tarden lo mismo en llegar a su destino.

3.5.3. Diseño de la placa de comunicación serie

El objetivo de esta placa era poder ejercitar simultáneamente los 14 transceptores del IP core, replicando las topologías de bus reales que se encontrarán en el satélite. Estas

¹Norma del *Automotive Electronics Council* que somete a los circuitos integrados a ensayos de estrés (ciclado térmico, humedad, vida acelerada) para garantizar su operación en el rango de temperatura y fiabilidad exigidos por la industria del automóvil.

son RS485 multipunto, con múltiples nodos en el mismo cable, y RS422 en configuración *master/slave*, con cruce de TX y RX. A diferencia de las placas CDHS y AOCS, cuyas especificaciones vinieron dictadas por Indra, esta placa fue diseñada con total libertad para maximizar la flexibilidad de los ensayos en laboratorio.

El transceptor seleccionado para los catorce canales es el **LTC2865** [23] de Analog Devices, un transceptor RS485/RS422 de hasta 20 Mbps. Su pin de alimentación lógica independiente (VL) le permite operar la interfaz digital directamente a los 1,8 V del FMC, tal como exige la sección 3.5.1. Expone además un pin **SL0** de limitación de *slew* controlable desde el MPSoC, que tendrá un papel protagonista en la caracterización de la placa (sección 4.3.3). Todas las líneas diferenciales incorporan diodos TVS **SM712-02HTG** contra transitorios. Cada driver dispone además de una resistencia de terminación de 120 Ω conmutable mediante *jumper*, lo que permite terminar el bus en los nodos extremos y dejar abiertos los intermedios.

La arquitectura general de la placa divide los 14 canales en dos grupos conectados al FMC:

- **7 drivers RS485 (RS0–RS6)** conectados a un bus diferencial común mediante *jumpers*, formando una topología multipunto configurable.
- **7 drivers RS422 (RS7–RS13)** organizados en dos buses *master/slave* independientes (BUS 1: 1 *master* + 3 *slaves*; BUS 2: 1 *master* + 2 *slaves*), con la línea TX cruzada con RX para emular la conexión real punto a punto RS422.

La hoja de nivel superior del esquemático (figura 3.4) recoge esta arquitectura: el conector FMC en el centro, el bus común RS485 (RS0–RS6) a un lado y los dos buses *master/slave* RS422 (RS7–RS13) al otro. Cada bus sale por sus propios conectores Dupont y Micro-D. El resultado físico del diseño se aprecia en el render de la figura 3.5.

Topología RS485 con bus compartido configurable por *jumper*

La decisión de diseño más relevante de esta placa es el mecanismo de bus compartido RS485 sin necesidad de recablear. Cada driver RS485 tiene un conector Dupont de 3 pines (A+, B–, GND) que actúa a la vez como punto de conexión hacia el exterior y como punto de interconexión entre drivers adyacentes. Colocando un *jumper* entre dos conectores consecutivos, las líneas A y B de ambos drivers quedan físicamente unidas, formando un segmento de bus RS485 compartido. Retirando el *jumper*, el driver queda independiente y puede conectarse a un periférico externo de forma individual.

Este mecanismo permite configurar por hardware cualquier partición de los 7 drivers RS485 en uno o varios buses, sin modificar el firmware ni el cableado exterior.

Selector de conector Micro-D para RS485

Los conectores de mayor densidad (Micro-D) de la placa son compartidos entre el Driver 0 y el Driver 6 mediante un *jumper* de selección de hardware. Dependiendo de la posición del *jumper*, los conectores Micro-D quedan asignados a uno u otro driver. Esto permite usar un único tipo de conector con cableado de satélite, sin duplicar el número de conectores en la placa.

TOP.SchDoc

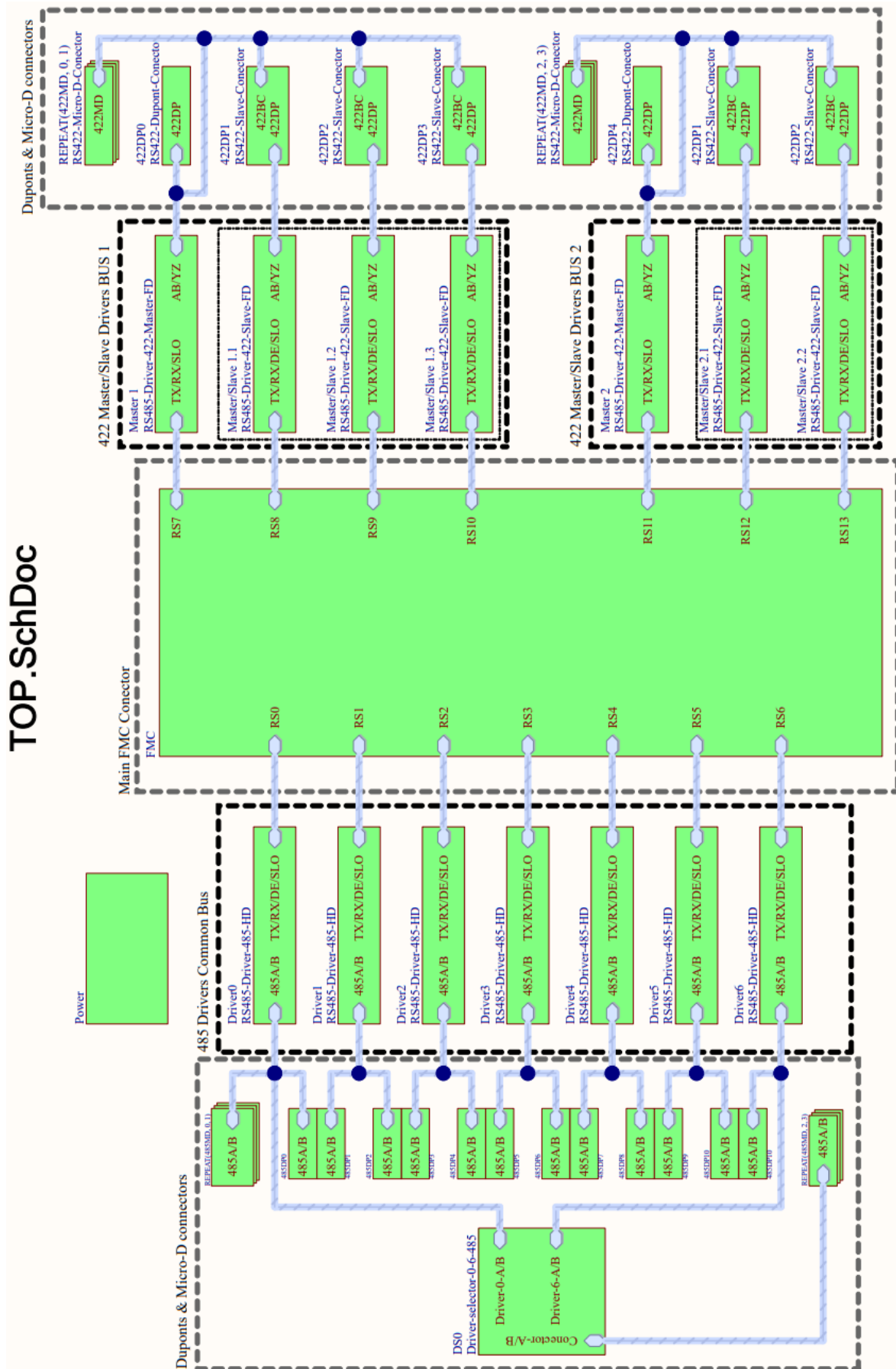
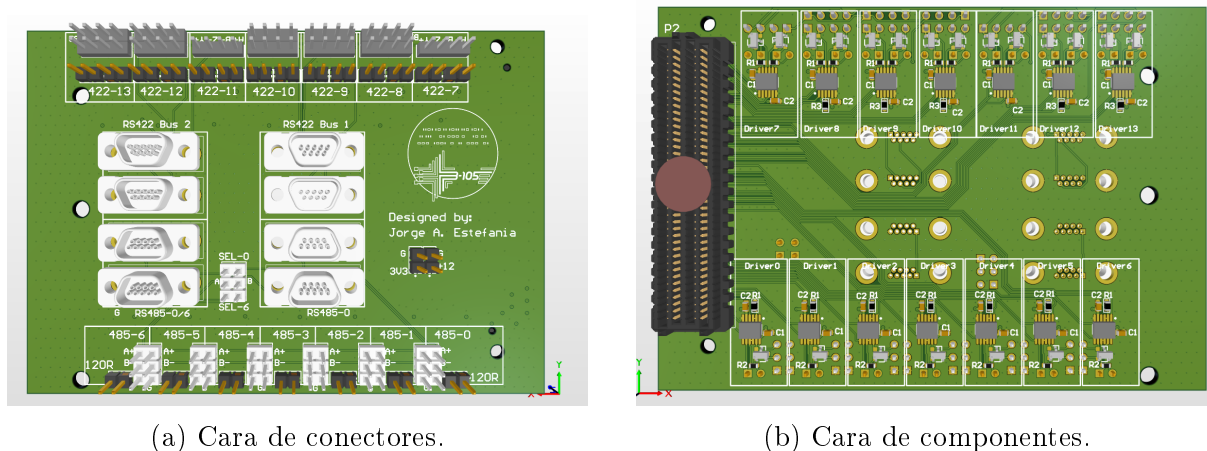


Figura 3.4: Hoja de nivel superior del esquemático de la placa de comunicación serie.



(a) Cara de conectores.

(b) Cara de componentes.

Figura 3.5: Render 3D de la placa de comunicación serie.

Topología RS422 maestro/esclavo configurable por *jumper*

Para RS422, cada driver *slave* dispone de un conector con las líneas TX cruzadas a RX (TX-Y+/TX-Z- conectadas a RX del bus). Esta disposición replica la conexión estándar punto a punto RS422, donde el receptor del esclavo escucha las transmisiones del *master*. Al igual que en RS485, un *jumper* determina si el driver se une al bus común o permanece independiente.

Alimentación

La placa se alimenta íntegramente del conector FMC, que suministra los carriles de 12 V, 3,3 V y VADJ a 1,8 V. Los transceptores LTC2865 operan con la etapa de bus a 3,3 V (VCC) y la interfaz lógica a 1,8 V (VL). A diferencia de las placas CDHS y AOCS, que generan con un convertidor DC/DC los 5 V de su etapa de potencia, esta placa no necesita ninguna etapa de conversión. Todos los carriles llegan directamente del FMC. Sendos conectores de dos pines dan acceso externo a los carriles de 12 V y 3,3 V.

El mapa completo de señales entre esta placa y el conector FMC se encuentra en el Anexo C de este documento. El esquemático completo (12 hojas, PDF) y la BOM están disponibles en `tfm/00_docs/pcbs/serial/` del repositorio del proyecto.

3.5.4. Diseño de la placa CDHS

La placa LINCE3 CDHS BreakoutBox es una tarjeta de interconexión entre la ZCU102 (conectada por FMC) y los periféricos del subsistema de gestión de datos y calentadores del satélite. Sus especificaciones fueron definidas por Indra: 6 conectores D-Sub-9 (J1-J6), interfaces CAN redundante, tres canales RS422/RS485, cuatro líneas PWM para calentadores y un ADC para termistores. El esquemático se organizó en un bloque jerárquico por subsistema, con el reparto que recoge la figura 3.6. El resultado físico del diseño se aprecia en el render de la figura 3.7.

Nivel de tensión y selección de componentes

Como en las otras dos tarjetas, todos los bancos de pines del conector FMC HPC utilizados por esta placa operan a 1,8 V. Los componentes se seleccionaron con *VIO* nativo a ese

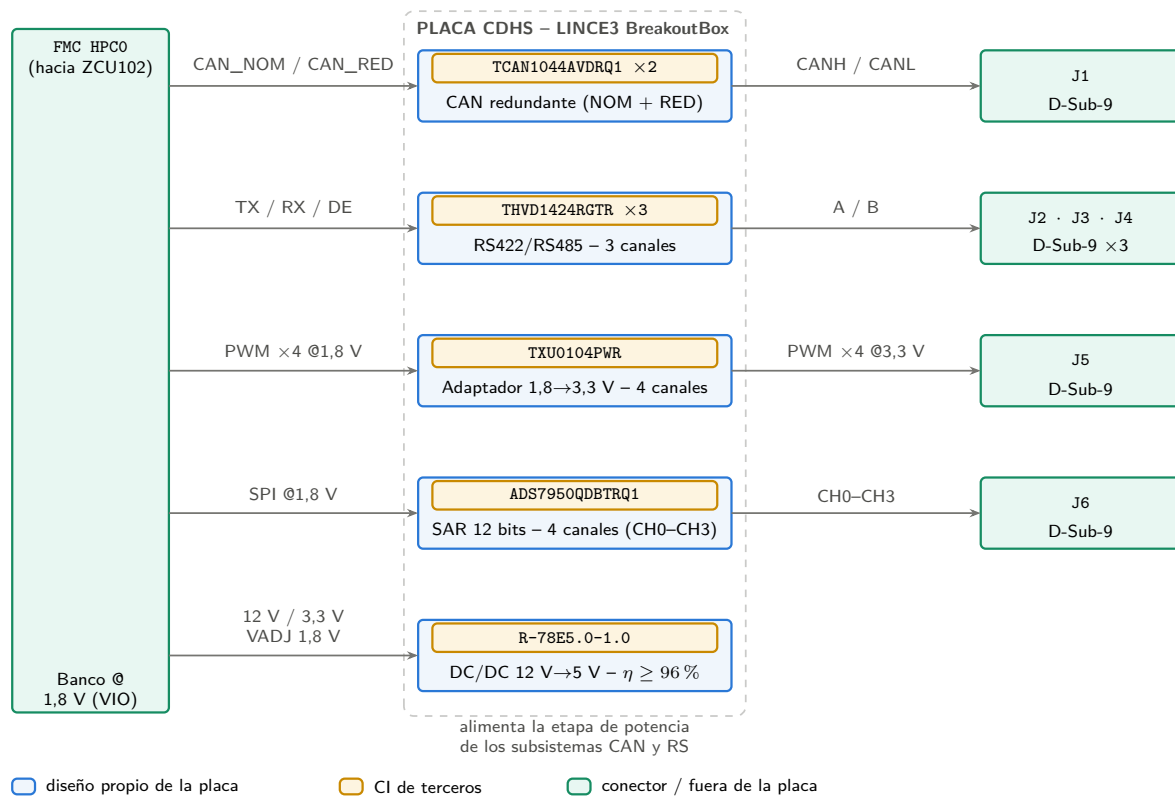


Figura 3.6: Diagrama de bloques de la placa CDHS.

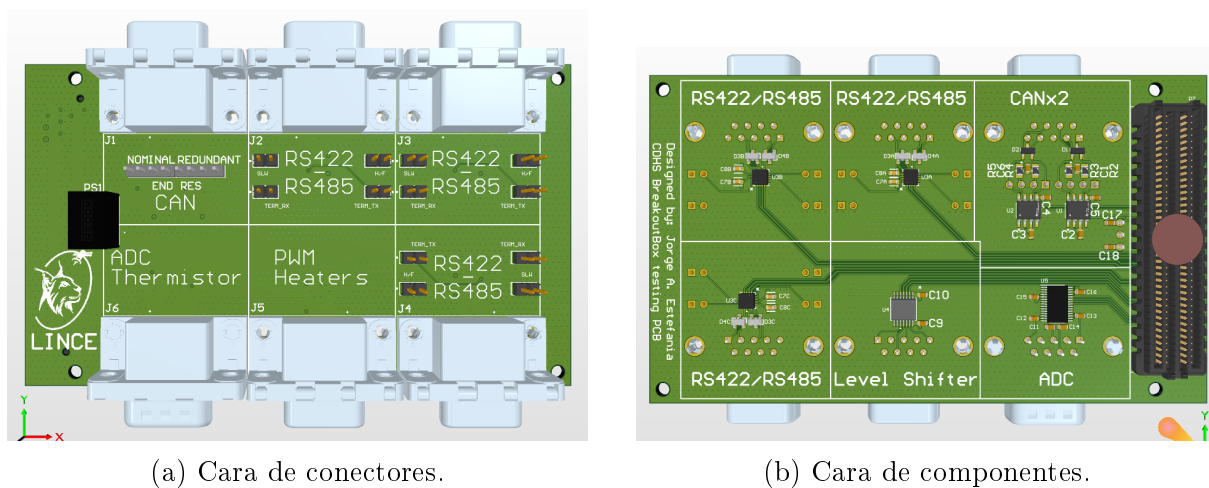


Figura 3.7: Render 3D de la placa CDHS.

nivel, para evitar etapas de adaptación en la cadena de señal. Las implicaciones de esta restricción, y el motivo por el que obligó a descartar los componentes con herencia de vuelo, se discuten en la sección 3.5.1. Los subsistemas que siguen aplican ese criterio, y las excepciones (las señales de PWM, que sí requieren adaptación de nivel) se señalan donde corresponde.

Subsistema CAN

El bus CAN implementa topología redundante con dos instancias independientes: CAN Nominal (CAN_NOM) y CAN Redundante (CAN_RED). Se seleccionó el transceptor **TCAN1044AVDRQ1** de Texas Instruments por su compatibilidad nativa con VIO de 1,8 V, grado automotriz y baja corriente en modo *standby*. Ambos canales convergen en el conector J1.

Las resistencias de terminación siguen el esquema *split termination* recomendado en las notas de aplicación del estándar CAN (ISO 11898-2). En lugar de una única resistencia de 120 Ω entre CANH y CANL, se emplean **dos resistencias de 60 Ω en serie** con un **condensador de 4,7 nF** conectado desde el punto medio al plano de masa. Cada una de las dos resistencias de 60 Ω está controlada por un *jumper* independiente, lo que permite habilitar o deshabilitar la terminación sin modificar el circuito. La resistencia equivalente sigue siendo 120 Ω cuando ambos *jumpers* están colocados. El condensador de derivación crea un camino de baja impedancia para las interferencias de modo común de alta frecuencia hacia masa, lo que mejora el comportamiento EMC del bus.

Las líneas CANH y CANL de cada canal incorporan diodos de protección ESD **ESDCAN24-2BLY**, específicamente diseñados para buses CAN. Clampean los transitorios por encima de su tensión de ruptura sin introducir una capacitancia parásita significativa. La figura 3.8 recoge el circuito completo de los dos canales.

Subsistema RS422/RS485

Se disponen tres canales serie idénticos e independientes (RS1, RS2, RS3), cada uno con el transceptor **THVD1424RGTR**. Este integrado, sugerido dentro del proyecto LINCE con posterioridad al diseño de la placa de comunicación serie propia, mejora al LTC2865 empleado en aquella. Además de soportar VIO de 1,8 V, incorpora resistencias de terminación internas conmutables y permite seleccionar entre modo *half-duplex* y *full-duplex* sin cambiar el hardware. Por estas ventajas se adoptó para las dos tarjetas oficiales. La configuración se expone mediante *jumpers* físicos de 2 pines: **H/F** (conmuta entre *half-duplex* y *full-duplex*), **SLR** (habilita control de *slew rate*), **TERM_TX / TERM_RX** (conectan las resistencias de terminación internas). Los tres canales salen por los conectores J2, J3 y J4.

Tanto las líneas RX como las TX de cada canal llevan un par de diodos TVS **SM712-02HTG** conectados entre las líneas diferenciales y el plano de masa.

Cortocircuito DE–RE. El THVD1424 expone dos pines de control independientes: **DE** (*Driver Enable*, activo en alto) habilita el transmisor, y **RE** (*Receiver Enable*, activo en bajo) habilita el receptor. En el esquemático, ambos pines están conectados a la misma señal de control procedente del MPSoC. Una única línea digital controla así, de forma complementaria, transmisor y receptor (tabla 3.5).

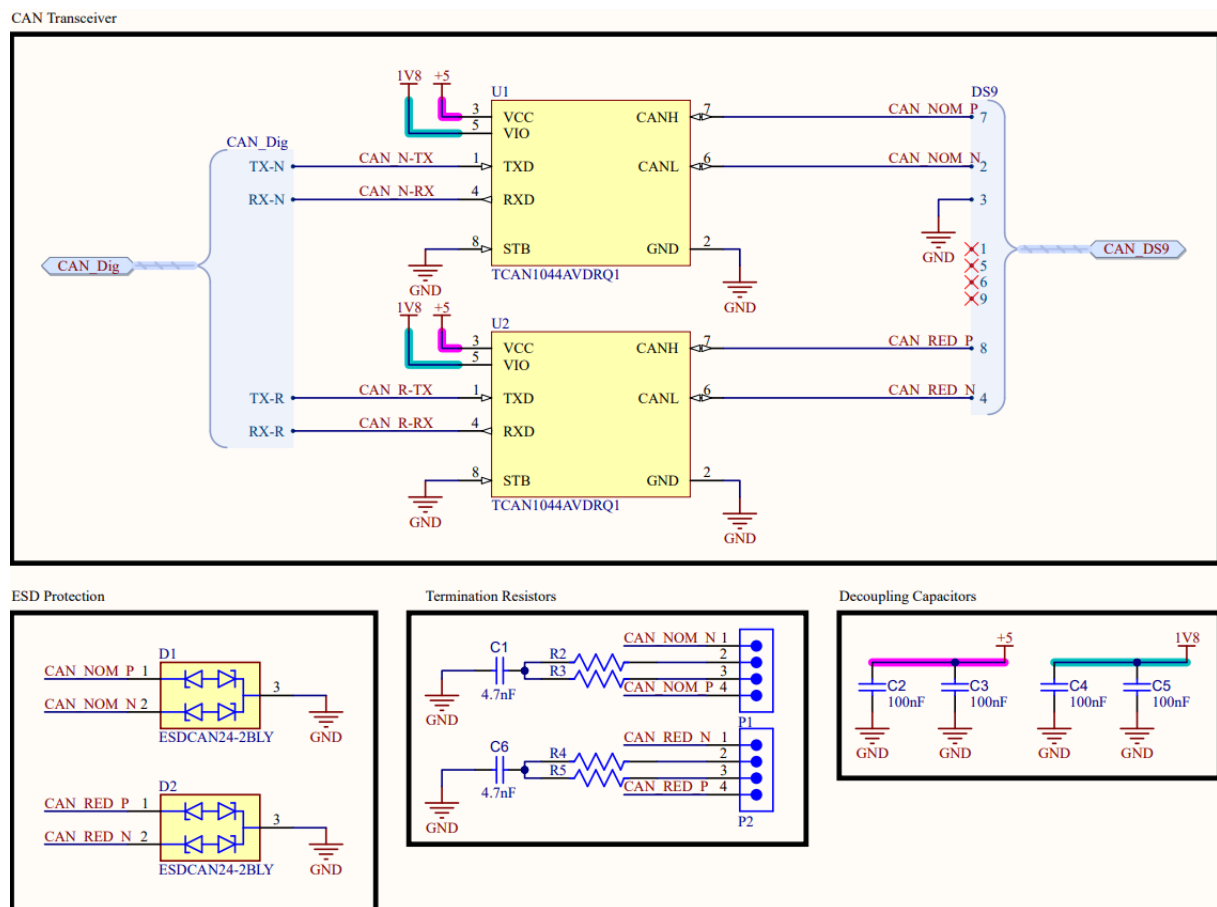


Figura 3.8: Subsistema CAN de la placa CDHS.

Señal DE/RE	Transmisor (DE)	Receptor (RE activo bajo)
1 (alto)	Habilitado	Deshabilitado
0 (bajo)	Deshabilitado	Habilitado

Tabla 3.5: Control complementario DE/RE en el THVD1424.

Esta decisión responde a dos motivos. Por un lado, el equipo de Indra comunicó que en la arquitectura del satélite los esclavos solo transmiten bajo demanda del OBC. Por ello, no existe un escenario real en el que un nodo deba transmitir y recibir simultáneamente. Por otro lado, en modo *half-duplex* las líneas TX y RX comparten físicamente el mismo par diferencial A/B. Si el receptor permaneciera activo durante la transmisión, el nodo escucharía su propio eco. El circuito completo de un canal, con esa conexión y los cuatro *jumpers* de configuración, es el de la figura 3.9.

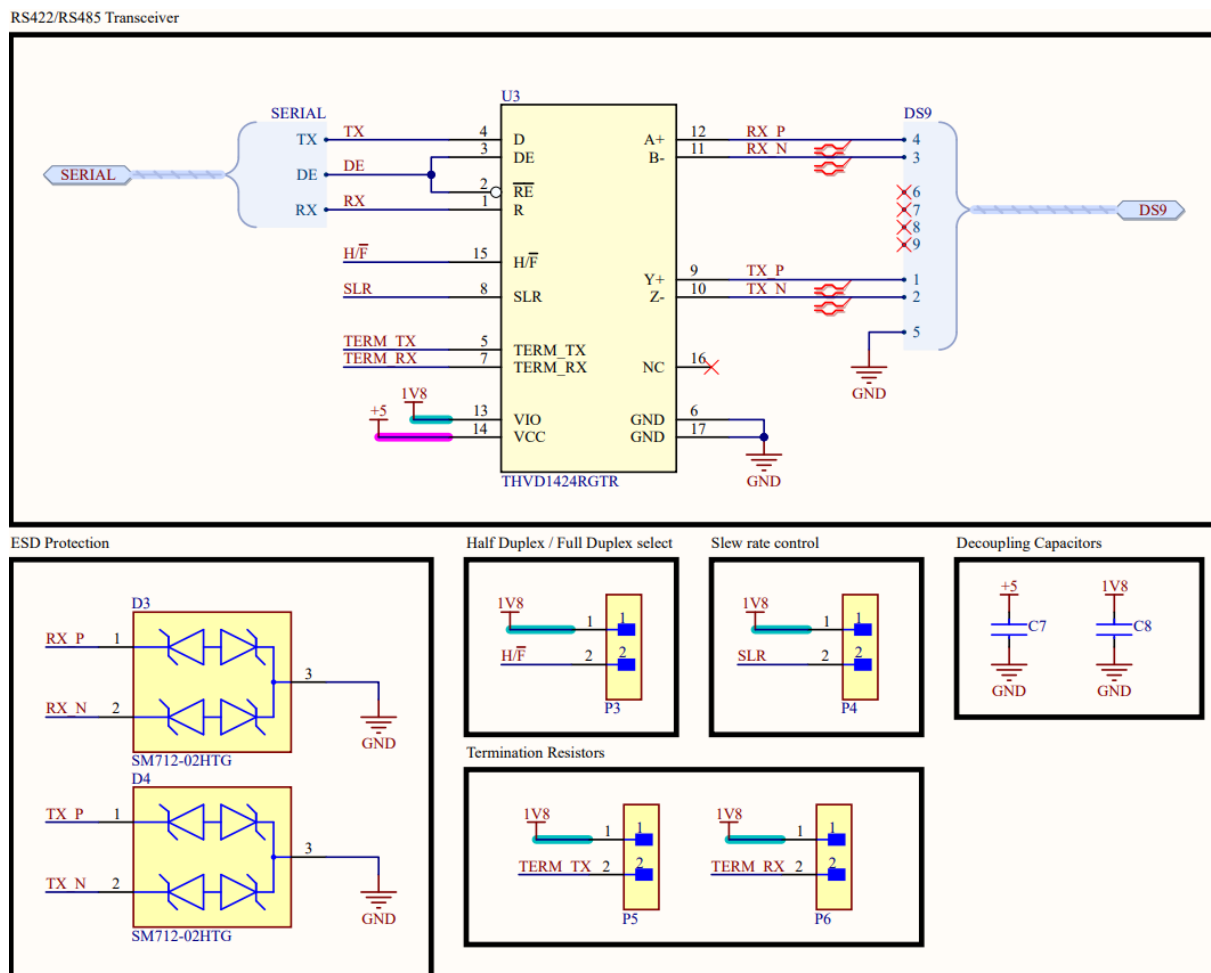


Figura 3.9: Canal RS de la placa CDHS con el THVD1424RGTR.

Subsistema PWM (calentadores)

Cuatro señales PWM de 1,8 V procedentes del FMC se elevan a 3,3 V mediante el adaptador de niveles **TXU0104PWR** para excitar los circuitos de control de calentadores externos. Las cuatro líneas adaptadas salen por J5.

Para la validación de esta interfaz se diseñó un bloque VHDL autónomo, `PWMx4_auto_test`, sintetizado en la PL junto al resto del diseño de Vivado. El bloque genera cuatro señales PWM independientes con *duty cycle* del 50 % a partir del reloj de 100 MHz de la FPGA: canal 0 a 10 kHz, canal 1 a 5 kHz, canal 2 a 1 kHz y canal 3 a 100 Hz. Al ser completamente autónomo no requiere ningún driver ni intervención del PS, lo que permitió verificar el subsistema PWM de la placa con el mismo BOOT.BIN utilizado para las pruebas serie.

Subsistema ADC (termistores)

Cuatro entradas analógicas de termistores (CH0–CH3) se digitalizan con el ADC SAR de 12 bits **ADS7950QDBTRQ1**, comunicado al procesador por SPI a 1,8 V. La circuitería analógica opera a 3,3 V. Ambos carriles (+VBD a 1,8 V y +VA a 3,3 V) son independientes, para evitar acoplos entre el dominio digital y el analógico [24]. Los termistores se conectan por J6.

Alimentación

El conector FMC suministra 12 V, 3,3 V y el carril VADJ a 1,8 V. El convertidor conmutado **R-78E5.0-1.0** genera los 5 V necesarios para la etapa de potencia de los transceptores RS y CAN a partir de los 12 V del FMC. Se eligió este módulo DC/DC por su integración en un *footprint* reducido de 3 pines (equivalente a un regulador lineal) y su eficiencia $\geq 96\%$, evitando la disipación térmica que tendría un LDO con ese diferencial de tensión.

Conectores externos

Los seis puertos D-Sub-9 (J1–J6) son de tipo macho o hembra según la convención de uso. Los escudos metálicos de todos los conectores están conectados al plano de GND para mantener la continuidad de apantallamiento EMI con los cables blindados.

El mapa completo de señales entre la placa CDHS y el conector FMC se encuentra en el Anexo C de este documento. El esquemático (PDF) y la BOM están disponibles en `tfm/00_docs/pcbs/cdhs/`.

3.5.5. Diseño de la placa AOCS

La placa LINCE3 AOCS BreakoutBox es la tarjeta de interconexión para el subsistema de control de actitud y órbita del satélite. Al igual que la CDHS, conecta la ZCU102 mediante FMC y expone las interfaces al exterior a través de 8 conectores D-Sub-9 (J1–J8). La figura 3.10 muestra cómo se reparten esas ocho salidas entre los distintos subsistemas, y la figura 3.11 recoge el aspecto físico de la tarjeta.

Lo que se describe a continuación es la **revisión V2** del diseño, que es la versión cerrada del esquemático. Las dos unidades fabricadas y toda la validación recogida en la sección 4.2 corresponden a la **revisión anterior**, de cinco canales serie. La V2 no llegó a fabricarse. Las diferencias entre ambas revisiones son cuatro: el desdoble del canal de J4 en dos, la separación de las líneas DE y RE, la adición de puntos de prueba y la corrección de un error de conexión en el eje Z del bloque de motores. Las tres primeras se detallan en el subsistema RS que sigue, y la cuarta en el de MOT-PWM.

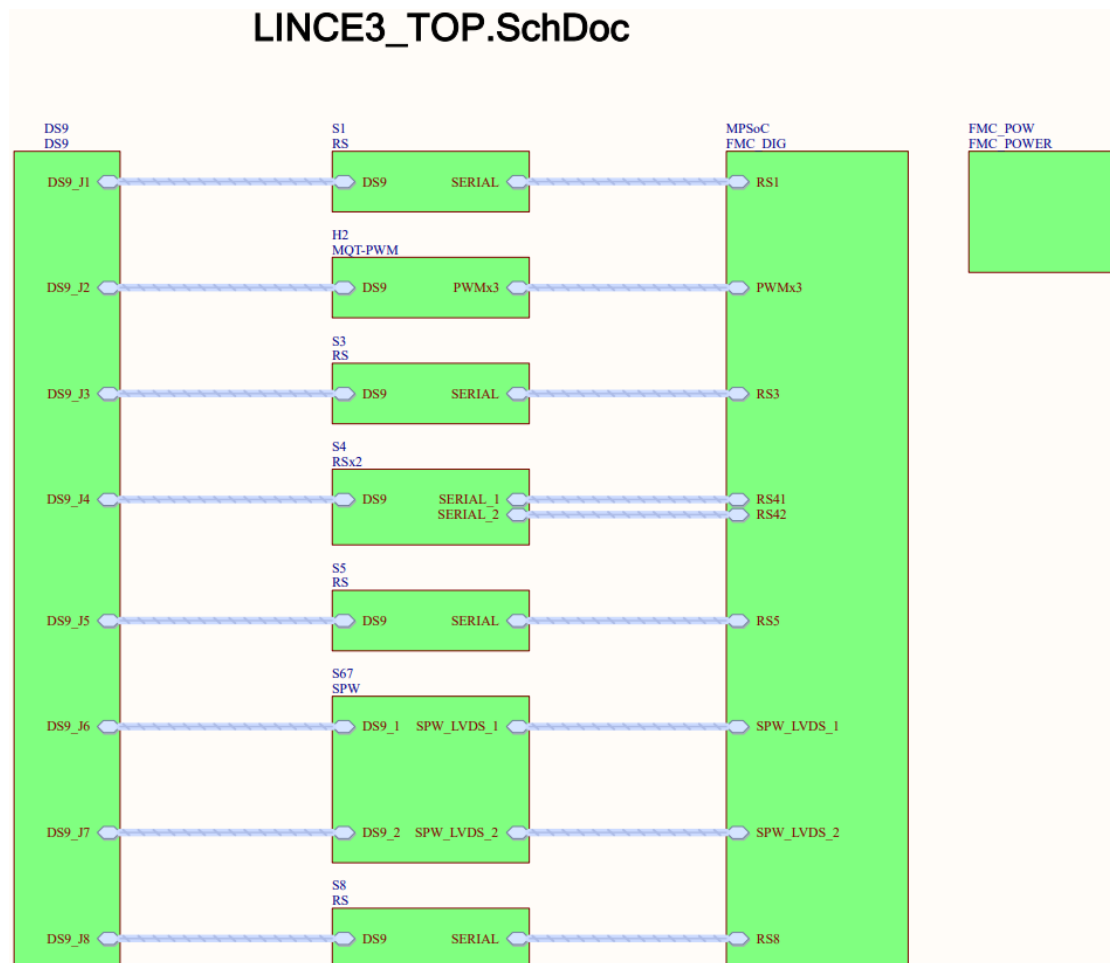
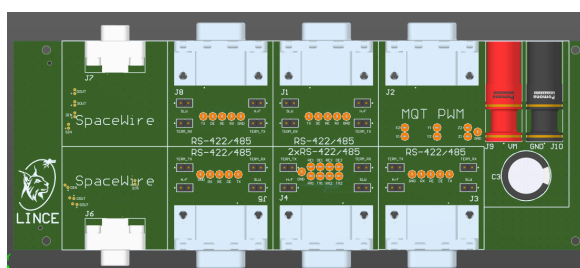
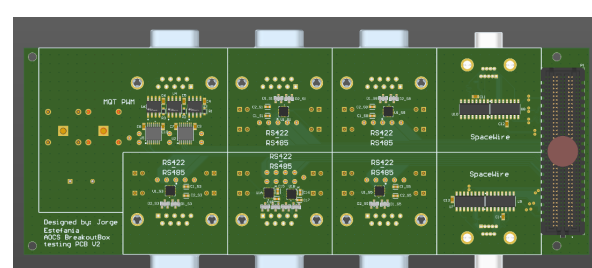


Figura 3.10: Hoja de nivel superior del esquemático de la placa AOCS.



(a) Cara de conectores.



(b) Cara de componentes.

Figura 3.11: Render 3D de la revisión V2 de la placa AOCS.

Subsistema RS422/RS485

La placa incorpora **seis canales serie** (RS1, RS3, RS41, RS42, RS5 y RS8) con el mismo transceptor THVD1424RGTR descrito en la sección de la placa CDHS. Usa los mismos *jumpers* de configuración H/F, SLR y terminación. Los canales RS41 y RS42 comparten el conector J4: la hoja RSx2 monta los dos transceptores de ese conector. Los seis canales salen así por cinco D-Sub-9 (J1, J3, J4×2, J5 y J8). Frente a la revisión anterior, de cinco canales, el desdoble permite ejercitar un par *master/slave* completo sobre un único conector.

Respecto al diseño de la CDHS hay una diferencia de fondo: en esta placa las líneas **DE y RE llegan por separado** desde el MPSoC, en lugar de cortocircuitadas. Cada canal consume así cuatro señales digitales del FMC (TX, RX, DE y RE) en vez de tres. El motivo es que el cortocircuito DE-RE fija el comportamiento en el cobre: obliga a que el receptor esté siempre en el estado complementario del transmisor, que es lo que la arquitectura del satélite necesita. Sin embargo, impide comprobar en banco cualquier otra combinación. Con las dos líneas independientes, el firmware puede seguir imponiendo el comportamiento complementario por software. Además, permite habilitar receptor y transmisor a la vez para observar el eco propio en *half-duplex*, o dejar ambos deshabilitados para verificar que la línea queda en alta impedancia.

Puntos de prueba. La revisión V2 añade **puntos de prueba** en las cuatro líneas lógicas de cada canal (TX, DE, RE y RX), más una toma de masa por bloque. Son accesibles desde la cara de conectores, junto a los *jumpers* (figura 3.11). El bloque de PWM de motores lleva los suyos sobre los seis pares X1/X2, Y1/Y2 y Z1/Z2, también con masa propia. La razón es la depuración. Los caracteres corruptos que motivaron los estados PreDE y PostDE del transmisor se diagnosticaron pinchando el osciloscopio en el lado lógico del transceptor. En la revisión anterior, esas líneas solo eran accesibles en las patas del encapsulado. Una toma de masa contigua a cada grupo de señales es parte del mismo objetivo: el lazo de masa de la punta es lo que limita la fidelidad de la medida en flancos de varios megabaudios.

Subsistema SpaceWire

Se implementan dos enlaces SpaceWire *full-duplex* independientes (SPW1 y SPW2) mediante señalización diferencial LVDS. El estándar SpaceWire opera directamente a niveles LVDS compatibles con los bancos del FMC. Ambos enlaces son idénticos y atraviesan **repetidores LVDS DS90CP22M**, como muestra la figura 3.12. Cada enlace lleva dos repetidores de dos canales: uno para el sentido entrante (los pares DIN y SIN) y otro para el saliente (SOUT y DOUT), lo que suma cuatro integrados en la placa. El repetidor reacondiciona los flancos y aísla el tramo de cobre de la tarjeta del cable externo, que es donde el enlace se degrada. Su coste es un retardo de propagación fijo, irrelevante frente a la temporización de SpaceWire. La simetría entre los dos enlaces permite además atribuir al cableado, y no a la placa, cualquier diferencia que se observe entre ellos en banco. Los pares diferenciales (cuatro por enlace: DIN, SIN, SOUT y DOUT) se han trazado en la PCB con técnicas de igualación de longitud (*length matching*), para minimizar el *skew* entre el par de datos y el par de *strobe*. También se ha controlado su impedancia característica, de 100 Ω diferencial.

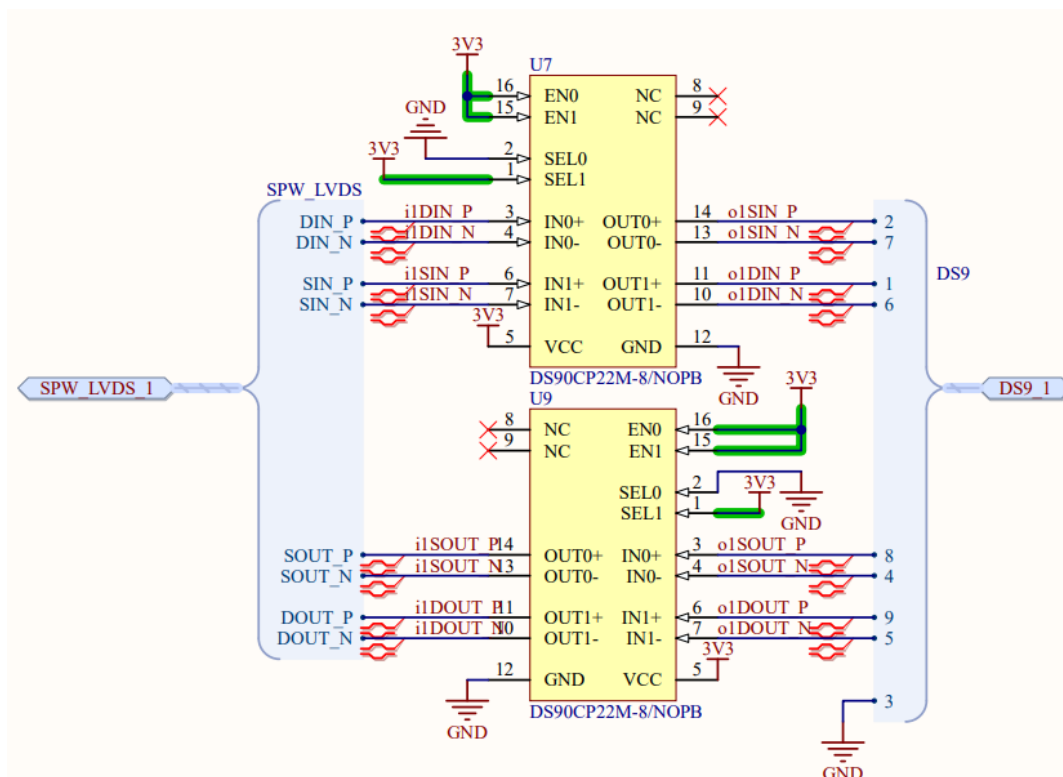


Figura 3.12: Subsistema SpaceWire de la placa AOCS.

Subsistema de control de motores (MOT-PWM)

Seis señales PWM de 1,8 V procedentes del FMC se elevan al nivel de tensión requerido por los puentes en H externos mediante un adaptador de niveles. Las señales se agrupan en tres pares (X, Y, Z para los tres ejes del satélite), con dos líneas por eje para controlar la dirección de giro del motor. Salen por J2. La alimentación de potencia de los motores se conecta mediante dos bornes banana independientes de la alimentación de la placa.

Los seis pares se reparten entre dos adaptadores TXU0104PWR de cuatro canales: U2 lleva los ejes X e Y, y U5 el eje Z, que ocupa solo dos de sus cuatro canales.

Corrección de la revisión V2: el eje Z. La revisión fabricada tenía un **error de conexión en el eje Z** que la V2 corrige. En U5, las señales PWM_Z_1 y PWM_Z_2 entraban por A2 y A1, cuyas salidas correspondientes eran B2Y y B1Y. Sin embargo, las entradas del puente en H de ese eje (U6) no colgaban de B2Y y B1Y, sino de **B4Y y B3Y**, que eran las salidas de los dos canales no utilizados. Esos dos canales tenían además sus entradas A4 y A3 **atadas a masa**, junto a los pines NC del integrado. El resultado era que el eje Z quedaba fijo a nivel bajo en el conector, por mucho que la PL generase la señal. Las dos líneas que la transportaban terminaban en salidas del adaptador sin conectar, y las que llegaban al puente en H estaban permanentemente a cero.

Ninguna comprobación automática detecta este fallo: el diseño es eléctricamente correcto y no incumple ninguna regla de Altium. El comprobador de conectividad no ve nada anómalo, porque las dos salidas sobrantes son terminaciones válidas y las entradas a masa son una conexión legítima. Solo se manifiesta al medir el eje Z en el conector. La V2 reencamina las entradas de U6 a B2Y y B1Y, que es donde salen efectivamente las señales del eje.

Alimentación

Misma arquitectura que la placa CDHS: 12 V del FMC, convertidor DC/DC para los 5 V de potencia, y VADJ 1,8 V para los transceptores. La única diferencia es la etapa de potencia de los motores, que no se alimenta del FMC: admite alimentación exterior a través de los dos bornes banana, independiente del resto de la placa.

El mapa completo de señales entre la placa AOCS y el conector FMC se encuentra en el Anexo C. Los esquemáticos (PDF) y la BOM están disponibles en `tfm/00_docs/pcbs/aocs/`. El fichero `LINCE3_AOCS_V2.pdf` es la revisión aquí descrita, la única que refleja el estado final del diseño, y del que proceden todas las hojas de esquemático reproducidas en esta sección. `LINCE3_AOCS.pdf` es un volcado anterior que se conserva por trazabilidad, pero está desactualizado incluso respecto a lo que se fabricó.

3.5.6. Fabricación

Una vez validados los esquemáticos y completado el rutado de las PCBs, se encargaron las placas con *stencil* a **PCBWay**, y los componentes a **Mouser** y **DigiKey** según disponibilidad. El proceso de montaje en el laboratorio siguió los pasos habituales de soldadura por reflujo SMD:

1. **Aplicación de pasta de soldadura.** Se fijó la placa a la mesa con placas de idéntico grosor alrededor y cinta de pintor. Se colocó el *stencil* alineado y se extendió la pasta con espátula.
2. **Colocación de componentes.** Con pinzas de punta fina, se colocaron los componentes SMD sobre la pasta. Se montaron dos placas en paralelo para optimizar el tiempo de proceso.
3. **Reflujo.** Las placas se introdujeron en el horno de reflujo del laboratorio seleccionando el perfil de temperatura adecuado para la pasta de soldadura utilizada.

Las figuras 3.13 y 3.14 muestran las placas CDHS y AOCS ya montadas al salir del horno, que son las que se emplean en toda la campaña de validación (sección 4.1 y siguientes).

La placa de comunicación serie se montó por el mismo procedimiento. La figura 3.15 muestra sus dos caras ya pobladas: los conectores D-Sub y los *jumpers* de configuración en el anverso, y los catorce transceptores LTC2865 en el reverso.

3.6. Desarrollo de herramientas de testing

Nada de lo descrito hasta aquí puede darse por bueno sin un banco desde el que ejercitarlo. El transceptor, el driver y las tres placas se validan desde el propio MPSoC. Para ello hizo falta construir el software que los pone a trabajar y que hace observable lo que ocurre: aplicaciones sobre RTEMS que configuran cada canal, generan tráfico y comprueban lo recibido. También hizo falta el equipamiento de laboratorio que cierra los enlaces sobre cable real.

Este apartado describe esas herramientas. Son las que producen los resultados del capítulo 4, y conocerlas es necesario para interpretar las trazas y las medidas que allí se presentan.

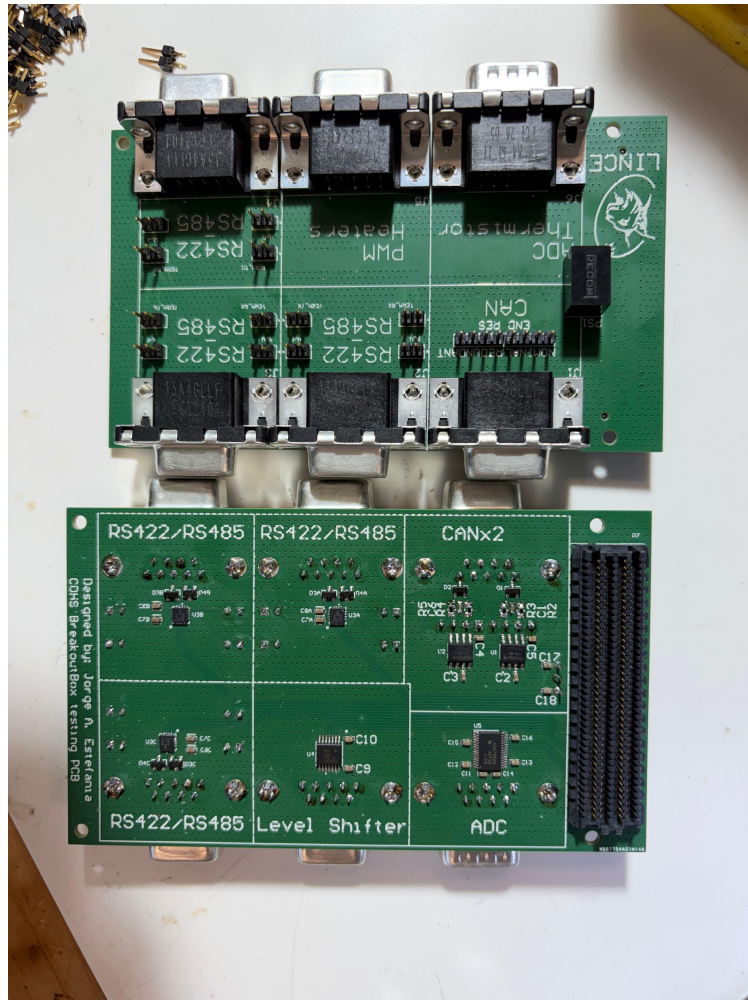


Figura 3.13: Placa CDHS tras el proceso de soldadura.

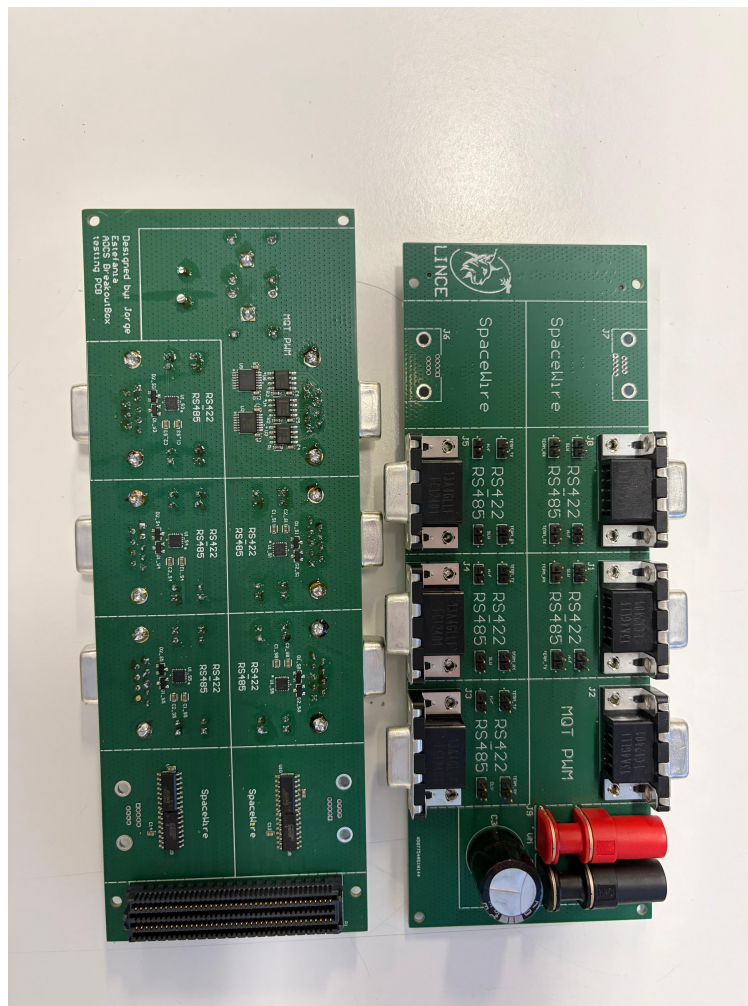
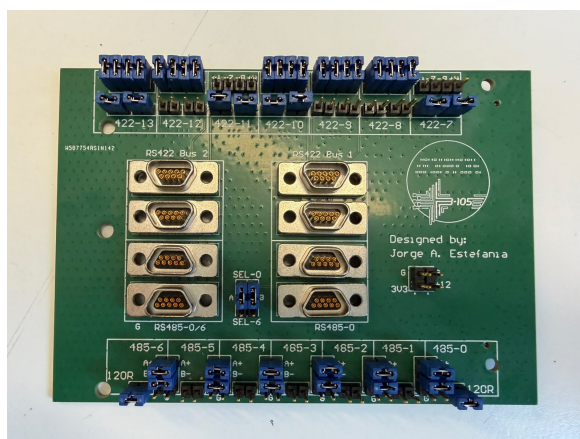
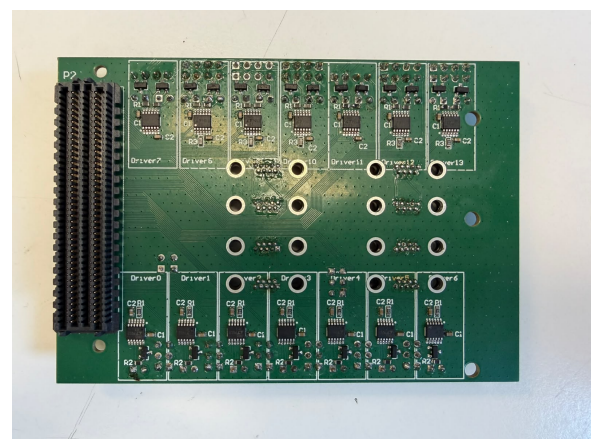


Figura 3.14: Placa AOCS tras el proceso de soldadura.



(a) Cara de conectores.



(b) Cara de componentes.

Figura 3.15: Placa de comunicación serie tras el proceso de soldadura.

3.6.1. Aplicación de testing del driver serie en RTEMS

Para ejercitar el driver sobre la placa se escribió una aplicación de pruebas, repartida en tres ficheros: `init.c`, con la configuración estática de RTEMS; el propio driver (`transceiver.c` y `transceiver.h`); y `main.c`, que usa su API pública.

La aplicación abre los catorce canales, registra en cada uno un *callback* de recepción que reconstruye las líneas byte a byte, y lanza una tarea de consola que espera comandos del usuario por el puerto USB. Desde ella se envía un mensaje por un canal concreto o por todos a la vez, que es la forma en que se ejercitaron los buses en las campañas del capítulo de validación hardware. La consola permite además conmutar el limitador de *slew rate* sin reiniciar el sistema, reejecutando la inicialización del canal con otra configuración.

El detalle de la configuración estática de RTEMS, la tabla de comandos de la consola y la secuencia de arranque de la aplicación se recogen en el apartado C.7 del Anexo C.

3.6.2. Aplicación de testing de las placas CDHS y AOCS

Ampliación en PL para soportar interfaces adicionales CDHS

Para poder probar la funcionalidad de la placa CDHS, fue necesario añadir soporte firmware que controlara las líneas adicionales a RS422 y RS485. La configuración del entorno fue la siguiente: se preparó el hardware en la PL para probar las líneas de RS422/RS485, SPI, CAN y PWM. Para ello se creó un proyecto en Vivado con el script TCL `new_rebuild_all.tcl`, con 3 transceptores serie, y se añadió el bloque de control PWM. Además se configuró el PS para conectar SPI y CAN con el exterior activando las líneas de SPI 0, CAN 0 y CAN 1 como interfaces externas hacia el conector FMC.

Para controlar las líneas de PWM se instanció el bloque `PWMx4_auto_test`, que genera desde la PL cuatro señales PWM a 10 kHz, 5 kHz, 1 kHz y 100 Hz con *duty cycle* del 50 %. No necesita ningún driver en el PS. Se utilizó el mismo BOOT.BIN para todas las pruebas de CDHS.

Para probar el ADC ADS7950 de la placa CDHS, se desarrolló una pequeña aplicación de lectura SPI sobre RTEMS. Dado que el BSP de RTEMS 7 para ZCU102 no incluía en ese momento un driver SPI de alto nivel, se accedió directamente al controlador SPI Cadence integrado en el PS mediante mapeado de registros en memoria (MMIO). Se siguió para ello el mapa de registros descrito en el Manual de Referencia Técnico del Zynq UltraScale+ [25]. El protocolo de comunicación con el ADC (formato de trama de 16 bits, selección de canal en *Manual Mode* y gestión de la latencia de conversión) se implementó conforme al *datasheet* del ADS7950 [24].

Para probar el CAN, se utilizó una aplicación desarrollada por Diego Ramos, compañero en el laboratorio B105 y en el proyecto LINCE. En su aplicación, se configuran las líneas de CAN y se establecen tests que comprueban que la conexión por CAN funciona correctamente.

Ampliación para soportar interfaces adicionales AOCS

Para probar la placa AOCS, se generó un proyecto de Vivado con `new_rebuild_all.tcl` con 5 transceptores. Se añadió además un bloque VHDL para controlar los puentes en H por PWM (`Motor_H_bridge_test.vhd`). Este bloque genera 3 PWMs y los enruta por la línea 1 o línea 2 de cada motor en función de la dirección deseada. Para estas pruebas,

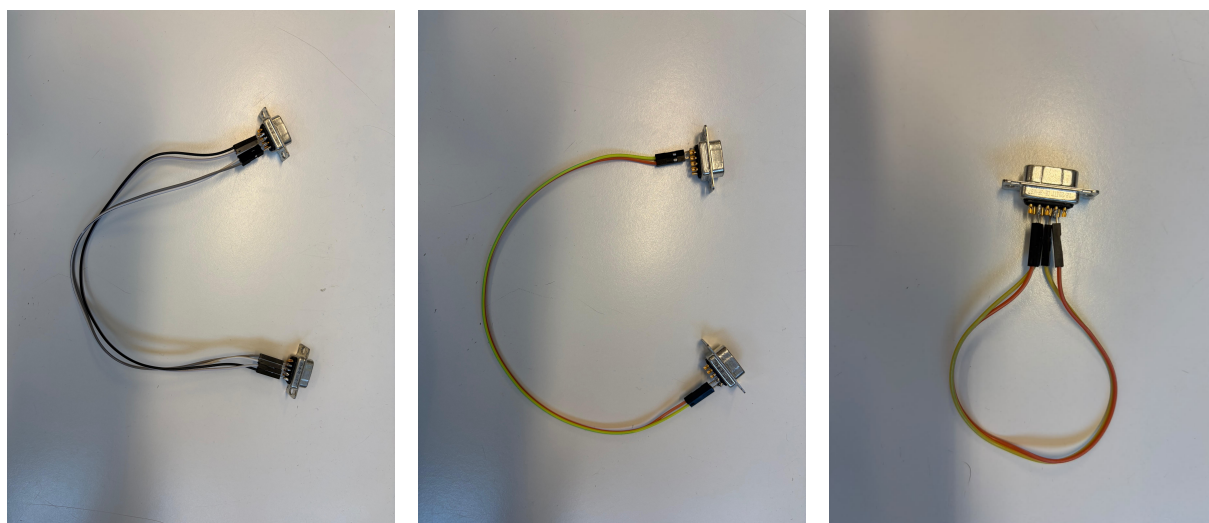
se fijó un tiempo para cada dirección: durante unos segundos el PWM iba en un sentido (línea 1 activa, línea 2 a 0), y durante los siguientes segundos, en el sentido contrario.

La comunicación por SpaceWire quedó fuera del alcance de la validación por su alta complejidad técnica. El estándar no se agota en la capa física LVDS que expone la placa, sino que exige implementar en la lógica programable la codificación *Data-Strobe*, la máquina de estados de establecimiento y recuperación del enlace, y los niveles de paquete y red. Este desarrollo, por sí solo, excede el alcance de este trabajo.

3.6.3. Arneses y equipamiento de validación

Arneses de *loopback*

Cada bus se cierra sobre sí mismo con un arnés propio, para poder ejercitarlo sin más equipo que la placa (figura 3.16). El de **RS422** va a 4 hilos y cruza las líneas TX-P/N de un driver con las RX-P/N del opuesto. El de **RS485** va a 2 hilos y conecta A+ y B- de un driver con los del otro. En **CAN** se unen las líneas CAN_H y CAN_L del canal nominal con las del redundante; como ambos canales convergen en el mismo conector J1, el arnés emplea un único D-Sub-9 con los hilos cruzados entre ambos pares, dejando puntas accesibles para la sonda del osciloscopio.



(a) RS422, 4 hilos.

(b) RS485, 2 hilos.

(c) CAN nominal-redundante.

Figura 3.16: Arneses de *loopback* sobre conector D-Sub-9.

Equipo de laboratorio utilizado

Se utilizó un osciloscopio y una fuente de alimentación de hasta 32 V.

3.7. Arquitecturas de transporte PS-PL

El factor que determina el rendimiento del sistema con los catorce canales activos no es el transceptor ni el software del driver, sino **el camino por el que los bytes viajan entre la DDR y la lógica programable**: en la arquitectura de partida cada byte recibido genera una interrupción, y el procesador no da abasto con las catorce líneas.

Para determinar con datos cuál es la arquitectura de transporte PS-PL adecuada para catorce líneas serie sobre este MPSoC se implementaron **tres arquitecturas completas**, resumidas en la tabla 3.6, y se midieron con el mismo programa de pruebas. El transceptor serie de la sección anterior es idéntico en las tres; lo único que cambia es el transporte y el driver `transceiver.c/h` que lo maneja.

Variante	Transporte PS↔PL	Idea
A — GPIO	AXI-Lite / AXI GPIO	Un registro por canal; el PS mueve byte a byte.
B — DMA14	14 × AXI DMA (PG021)	Un motor DMA completo por canal.
C — MCDMA	1 × AXI MCDMA (PG288) + puente VHDL	Un motor multicanal compartido.

Tabla 3.6: Las tres arquitecturas de transporte implementadas y comparadas.

Las tres son funcionales y arrancan sobre la misma placa con el mismo código de aplicación. Esta sección describe la construcción de cada una, con las cifras de coste que arroja, y cierra con la comparación que lleva a elegir la última. La comparativa cuantitativa completa, con *throughput* y latencia, se recoge en el capítulo de resultados (sección 4.4).

3.7.1. Variante A — AXI GPIO (referencia)

Es la arquitectura de partida, la descrita en las secciones anteriores de este capítulo, y la referencia contra la que se mide todo lo demás. Cada canal es una instancia de `CONFIGURABLE_SERIAL_TOP` con su propio AXI GPIO de doble canal, colgado de un *smartconnect* AXI-Lite: un registro de escritura (dato de transmisión, bits de control y configuración de protocolo) y un registro de lectura (dato recibido y banderas de estado). Un AXI INTC concentra las veintiocho fuentes de interrupción (RX y TX de cada canal) en una sola línea hacia el PS.

Su virtud es la simplicidad: es la variante con menos lógica (**6 434 LUT**, un 2,35 % del dispositivo, 460 LUT por canal), no necesita ningún mapeo especial de la MMU y no tiene que preocuparse por la coherencia de caché, porque todos los accesos son del propio procesador. Su defecto es estructural y no tiene arreglo dentro de esta arquitectura: **el procesador tiene que tocar cada byte**. En recepción, cada carácter completo genera una interrupción; la ISR maestra despierta a la tarea *worker* del canal, que drena el FIFO hardware byte a byte. En transmisión, la ISR escribe el siguiente byte cada vez que el transmisor queda libre.

El coste medido es de **1 250 interrupciones por kilobyte** transferido. Con eso, el *throughput* de un canal se estanca en torno al 30 % del techo físico de la línea y **no mejora al subir la velocidad**: de 115 200 a 4 Mbaudios el rendimiento apenas pasa de 3 471 a 4 907 B/s, porque el cuello de botella ya no es la línea serie sino el manejo de interrupciones. El sistema está en el régimen de *interrupt livelock* descrito en la sección 2.6.3.

3.7.2. Variante B — 14× AXI DMA

Esta variante saca a la CPU del movimiento de datos delegándolo en motores DMA. Como un `axi_dma` sirve a un único *stream*, cubrir los catorce canales exige catorce instancias.

Adaptación del UART a AXI-Stream

El transceptor original habla con registros, no con *streams*. El módulo `UART_AXIS_TOP.vhd` hace de adaptador: envuelve un `CONFIGURABLE_SERIAL` y le añade una interfaz AXI4-Stream esclava en el lado de transmisión (TDATA, TVALID, TREADY) y una maestra en el de recepción (con TLAST para delimitar el paquete). Cada uno de los catorce UART se convierte así en un periférico AXI-Stream que cuelga de su propio DMA.

Operación

Los DMA se configuran en modo directo, sin *scatter-gather*, que es todo lo que necesita un canal con un único *buffer* activo:

- **Transmisión.** El driver escribe la dirección del *buffer* y la longitud en los registros del canal MM2S; el motor lee la DDR y alimenta el *stream*. La ISR de MM2S libera un semáforo al completar la transferencia.
- **Recepción.** El canal S2MM se arma con longitud 1. El TLAST que el adaptador emite con cada byte completa el paquete, la ISR de S2MM copia el dato al *ring* de software y rearma el descriptor.

Coste de la replicación de motores DMA

La variante opera en placa y en transmisión alcanza **3 interrupciones por kilobyte**, frente a las 1 250 de la variante GPIO. El límite de la variante no está, por tanto, en el rendimiento que alcanza, sino en el precio que paga por alcanzarlo. Replicar catorce veces el motor DMA choca con dos recursos finitos del dispositivo:

El coste en lógica. Catorce motores DMA completos ocupan **40 217 LUT** (*Look-Up Table*), un 14,7 % del dispositivo, esto es **2 873 LUT por canal**: más de seis veces la huella por canal de la variante GPIO. Ese coste condiciona además el endurecimiento futuro del diseño, porque una triplicación modular (TMR, *Triple Modular Redundancy*) de la lógica, que es la técnica habitual para tolerar los efectos de la radiación en órbita, multiplicaría por tres esa huella y la llevaría a cerca de la mitad del dispositivo.

El límite de las interrupciones. Cada `axi_dma` expone dos líneas de interrupción, MM2S y S2MM. Catorce DMA son **veintiocho líneas**, y el PS solo ofrece ocho en `pl_ps_irq0`. Hubo que reducir las por OR en la propia PL hasta dejarlas en dos, con lo que la ISR pierde la información de *qué* canal ha interrumpido y debe escanear los catorce registros de estado en cada evento. El ahorro de interrupciones que aportaba el DMA se paga en parte con un coste de escaneo dentro de la ISR.

El límite de ocho líneas es del dispositivo, no del diseño, y no se resuelve replicando más motores: es la restricción que conduce a la tercera variante.

La recepción de esta variante **no llegó a medirse**. El banco de pruebas de la sección 3.7.6 exige insertar el módulo de *loopback* en el diseño de cada variante, y sobre esta el lazo de recepción no llegó a operar: el autodiagnóstico del banco lo señaló como abierto y todos

los tests que dependen de recibir salieron a cero. Con el coste en lógica y el límite de líneas de interrupción ya medidos, la arquitectura quedaba descartada como solución final con independencia de lo que midiera su recepción, por lo que la depuración no se llevó más allá y el esfuerzo se dedicó al diseño de la variante MCDMA. Sus cifras de recepción constan como no disponibles en los resultados; las válidas de esa corrida son el coste de hardware, la huella de memoria y las interrupciones de transmisión.

3.7.3. Variante C — AXI MCDMA con puente VHDL

La tercera arquitectura, y la que se adopta como solución final, sustituye los catorce motores por **uno solo multicanal** (`axi_mcdma`, PG288) y añade un **puente en VHDL** que reparte y recoge de los catorce UART. Es la variante con más diseño propio y la única en la que el problema deja de resolverse instanciando IP de terceros.

La topología es deliberadamente **asimétrica**, porque los dos sentidos tienen exigencias distintas:

- En **transmisión** el PS decide cuándo y a quién escribe. Basta con un canal DMA si el destino viaja *dentro* del dato.
- En **recepción** los catorce canales pueden hablar a la vez y sin avisar. Cada uno necesita su propio *buffer* en la DDR para que los flujos no se mezclen, lo que exige catorce canales S2MM.

De ahí la asimetría de la figura 3.17: un solo canal en el sentido de transmisión y catorce en el de recepción, sobre un mismo motor multicanal.

Camino de transmisión

El canal MM2S del MCDMA emite un único *stream* de 32 bits, que un conversor de anchura reduce a 8. Con un solo canal de DMA, **el destino de cada byte viaja en la cabecera del propio paquete**:

```
[ {CH_ID[3:0], LEN[11:8]} | {LEN[7:0]} | payload...]
```

Dos bytes de cabecera con el identificador de canal y la longitud del *payload*. El bloque `BRIDGE_TX_TOP` los interpreta y encamina el resto del paquete:

- `TX_DMA_ROUTER` parsea la cabecera y extrae el canal y la longitud.
- `MUX_2x16` demultiplexa la escritura hacia la FIFO del canal indicado.
- Catorce `TX_WORKER`, uno por canal, con una FIFO de 8 bits en modo *first-word-fall-through* y una máquina de estados (`S_IDLE` → `S_START` → `S_BUSY`) que alimenta el núcleo UART byte a byte, respetando su *handshake*: presenta el byte, espera a que el transmisor lo acepte (`Send_ok`), lo extrae de la FIFO y espera a que termine de serializarlo (`TX_RDY`) antes de presentar el siguiente.

Un único canal DMA sirve así a los catorce transceptores. El precio es que las transmisiones se serializan: el driver protege el acceso al canal compartido con un *mutex*, y por eso esta variante es la única en la que `Transceiver_Send` es bloqueante.

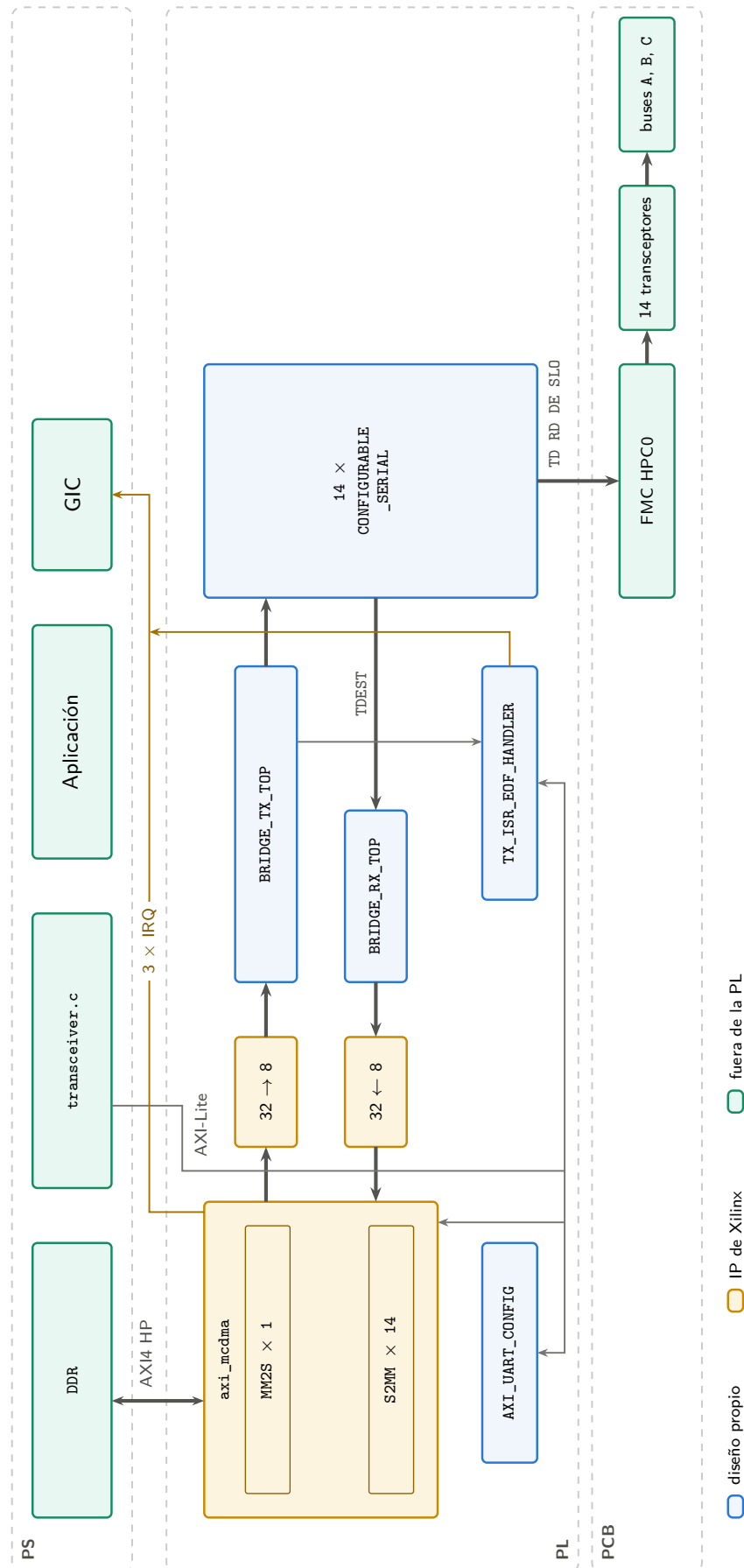


Figura 3.17: Arquitectura de la variante MCDMA.

Notificación de fin de transmisión (TX_ISR_EOF_HANDLER)

El DMA sabe cuándo ha terminado de *entregar* el paquete al puente, pero no cuándo el UART ha terminado de *emitirlo* por el cobre. Para una línea RS485 con control de dirección esa diferencia importa: no se puede soltar el bus antes de que el último bit haya salido.

Sacar los catorce pulsos de fin de trama a catorce líneas de interrupción no es una opción porque las líneas son ocho. Se diseñó por tanto un bloque propio, TX_ISR_EOF_HANDLER.vhd, que aplica el patrón de **registro *sticky* más una única interrupción**: engancha el pulso de fin de trama de cada *worker* en un bit de un registro, lo expone por AXI-Lite y genera una sola interrupción de nivel. Su mapa de registros es el de la tabla 3.7:

Offset	Registro	Acceso	Función
0x00	TX_DONE	RO	Un bit por canal; se pone a 1 al fin de trama.
0x04	TX_DONE_CLEAR	W1C	Escribir 1 borra el bit correspondiente.
0x08	IRQ_ENABLE	RW	Máscara de canales que generan interrupción.
0x0C	IRQ_STATUS	RO	TX_DONE AND IRQ_ENABLE.

Tabla 3.7: Mapa de registros del bloque TX_ISR_EOF_HANDLER (base 0xA0001000).

La interrupción es el OR de los bits habilitados, y la puesta a 1 por fin de trama tiene prioridad sobre el borrado por software, para que no se pierda un evento que llegue justo durante la escritura de *clear*. El driver espera en un semáforo que esta ISR libera, de modo que *Transceiver_Send* retorna cuando el byte ha salido físicamente, no cuando el DMA lo ha copiado.

Camino de recepción y etiquetado por TDEST

En recepción cada UART entrega bytes de forma autónoma. El bloque BRIDGE_RX_TOP recoge los catorce flujos y los presenta al MCDMA como un único *stream* etiquetado:

- Una **FIFO por canal** absorbe la ráfaga de bytes que llega mientras el árbitro atiende a otro.
- Un **DEMUX** recorre las catorce FIFO.
- El bloque **DRAIN** vacía la FIFO seleccionada hacia el *stream* de salida, cierra el paquete con **TLAST** y fija el **TDEST** con el número de canal de procedencia.

El MCDMA demultiplexa por **TDEST**: el paquete etiquetado con el canal *N* acaba en el descriptor y el *buffer* de DDR del canal *N*. Sin esa etiqueta el hardware no tiene forma de saber de quién es cada byte, y todo el tráfico se deposita en el canal cero, comportamiento que se documenta en la sección 3.7.5.

La figura 3.18 recoge el mecanismo completo. Los catorce canales entran y salen por un único par de interfaces AXI-Stream, y es la etiqueta **TDEST** la que decide en qué *buffer* de la DDR acaba cada paquete.

El paquete se cierra por dos condiciones: cuando la FIFO del canal se vacía (fin natural de la ráfaga) o cuando se alcanza el tope duro de longitud del generico **MAX_PKT**, fijado

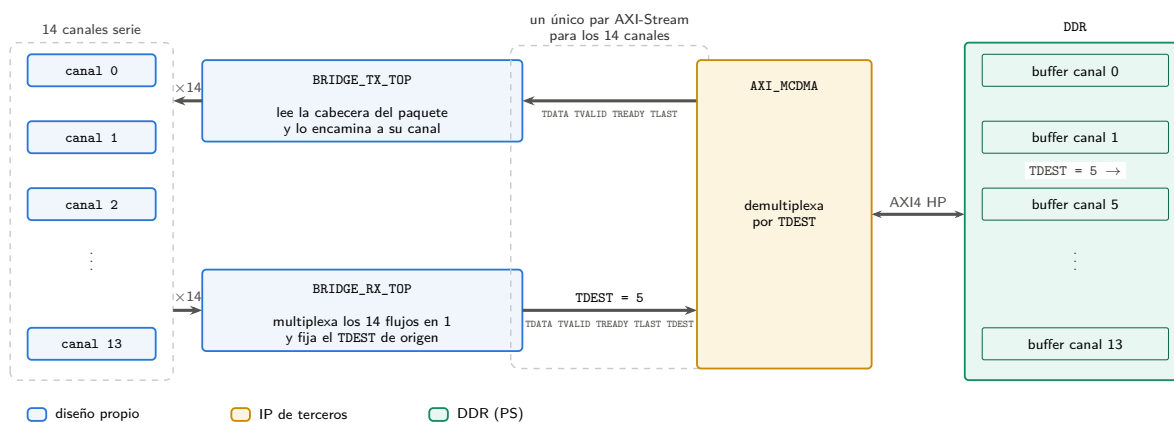


Figura 3.18: Multiplexado de los catorce canales por TDEST.

en 256 bytes. Ese segundo criterio es una **invariante de seguridad** impuesta por el IP, cuya razón se detalla más abajo.

Mapa de memoria y de interrupciones

Los tres bloques que el PS debe alcanzar se sitúan en el mapa de memoria como recoge la tabla 3.8.

Dirección	Bloque	Tamaño
0xA000_0000	AXI_UART_CONFIG — un registro de 32 bits por canal	4 KB
0xA000_1000	TX_ISR_EOF_HANDLER	4 KB
0xA001_0000	Registros de control del <code>axi_mcdma</code>	64 KB

Tabla 3.8: Mapa de memoria de la variante MCDMA.

Las tres líneas de interrupción hacia el PS son `mm2s_introut` (SPI 89, vector 121 en RTEMS), `s2mm_introut` (SPI 90, vector 122) y la del bloque propio de fin de trama (SPI 91, vector 123). Las catorce salidas de interrupción de los canales S2MM se reducen por OR a una sola línea, y la ISR escanea los registros de estado para saber qué canales han completado.

Construcción de descriptores *scatter-gather* en el driver

El MCDMA opera siempre en modo *scatter-gather*, de modo que el driver de esta variante no escribe direcciones en registros, sino que **construye descriptores en memoria**. Cada BD ocupa 64 bytes alineados a 64, con la estructura de la tabla 3.9, y el driver mantiene un anillo de un solo descriptor por canal, cuyo puntero al siguiente apunta a sí mismo:

Offset	Campo	Contenido
+0x00	NDESC	Dirección del siguiente descriptor (aquí, él mismo).
+0x08	BUFA	Dirección del <i>buffer</i> de datos.
+0x14	CTRL	SOF (bit 31), EOF (bit 30), longitud (bits 25:0).
+0x18	STS	Recepción: COMPLETE (bit 31) y bytes realmente escritos.
+0x1C	SIDEBAND	Transmisión: COMPLETE (bit 31).

Tabla 3.9: Estructura del descriptor de *buffer* (BD) del MCDMA.

Rearmar un canal de recepción consiste en borrar el campo de estado, volcar la línea de caché del descriptor y volver a escribir el puntero de cola: el motor lo relee y reanuda. La recepción entrega el paquete cuando el puente cierra el TLAST, no byte a byte, que es justamente el origen de la reducción de interrupciones.

Coste de la solución multicanal

Compartir un único motor entre los catorce canales deja la ocupación en **14 191 LUT**, un 5,18 % del dispositivo, **1 014 LUT por canal**, y eso incluyendo lógica propia que las otras dos variantes no llevan: el puente de transmisión, las catorce FIFO de recepción con su árbitro y el bloque de fin de trama.

En interrupciones bastan las **tres líneas** ya descritas, MM2S, S2MM y fin de trama, de las ocho que ofrece `pl_ps_irq0`. La ISR de recepción sigue teniendo que escanear registros de estado para saber qué canales han completado, pero los escanea sobre un solo motor, y quedan cinco líneas libres para futuras ampliaciones del diseño.

El precio de compartir se paga en el sentido de transmisión: con un único canal MM2S, dos tareas que quieran escribir a la vez se turnan tras el *mutex* del driver, y por eso `Transceiver_Send` es bloqueante aquí y no en las otras dos variantes. Cuánto cuesta esa serialización se mide en la sección 4.4.

3.7.4. Comparación y elección de la arquitectura final

Con las tres arquitecturas construidas y medidas sobre la misma placa, la elección se decide en dos ejes: el coste de CPU y el coste en la lógica programable. La tabla 3.10 recoge las cifras de ambos, ya justificadas en los apartados anteriores.

Variante	IRQ/KB	LUT	LUT/canal	Líneas IRQ
A — GPIO	1 250	6 434	460	1*
B — DMA14	3	40 217	2 873	28
C — MCDMA	3	14 191	1 014	3

* Un AXI INTC reduce por OR las veintiocho fuentes a una sola línea.

El PS ofrece ocho líneas en `pl_ps_irq0`. Ocupación sobre las 274 080 LUT del XCZU9EG.

Tabla 3.10: Coste de las tres variantes de transporte.

En **coste de CPU** la variante GPIO queda descartada por sí sola: sus 1 250 interrupciones por kilobyte la mantienen en *livelock* y su rendimiento no mejora al subir la velocidad de línea. Las dos variantes con DMA resuelven ese problema por igual, de modo que la diferencia entre ellas no está en lo que rinden sino en lo que cuestan.

Ahí la comparación es concluyente. Replicar catorce motores cuesta **2,8 veces** la lógica de compartir uno solo para el mismo número de canales, y esa distancia se triplica en cuanto se plantee el endurecimiento por TMR: la variante DMA14 llevaría cerca de la mitad del dispositivo, mientras que la multicanal sigue siendo asumible. En interrupciones la diferencia no es de grado sino cualitativa, porque veintiocho líneas contra las ocho disponibles obligan a reducirlas por OR y a escanear catorce registros de estado en cada evento, mientras que tres líneas caben con holgura.

La variante MCDMA se adopta por ello como **solución final**. A cambio hay que asumir la serialización de la transmisión y diseñar y verificar lógica propia en lugar de instanciar IP de terceros, que es a lo que se dedica el resto de esta sección.

3.7.5. Verificación y depuración iterativa del driver

El driver descrito en las páginas anteriores es el resultado de **varias iteraciones** de una metodología de verificación aplicada de forma sistemática a lo largo del trabajo:

1. **Verificación aislada en simulación.** Ningún bloque de la PL se integraba en el diseño sin pasar antes su propio *testbench* en Vivado. El puente y el módulo de *loopback* tienen bancos de prueba dedicados que ejercitan el protocolo AXI-Stream y la topología de buses, respectivamente.
2. **Integración incremental.** Cada bloque validado se incorporaba al diseño de bloques y se comprobaba de nuevo en placa antes de añadir el siguiente, para que cualquier regresión quedara acotada a la última pieza introducida.
3. **Batería de pruebas automáticas en placa.** La aplicación de *benchmark* (sección 3.7.6) actúa como prueba de regresión del sistema completo: una campaña de treinta y seis comprobaciones cuyo resultado, cuántas pasan y cuáles fallan, dirige la siguiente iteración.

Las primeras iteraciones del driver resolvieron cuestiones de integración relativamente directas: una versión inicial por *polling*, que se sustituyó por un modelo íntegramente dirigido por interrupción una vez el IP se regeneró con la interrupción por canal habilitada (`c_enable_multi_intr`); y la propagación de la señal TDEST a lo largo de toda la jerarquía VHDL, sin la cual el MCDMA no puede demultiplexar y todo el tráfico de recepción acaba atribuido al canal cero.

La parte más costosa fue la **depuración del bus AXI-Stream contra el motor de la DMA**. Los fallos de este bus no se manifiestan como un error: no hay bit de estado, ni interrupción, ni descriptor marcado; el motor simplemente deja de aceptar datos y el canal enmudece a mitad de una transferencia. Esa ausencia total de observabilidad desde el software hace que depurar sobre la placa sea inviable, y obligó a **reproducir el escenario en simulación**. Fueron necesarias numerosas iteraciones de *testbench*, inyectando contrapresión, paquetes de distinto tamaño y ráfagas solapadas, hasta poder observar el *handshake* congelándose ciclo a ciclo e identificar las dos condiciones que lo provocaban.

De esas simulaciones salieron las dos **invariantes estructurales** que la lógica del puente garantiza hoy por construcción:

- **Tamaño de paquete acotado.** El motor de recepción está configurado en modo *store-and-forward* (`c_include_s2mm_sf = 1`), de modo que acumula el paquete AXI-Stream completo en un *buffer* interno de **512 bytes** antes de volcarlo a la DDR; un paquete mayor que ese *buffer* lo bloquea indefinidamente, a la espera de un TLAST que ya no puede llegar. El bloque DRAIN impone por ello un tope duro `MAX_PKT = 256`, la mitad del *buffer*, además de cerrar el paquete cuando la FIFO se vacía. El tope por longitud no es redundante: hace que la invariante sea estructural y no dependa de que el drenado gane siempre la carrera a la línea serie.
- **TLAST registrado.** La señal de fin de paquete no puede ser combinacional sobre el estado de «FIFO vacía» mientras hay un dato ofrecido. Si un byte nuevo llegara durante una espera por contrapresión, tumbaría el TLAST ya presentado, reabriendo el paquete y devolviendo el sistema al mismo bloqueo. La señal se congela en cuanto el byte se ofrece.

Con ambas incorporadas, el *testbench* del puente de recepción, `tb_bridge_rx.vhd`, pasa la batería completa. Sus cuatro pruebas están escogidas para cubrir justamente los casos que provocaban el bloqueo: un canal aislado, tres canales entremezclados, un paquete de 256 bytes que fuerza el cierre por longitud y dos ráfagas seguidas sobre el mismo canal. Un *scoreboard* comprueba byte a byte el TDEST de la cabecera, que el canal esté en rango y que el dato coincida con el inyectado:

Listado 3.3: Salida de la simulación de `tb_bridge_rx.vhd`

```
-- Reset liberado --
T1: canal unico ch2 (3 bytes, timeout)
T2: multi-canal ch0/ch5/ch13 (1 byte c/u, timeout)
T3: 256 bytes ch1, prog_full auto-trigger
T4: back-to-back ch7 (2 eventos de timeout)
-----
Checks totales   : 802
Errores          : 0
Pendientes exp   : 0
-----
SIM_RESULT: PASS
```

El contador de *pendientes* a cero es tan relevante como el de errores: significa que no quedó ni un byte inyectado sin salir por el *stream*, que es el síntoma del bloqueo descrito.

Por encima de esa prueba de bloque, `tb_system_e2e.vhd` cierra el lazo completo en simulación (MCDMA → puente de transmisión → transceptor serie → realimentación de la línea → puente de recepción → MCDMA), configurando los canales a 921 600 8N1 y comprobando que cada byte reaparece etiquetado con su canal de origen:

Listado 3.4: Salida de la simulación del sistema completo, `tb_system_e2e.vhd`

```
[CONFIG] Escribiendo 921600 8N1 en canales 0 y 1...
[TEST-1] TX->CH0 loopback: 0xA5, 0x3C ...
[TEST-1] PASS
[TEST-2] TX->CH0(0xDE) y CH1(0xAD) simultaneo...
[TEST-2] PASS
[TEST-3] Inyeccion serie directa CH1 <- 0x55 ...
[TEST-3] PASS
SIM_RESULT: PASS
```

El TEST-2 verifica el demultiplexado por TDEST con dos canales hablando a la vez, que

es la condición que en placa se manifestaba como «todo el tráfico acaba en el canal cero».

El flujo de trabajo impone además una comprobación adicional. El bloque que engloba el puente y los transceptores está integrado como *module reference*, y Vivado **cachea su *checkpoint*** de síntesis fuera de contexto. Editar el VHDL y relanzar la implementación no cambia el *bitstream*: el flujo termina sin aviso, cierra *timing* y escribe un *.bit* con el RTL antiguo. La comprobación quedó incorporada al procedimiento: verificar que el *hash* del BOOT.BIN cambia antes de dar por bueno un *bitstream* nuevo.

Con estas correcciones el driver quedó estable: la campaña completa pasó de **0 de 36 a 36 de 36** comprobaciones, la prueba de *throughput* entrega los 5120 bytes de las cinco repeticiones sin pérdidas y el barrido de velocidad supera los siete puntos, de 115 200 a 4 Mbaudios.

3.7.6. Banco de pruebas de *loopback* en la PL

Comparar tres arquitecturas exige medirlas en igualdad de condiciones. El problema práctico es que una medida sobre la PCB real depende del cableado, de los *jumpers* de configuración de bus y de la integridad física de las líneas, factores que varían entre montajes y contaminan la comparación del transporte, que es lo que se quiere aislar.

Los buses se cierran por ello **dentro de la propia FPGA**. Un módulo de *loopback* insertado en la PL reproduce la topología de buses que la PCB implementa en cobre, recogida en la tabla 3.11:

Bus	Maestro	Esclavos
A (multipunto)	CH0	CH1–CH6
B	CH7	CH8–CH10
C	CH11	CH12–CH13

Tabla 3.11: Topología de buses reproducida por el módulo de *loopback* de la PL.

La emulación es fiel al comportamiento eléctrico de un bus RS485: en reposo la línea está a nivel alto y los transmisores hacen *wired-AND*, de modo que la señal de recepción de un canal es el AND de las transmisiones de **los demás** canales de su bus. En particular, **un canal no se oye a sí mismo**, igual que en la PCB, donde el receptor se inhibe mientras el control de dirección está activo. Sin esa inhibición, cada byte transmitido se contaría dos veces en los contadores de recepción y la prueba de desbordamiento mediría el doble de pérdidas. Los tres buses están además aislados entre sí, para que la prueba de transmisión concurrente mida solapamiento real y no diafonía.

El módulo se verifica con su propio *testbench*, que comprueba en dieciséis puntos las tres propiedades de las que depende la validez de toda la comparativa: que en reposo las catorce líneas están en alto, que un maestro llega a todos los esclavos de su bus pero **no a sí mismo**, y que ningún bus se oye desde otro:

Listado 3.5: Salida de la simulación de *tb_bench_loopback.v*

```
ok  reposo rd = 1
ok  rd[1] sigue a td[0] = 0
ok  rd[6] sigue a td[0] = 0
ok  rd[0] no se oye a si mismo = 1
```

```
ok rd[8] aislado del bus A = 1
ok rd[12] aislado del bus A = 1
ok bus A: rd[1] = 0
ok bus B: rd[8] = 0
ok bus C: rd[12] = 0
ok maestro A mudo a si mismo = 1
ok maestro B mudo a si mismo = 1
ok maestro C mudo a si mismo = 1
ok multidrop rd[1..6] todos a 0 = 0
ok respuesta: rd[0] oye a td[1] = 0
ok respuesta: rd[2] tambien la oye (multidrop) = 0
ok respuesta no cruza al bus B = 1

TB OK: 0 fallos
```

Verificado el módulo, se inserta en cada una de las tres variantes mediante un *script* TCL que desconecta la red que unía cada pin físico de recepción con su transceptor y la alimenta desde el *loopback*. Las líneas de transmisión siguen llegando a los pines, con lo que los *constraints* de la placa siguen siendo válidos y ninguna asignación de pines cambia.

Sobre este banco se ejecuta `bench_main.c`, el mismo programa en las tres variantes, con ocho pruebas: autodiagnóstico del lazo (T0), latencia de un *frame* (T1), *throughput* de un canal (T2), transmisión concurrente de tres maestros (T3), recepción multipunto de seis esclavos (T4), desbordamiento del *buffer* de software (T5), barrido de velocidad de 115 200 a 4 Mbaudios (T6) y recuperación ante *frames* corruptos (T7). El cronómetro para siempre **en el receptor**, nunca al retornar de la función de envío: hacerlo al retornar daría cifras falsas, porque el envío es bloqueante en la variante MCDMA y no bloqueante en las otras dos.

Con el lazo dentro de la FPGA, la señal no atraviesa los transceptores RS485 ni el cableado, de modo que lo que se mide es exactamente el **coste del transporte PS–PL**, que es el objeto de la comparativa. Lo que **no** se mide es la integridad física del bus (reflexiones, terminación, tiempos de vuelta del control de dirección, ruido); para eso hace falta la PCB. En particular, el barrido de velocidad da aquí mejores cifras de las que dará en cobre.

4. Resultados

En este capítulo se recogen los resultados de la validación del sistema completo. Los tres primeros bloques se organizan según la placa validada, todas ellas conectadas a la ZCU102 mediante FMC: la sección 4.1 cubre las pruebas realizadas con la placa CDHS, la sección 4.2 las realizadas con la placa AOCS y la sección 4.3 la caracterización eléctrica de la placa de comunicación serie sobre sus catorce transceptores. El cuarto bloque no valida una placa sino una decisión de diseño: la sección 4.4 compara las tres arquitecturas de transporte PS-PL con los buses cerrados dentro de la lógica programable, y selecciona la definitiva.

El orden de validación no coincide con el de diseño de la sección 3.5: las placas CDHS y AOCS, diseñadas después de la de comunicación serie, se validaron antes que ella. La causa es de calendario y no técnica: Sener e Indra necesitaban ambas tarjetas en fecha para validar su propio firmware sobre la ZCU102, por lo que su verificación tuvo prioridad.

4.1. Validación CDHS

4.1.1. Montaje

La placa CDHS, cuyo aspecto tras la soldadura se recoge en la figura 3.13, se conecta a la ZCU102 por el conector FMC HPC0. La figura 4.1 muestra el banco tal y como se utilizó durante toda la campaña, con la alimentación, la consola serie por USB y los arneses de *loopback* conectados.

4.1.2. Comunicaciones serie RS422/RS485

Las pruebas de comunicaciones serie se realizaron con la aplicación de testing en RTEMS, que expone una consola interactiva por USB donde el usuario puede enviar mensajes a cualquier transceptor con el formato <ID><MENSAJE> o a todos simultáneamente con ALL <MENSAJE>. Se construyeron arneses de *loopback* para interconectar los canales:

- **RS422:** arnés a 4 hilos cruzando TX_P/N de un canal con RX_P/N del opuesto.
- **RS485:** arnés a 2 hilos conectando A+ y B- de un canal con A+ y B- del otro.

Al arrancar, el driver descubrió automáticamente los 3 transceptores instanciados en el *bitstream* e imprimió sus direcciones base (0xA0000000, 0xA0001000, 0xA0002000), confirmando el mecanismo de descubrimiento dinámico de hardware. A continuación, la consola reportó UART 00 [OK], UART 01 [OK], UART 02 [OK].

Las pruebas de *loopback* mostraron que los mensajes enviados por UART 0 llegaban a UART 2 y viceversa, y los enviados por UART 1 llegaban correctamente. En algunas

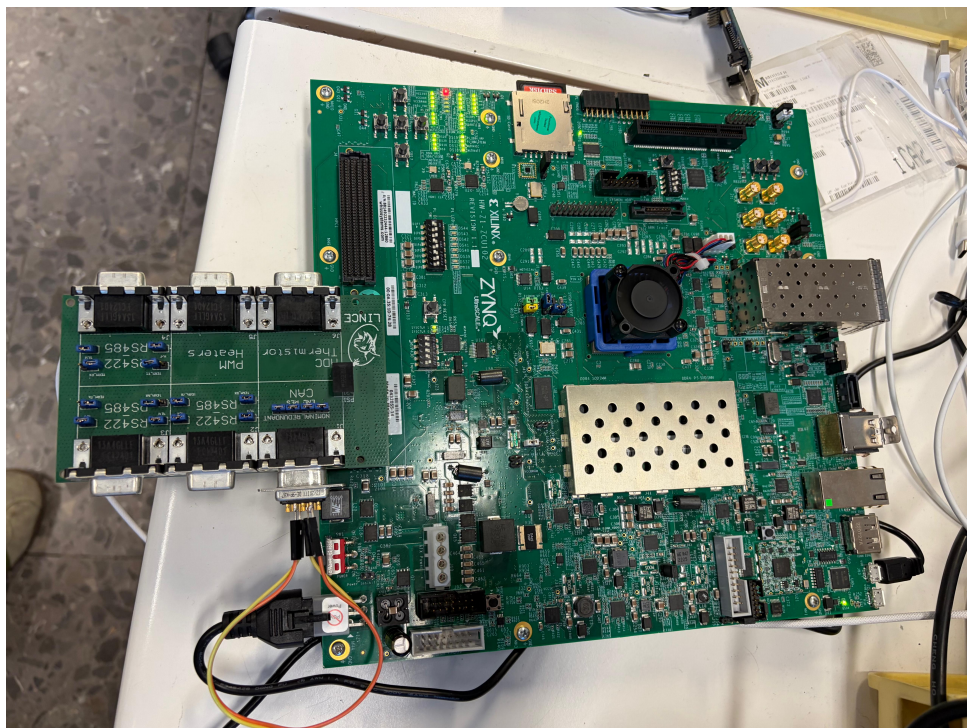


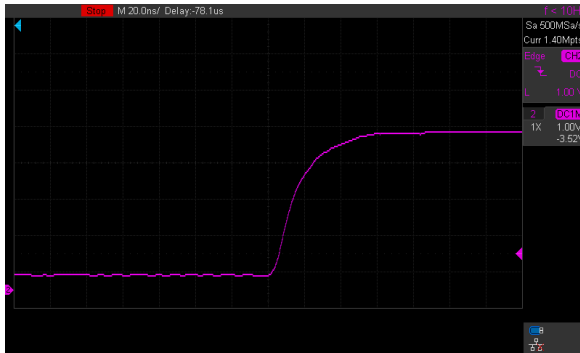
Figura 4.1: Sistema completo montado: ZCU102 con la placa CDHS.

recepciones de esta captura aparecen caracteres corruptos (?), correspondientes a una iteración previa del diseño antes de aplicar la corrección de robustez del receptor descrita en la sección 3.2.3 (verificación de *start-bit* a mitad de período). Una vez implementada esa mejora en la FSM del receptor, los caracteres corruptos desaparecieron por completo en las pruebas posteriores. La salida completa del terminal se recoge en el apartado C.9 del Anexo C.

Las líneas `-- Sent utf8 encoded message --` las escribe el terminal del PC al enviar cada orden, no la aplicación en placa. El encabezado (5 UARTs) es una cadena fija de la aplicación de prueba, heredada de la campaña de la placa AOCS y no actualizada al recompilar; el número de canales realmente activos es el que reporta el descubrimiento dinámico en las líneas anteriores, tres en esta sesión. La traza se conserva íntegra en `tfm/00_docs/logs_terminal/cdhs_testing_rs`.

Efecto del control de *slew rate*

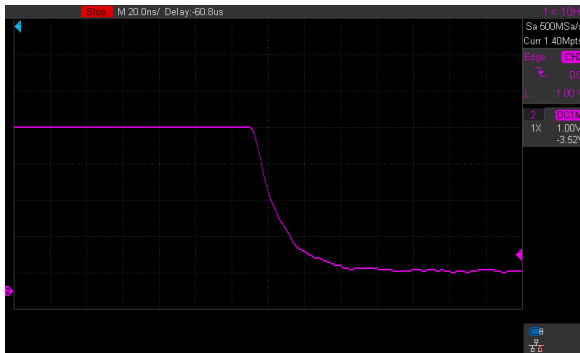
Sobre estos mismos canales se midieron con el osciloscopio los flancos de la línea diferencial con el control de *slew rate* del THVD1424 deshabilitado y habilitado, que en esta placa se selecciona con el *jumper* SLR de cada canal (P4 en la figura 3.9). La diferencia es de casi un orden de magnitud: con el limitador fuera, la transición ocupa unas pocas decenas de nanosegundos, mientras que con el limitador activo se estira hasta cerca de un cuarto de microsegundo (figura 4.2). Esa pendiente más suave es la que reduce las emisiones radiadas y la sensibilidad a las reflexiones en cables largos, y es también la que acota la velocidad máxima alcanzable, un compromiso que se cuantifica sobre la placa de comunicación serie en la sección 4.3.3. La base de tiempos de las capturas de la derecha es cinco veces mayor que la de las de la izquierda.



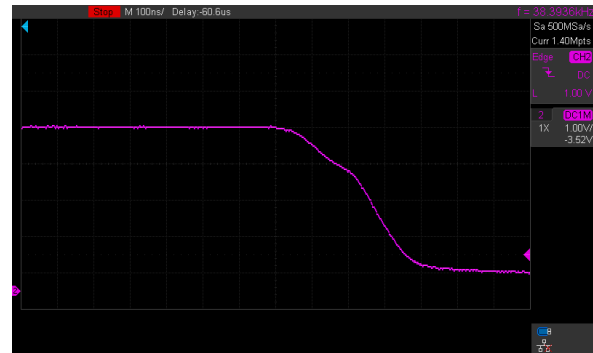
(a) Subida sin limitador (20 ns/div).



(b) Subida con limitador (100 ns/div).



(c) Bajada sin limitador (20 ns/div).



(d) Bajada con limitador (100 ns/div).

Figura 4.2: Flancos de la línea RS485 de la placa CDHS con y sin control de *slew rate*.

4.1.3. Bus CAN

Las pruebas del bus CAN se realizaron con la aplicación de test desarrollada por Diego Ramos (compañero del laboratorio B105 en el proyecto LINCE), que configura los canales CAN y ejecuta una batería de tests de enlace.

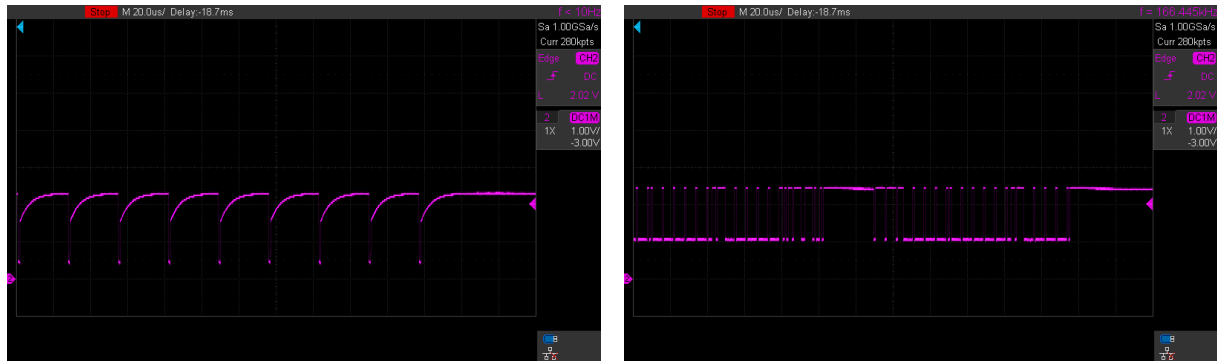
La suite de tests ejecutó 26 pruebas cubriendo inicialización, *loopback* interno, transferencia física CAN0↔CAN1, filtrado de identificadores estándar y extendido, manejo de tramas RTR y pruebas de interrupción por hardware. El resultado fue **26/26 tests PASS, 0 FAILED**:

Listado 4.1: Resumen de resultados del test CAN (placa CDHS)

```
[PASS] CAN0 initialization (Fast Mode)
[PASS] CAN1 initialization (Slow Mode)
[PASS] Loopback RX ID matches TX ID
[PASS] Loopback RX Data matches TX Data
[PASS] CAN0 received correct ID from CAN1      (fisico)
[PASS] CAN0 received correct Data from CAN1     (fisico)
[PASS] Standard ID: Exact Match Accepted (0x1A4)
[PASS] Extended ID: Exact Match Accepted (0x12345678)
[PASS] RTR Filter: Accepted matching Remote Request
[PASS] Tx0k fired successfully
[PASS] Rx0k fired and FIFO was drained successfully (No Storm)
... (26/26 PASS, 0 FAILED)
```

Un resultado relevante adicional fue la verificación del efecto de las resistencias de terminación sobre la integridad de la señal CAN. Sin terminación, la señal presentaba reflexiones visibles que degradaban los flancos (figura 4.3a); con ambas resistencias de 60 Ω conectadas, la forma de onda resultó limpia y bien definida (figura 4.3b). La figura 4.3 enfrenta

las dos capturas.



(a) Sin terminación.

(b) Con $60\ \Omega + 60\ \Omega$.

Figura 4.3: Señal diferencial CAN con y sin resistencias de terminación.

4.1.4. ADC SPI (termistores)

La aplicación de lectura del ADC ADS7950 muestreó los cuatro canales a 500 ms de intervalo e imprimió los valores digitales por el terminal. Para verificar la linealidad de la cadena de adquisición se aplicaron tensiones de referencia conocidas a las entradas analógicas:

- Tensión de entrada 0 V $\rightarrow$ valor digital próximo a 0 (fondo de escala inferior).
- Tensión de entrada 3,3 V $\rightarrow$ valor digital próximo a 4095 (fondo de escala superior, 12 bits).

Los resultados confirmaron el rango de conversión esperado, validando el driver SPI de bajo nivel y la cadena analógica de la placa CDHS. La captura registra el canal CH2 siendo sometido a un barrido de tensión con la fuente de laboratorio:

Listado 4.2: Lectura del ADC ADS7950 durante barrido de tensión en CH2

ADS7950:	CH0= 244	CH1= 43	CH2= 0	CH3= 19	<- tension en CH2 = 0 V
ADS7950:	CH0= 245	CH1= 43	CH2=1577	CH3= 19	<- subida
ADS7950:	CH0= 245	CH1= 43	CH2=3419	CH3= 21	<- cerca del maximo
ADS7950:	CH0= 245	CH1= 43	CH2=3565	CH3= 21	<- ~2.9 V aplicados
ADS7950:	CH0= 245	CH1= 43	CH2=2436	CH3= 22	<- bajada
ADS7950:	CH0= 246	CH1= 43	CH2= 102	CH3= 22	<- de vuelta a 0 V

Los canales CH0 (≈ 245), CH1 (≈ 43) y CH3 (≈ 19) mostraron valores estables bajos durante todo el ensayo, correspondientes a las entradas no excitadas.

El volcado completo del barrido se recoge en el apartado C.10 del Anexo C. Se aprecian en él los dos tramos de subida y bajada de la rampa aplicada a CH2, los dos escalones intermedios en los que la fuente se dejó fija y la estabilidad de los otros tres canales durante todo el ensayo.

La traza se conserva en `tfm/00_docs/logs_terminal/cdhs_testing_adc.txt`.

4.1.5. PWM (calentadores)

Las señales PWM generadas por el bloque `PWMx4_auto_test` se midieron con el osciloscopio sobre el conector J5 de la placa CDHS, verificando los cuatro canales: 10 kHz, 5 kHz,

```
[TRANSCIVER DEBUG] Detectados: 5 | Base: 0xA0000000 | INT: 0xA0005000
CMD> 0 HOLA -> [RX UART 04]: HOLA
CMD> 4 HOLA -> [RX UART 00]: HOLA
CMD> 0 HOLA -> [RX UART 03]: HOLA
CMD> 0 HOLA -> [RX UART 01]: HOLA
```

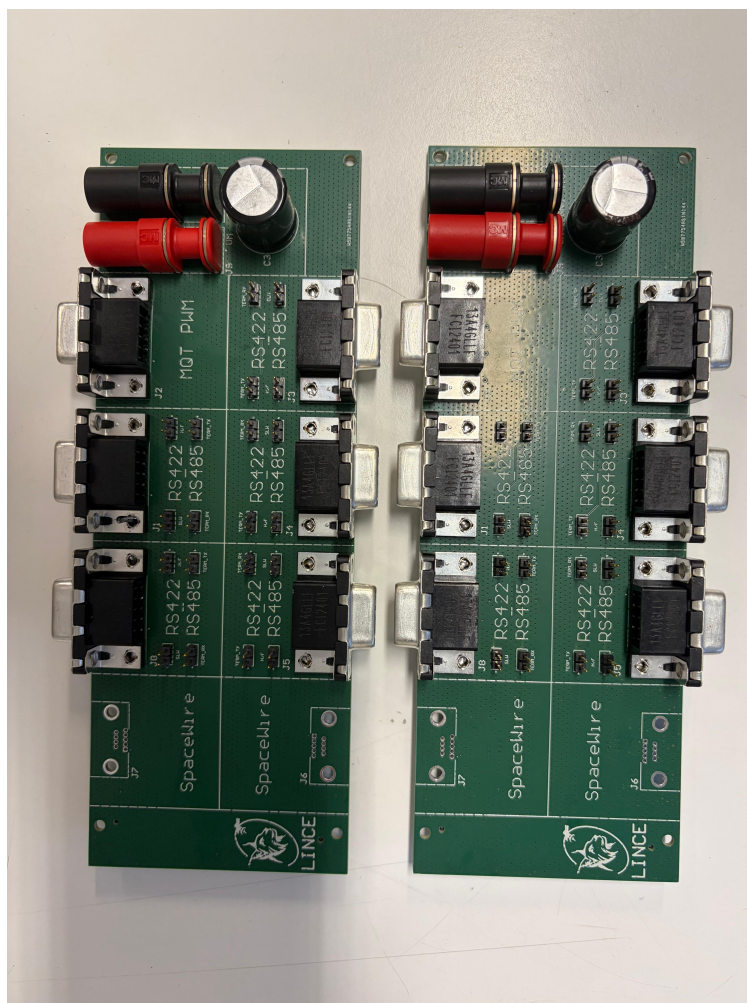


Figura 4.5: Placas AOCS con componentes montados.

La traza completa de la sesión se recoge, junto a la de la placa CDHS, en el apartado C.9 del Anexo C.

A diferencia de la captura de la placa CDHS, aquí no aparece ningún carácter corrupto: esta sesión ya se ejecutó con la verificación de *start-bit* a mitad de período incorporada al receptor. La traza se conserva en `tfm/00_docs/logs_terminal/aocs_testing_rs.txt`.

4.2.3. PWM (puentes en H)

Para la placa AOCS, el bloque VHDL de control de motores generó señales PWM alternando la dirección de giro cada pocos segundos (línea 1 activa con línea 2 a 0, y a continuación línea 1 a 0 con línea 2 activa), permitiendo verificar el control de puentes en H con una fuente de alimentación externa conectada a los bornes banana. La figura 4.6 muestra el par de señales complementarias medido en el conector J2.

La medida recoge además el error de conexión del eje Z descrito en la sección 3.5.5: ese eje no entrega señal en el conector porque las entradas de su puente en H colgaban de dos salidas del adaptador de nivel cuyas entradas están atadas a masa. Es un fallo de la placa, no del firmware, y la revisión V2 lo corrige.

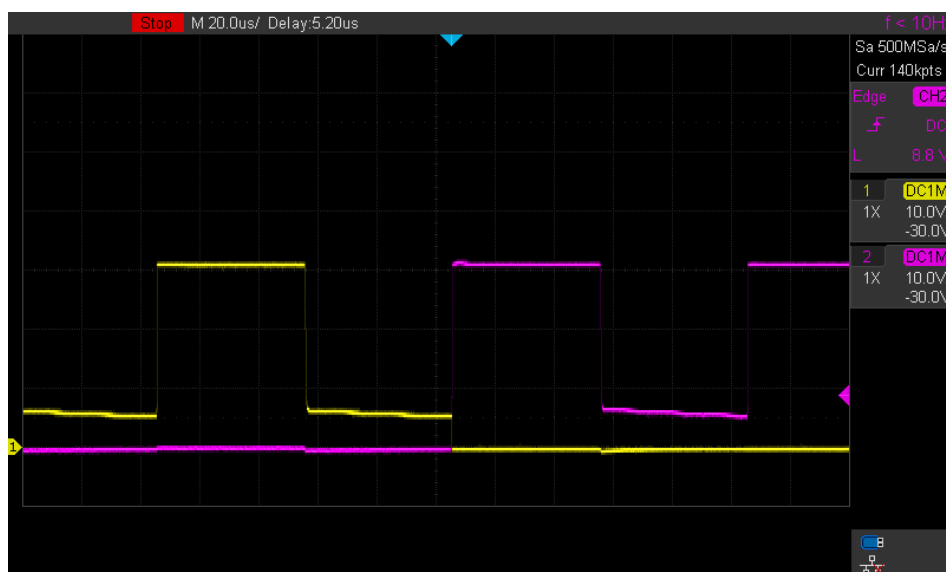


Figura 4.6: Señales PWM de control de motor en la placa AOCS.

4.3. Validación de la placa de comunicación serie

Esta sección caracteriza eléctricamente la **placa de comunicación serie** diseñada y fabricada en la sección 3.5.3. Las señales medidas salen por los pines del MPSoC, atraviesan los catorce transceptores RS485/RS422, recorren el cobre y regresan.

La campaña se ejecuta sobre el transporte MCDMA, que constituye la solución final, con un *bitstream* **sin el módulo de *loopback***. Esta condición es necesaria: con el *bitstream* del banco de pruebas los pines de recepción quedan desconectados de los transceptores, por lo que la PCB no intervendría en la medida.

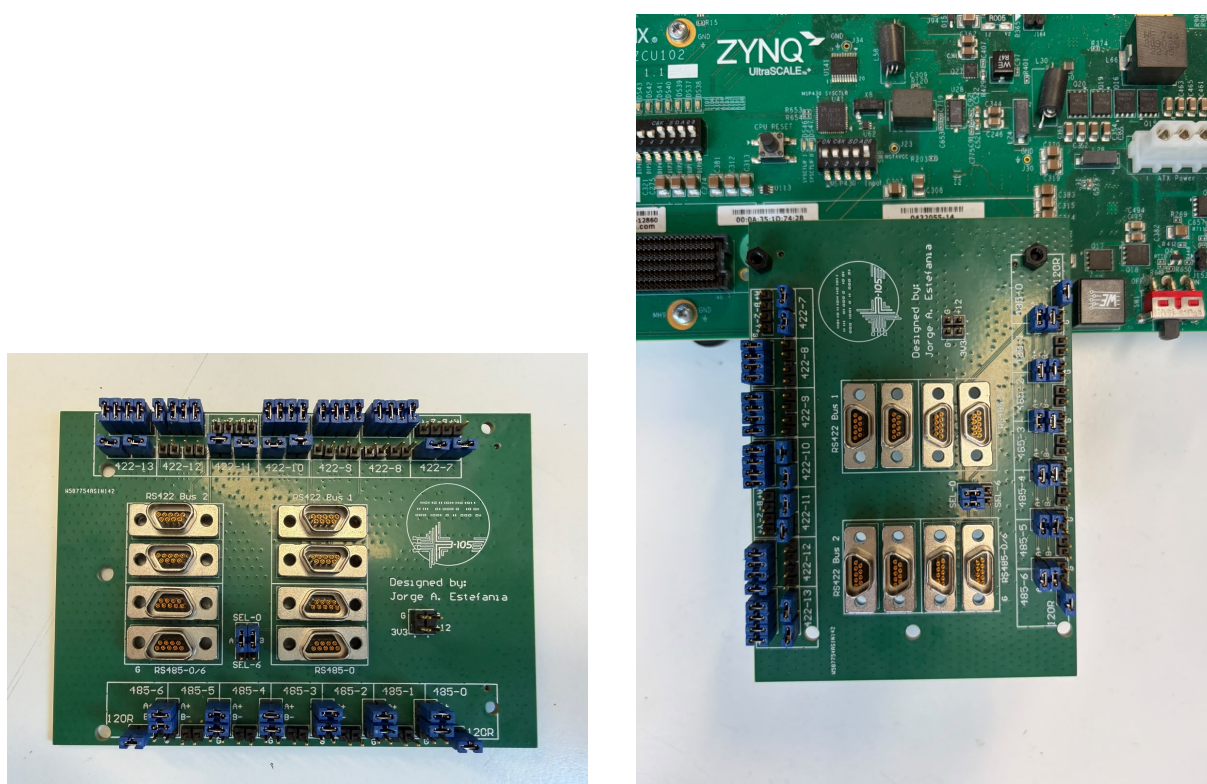
4.3.1. Montaje y topología de buses

La topología física de la placa, fijada por los *jumpers*, es la de la tabla 4.1:

Bus	Tipo	Maestro	Esclavos
A	RS485 multipunto (7 nodos en un bus compartido)	CH0	CH1-CH6
B	RS422 punto a punto (estrella desde el maestro)	CH7	CH8-CH10
C	RS422 punto a punto (estrella desde el maestro)	CH11	CH12-CH13

Tabla 4.1: Topología de buses de la PCB de comunicación serie.

La figura 4.7 muestra la placa con esos *jumpers* ya colocados y montada sobre la ZCU102, que es la configuración con la que se tomaron todas las medidas de esta sección.



(a) Vista superior de la placa.

(b) Placa montada en el conector FMC HPC0 de la ZCU102.

Figura 4.7: PCB de comunicación serie en la campaña de caracterización.

Ningún test presupone esta topología: el test T2 la mide de forma independiente y el resto de pruebas se apoya en lo medido. La diferencia entre el bus A y los buses B y C condiciona la lectura de todos los resultados: en el bus multipunto los seis esclavos se oyen entre sí, mientras que en los buses punto a punto los esclavos solo oyen al maestro.

4.3.2. Batería de pruebas de caracterización

La campaña consta de nueve pruebas, cuyo resultado resume la tabla 4.2. Los apartados siguientes desarrollan las que necesitan explicación.

Test	Resultado
T1 Reposo	0 bytes espurios en los catorce canales: la polarización de <i>fail-safe</i> de la placa funciona y las líneas no flotan.
T2 Topología	Los tres buses están puenteados como indican los <i>jumpers</i> y no hay ni un cruce entre buses (0 enlaces no esperados).
T3 Multipunto	6/6 esclavos del bus A oyen íntegro al maestro, hasta 460 kbaudios.
T4 Velocidad máx.	Bus A: 460 kbaudios . Buses B y C: 1 Mbaudios . Por encima, la tasa de error se dispara.
T5 Limitador SLO	Con el bit SLO a 1, los tres buses llegan limpios a 4 Mbaudios .
T6 Vuelta del bus	<i>Guard time</i> mínimo de 0 μs : el control de dirección del transceptor suelta la línea a tiempo y no hace falta guarda en software.
T7 Diafonía	0 bytes de fuga del bus A hacia los buses B y C.
T8 Colisión	La trama sale corrupta, como debe, y el bus se recupera : el intercambio siguiente es limpio.
T9 Estabilidad	10 s de tráfico continuo: bus A a 460 kbaudios, tasa de error de bit (BER) de 147 ppm; buses B y C a 1 Mbaudios, BER de 0 .

Tabla 4.2: Resultado de los nueve tests de caracterización de la PCB.

El contador de fallos de transmisión de la campaña completa es **cero**: ningún envío quedó sin transmitir, por lo que todas las cifras corresponden a bytes que efectivamente viajaron por el bus. Esta distinción es relevante para la validez de los datos: distinguir «el transporte no llegó a transmitir» de «el bus corrompió los bytes» es esencial, ya que una primera tanda de medidas que no separaba ambos casos daba los buses B y C como defectuosos de forma incorrecta.

4.3.3. Efecto del limitador de *slew rate* sobre la velocidad máxima

El pin SLO del transceptor LTC2865 es un habilitador de modo lento **activo a nivel bajo**: con el valor por defecto (0) el limitador de *slew rate* está activo y limita el driver a 250 kbps nominales; con el bit a 1 el limitador se desactiva y el driver alcanza hasta 20 Mbps. Las medidas de esta sección se obtuvieron con el limitador desactivado (SLO = 1). En esa configuración los tres buses operan de forma limpia a **4 Mbaudios**, cuatro veces la velocidad del modo por defecto. La comparación, repetida en dos tandas independientes, se recoge en la figura 4.8.

Este comportamiento lo fija la hoja de características del transceptor **LTC2865** [23], según la cual el pin SLO es activo a nivel bajo:

«SLO (Slow Mode Enable): a low input switches the transmitter to the slew rate limited 250 kbps max data rate mode. A high input supports 20 Mbps.»

El bit de configuración viaja del registro directamente al pin del chip, sin lógica intermedia. Con el valor por defecto (0) el driver queda limitado a 250 kbps nominales, y con el bit a 1 el limitador se desactiva y el driver alcanza hasta 20 Mbps.

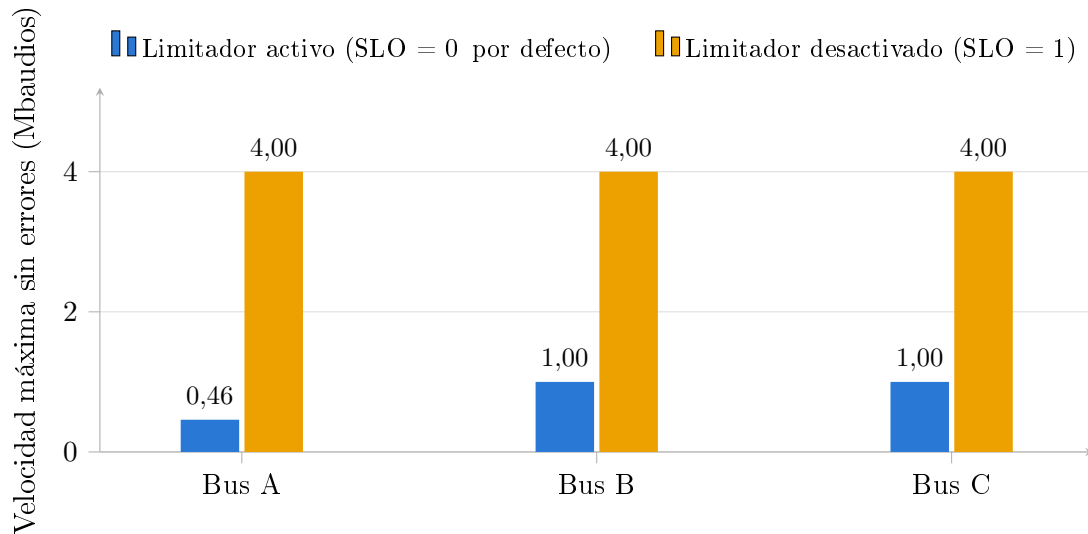


Figura 4.8: Velocidad máxima sin errores en cada bus según el valor del bit SLO.

El efecto se observa en las pruebas. La figura 4.9 muestra la tasa de error del bus multipunto (bus A, RS485 con 7 nodos), el más exigente de los tres, para ambos valores del bit. Con el limitador activo, que es el valor por defecto, el enlace se degrada a partir de 460 kbaudios y colapsa por completo a 2 Mbaudios, coherente con el máximo nominal de 250 kbps del chip en ese modo; desactivándolo, la tasa de error es nula en todo el barrido. Los puntos de 3 y 4 Mbaudios no existen en la curva de SLO = 0 porque el barrido se interrumpe en cuanto un punto no deja pasar ningún byte correcto.

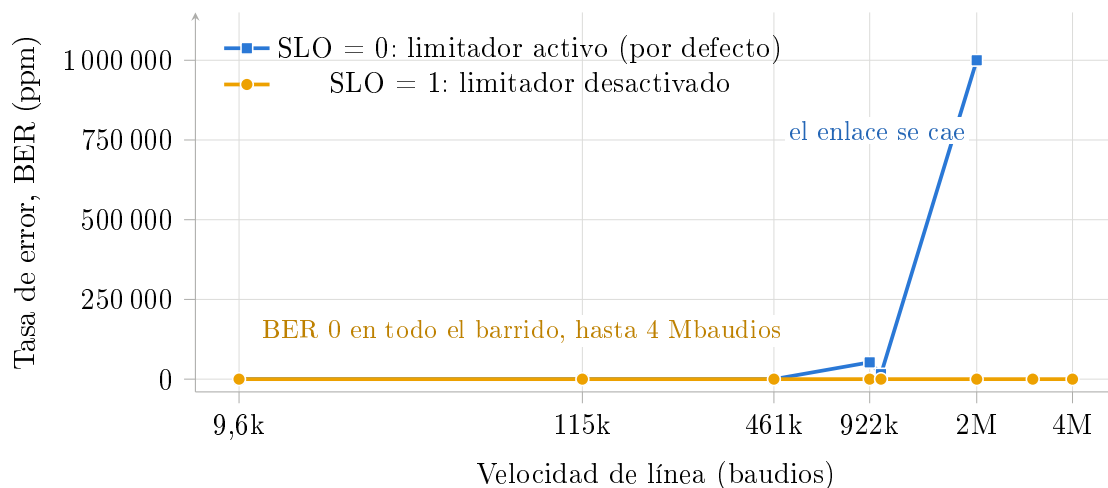


Figura 4.9: Tasa de error del bus multipunto frente a la velocidad, con y sin limitador de *slew*.

4.3.4. Comportamiento del bus multipunto

El bus A alcanza su límite antes que los otros dos (460 kbaudios frente a 1 Mbaudios con el mismo ajuste de SLO). Este comportamiento es coherente con su naturaleza y no constituye un defecto de la placa: siete nodos conectados a un bus compartido cargan más la línea y generan más reflexiones que un enlace punto a punto. Con el bit SLO a 1, en cambio, los tres buses alcanzan por igual los 4 Mbaudios (figura 4.8): esta penalización

del bus multipunto solo se observa con el limitador de *slew rate* activo, que es su valor por defecto.

El test T3 lo cuantifica en la tabla 4.3: registra cuántos de los seis esclavos reciben la trama del maestro **íntegra** a cada velocidad.

Velocidad	9 600	115 200	460 800	921 600	1 M
Esclavos íntegros (de 6)	6	6	6	5	0
Esclavos parciales	0	0	0	1	6

Tabla 4.3: Integridad del bus multipunto (T3) frente a la velocidad, con SLO = 0.

La transición no es gradual sino abrupta: a 460 kbaudios los seis esclavos reciben la trama completa; a 921 kbaudios uno la recibe troncada; a 1 Mbaudios ninguno la recibe entera. Este punto de ruptura define la velocidad de operación del bus, y es el que el bit SLO desplaza hasta 4 Mbaudios.

4.3.5. Integridad, aislamiento y estabilidad

El resto de la campaña confirma que la placa se comporta correctamente en los aspectos que no dependen de la velocidad:

Polarización de reposo (T1). Con los catorce transceptores en silencio y a la escucha durante 300 ms, no aparece **ningún byte espurio** en ningún canal. Es la comprobación de que la red de polarización de *fail-safe* está presente y cumple su función: si la línea flotara, el receptor amplificaría el ruido y lo entregaría como datos.

Aislamiento entre buses (T7). Saturando el bus A con una ráfaga a su velocidad máxima, no se filtra **ningún byte** a los buses B y C. No existe acoplamiento apreciable entre pistas ni cruces de cableado, lo que la matriz de conectividad de T2 confirma de forma independiente: cero enlaces no esperados.

Vuelta del bus (T6). En *half-duplex*, el esclavo no puede responder hasta que el maestro haya liberado la línea. Barriendo el tiempo de guarda de 0 a 1 000 μ s, las ocho respuestas llegan íntegras **ya con guarda nula**: el control de dirección que genera el propio transceptor libera la línea a tiempo, por lo que un protocolo construido sobre este nivel no necesita añadir guarda por software.

Colisión (T8). Cuando dos esclavos emiten simultáneamente, la trama resulta corrupta por el AND cableado de los drivers, pero el bus **se recupera** y el intercambio siguiente es correcto. Ningún driver permanece conduciendo de forma indefinida.

Estabilidad sostenida (T9). Diez segundos de tráfico continuo por bus, a la velocidad máxima limpia de cada uno:

Bus	Velocidad	Tramas	Bytes	BER
A (RS485 multipunto)	460 800	6 781	433 984	147 ppm
B (RS422)	1 000 000	14 408	922 112	0
C (RS422)	1 000 000	14 466	925 824	0

Tabla 4.4: Tasa de error en régimen permanente (T9), 10 s de tráfico continuo por bus.

Los dos buses RS422 no cometen **ningún error** en casi un millón de bytes cada uno. El bus multipunto se queda en 147 ppm, una cifra baja pero no nula, coherente con la mayor exigencia de una línea compartida por siete nodos cerca de su límite de velocidad.

4.3.6. Limitaciones del método de medida

Deben tenerse en cuenta tres precisiones sobre el alcance de estos resultados:

Los conteos parciales de la matriz de conectividad no miden la calidad del enlace. En la matriz de T2 aparecen enlaces con 25 de 32 bytes correctos, mientras que T3 y T4 a esa misma velocidad dan seis de seis esclavos íntegros y BER nulo sobre 1024 bytes. Se trata de un artefacto del método de T2, que espera un tiempo fijo y lee una sola vez: si un byte llega tarde se pierde y descuadra la comparación. T2 es válido para determinar el **mapa** de conectividad, que es su propósito, pero no para cuantificar la calidad.

Los ocho enlaces ausentes en la matriz no constituyen un fallo. De los sesenta enlaces esperados se miden cincuenta y dos. Los ocho ausentes corresponden a los esclavos de los buses B y C, que no se oyen entre sí porque esos buses son RS422 punto a punto en estrella desde el maestro, y no multipunto. La topología medida coincide exactamente con la física de la placa.

El contador de errores de hardware de T9 no es fiable, por lo que no se ha empleado en ninguna conclusión: marca millones de errores en tandas cuyo BER medido es nulo, lo que indica que contabiliza eventos que no son errores de trama. Las cifras de BER de la tabla 4.4 se calculan comparando byte a byte lo recibido con lo enviado.

4.4. Comparativa de las arquitecturas de transporte PS-PL

Esta sección recoge la medida de las tres arquitecturas de transporte descritas en la sección 3.7, con la que se cierra el segundo objetivo del trabajo: seleccionar la que mejor escala en rendimiento, en ocupación de la lógica programable y en consumo de recursos de CPU. A diferencia de las secciones anteriores, la campaña **no involucra ninguna de las tres placas**: los buses se cierran dentro de la propia lógica programable, de modo que lo que se mide es el camino de datos entre la PL y el PS y no la capa física.

4.4.1. Condiciones de la campaña

Como paso previo se cargó un *bitstream* con 14 transceptores instanciados (canales 0 a 13), con los canales adicionales conectados a los pines externos del conector J3 de la

ZCU102. El sistema opera correctamente con todos ellos, de modo que la arquitectura de ISR maestra con tareas *worker* individuales escala hasta el número máximo de instancias soportado. La prueba equivalente sobre la placa de comunicación serie fabricada se recoge en la sección 4.3.

Las tres variantes se ejecutaron sobre la misma ZCU102, con el mismo transceptor serie de catorce canales, el mismo programa de pruebas (`bench_main.c`) y los buses cerrados por el módulo de *loopback* de la PL descrito en la sección 3.7.6. Lo único que difiere entre las tres corridas es el transporte y su driver.

Limitación del método de medida en la variante DMA14. El módulo de *loopback* no llegó a operar sobre esta variante: su autodiagnóstico señaló el lazo de recepción como abierto y no se recibió ningún byte. Por el coste en lógica y el límite de líneas de interrupción que se exponen en la sección 3.7, la depuración de ese lazo no se llevó más allá. Las magnitudes que dependen de la recepción (latencia, *throughput*, multipunto, desbordamiento y recuperación) figuran con un guion en las tablas: su valor nulo **no mide el transporte**, sino la ausencia de lazo. Las cifras válidas de esa corrida son el coste de hardware, la huella de memoria y el coste en interrupciones de la transmisión.

4.4.2. Coste de hardware

Las tres arquitecturas están implementadas sobre el mismo dispositivo (XCZU9EG, 274 080 LUT, 548 160 registros y 912 bloques de RAM), por lo que las cifras de la tabla 4.5 salen directamente de los informes de utilización de Vivado y son comparables sin normalización alguna.

Variante	LUT	%	Registros	BRAM	Potencia	WNS	LUT/canal
GPIO	6 434	2,35 %	7 726	7	3,52 W	4,69 ns	460
DMA14	40 217	14,67 %	53 943	35	3,86 W	4,05 ns	2 873
MCDMA	14 191	5,18 %	16 438	18	3,62 W	5,14 ns	1 014

Tabla 4.5: Ocupación de la lógica programable de las tres variantes sobre el XCZU9EG.

La figura 4.10 representa las tres magnitudes normalizadas como porcentaje de los recursos disponibles en el dispositivo, lo que permite dibujarlas sobre un mismo eje. El banco de catorce DMA triplica el consumo de la solución multicanal para el mismo número de canales.

La cifra que resume la comparación es la última columna de la tabla. El banco de catorce AXI DMA cuesta **2,8 veces más lógica que el MCDMA** para exactamente el mismo número de canales, porque replica catorce motores completos en lugar de compartir uno multicanal. La variante GPIO es la más pequeña, porque apenas contiene los transceptores y unos pocos registros.

El margen de *timing* (WNS) es positivo y holgado en las tres, por lo que ninguna cierra al límite y las diferencias de rendimiento no son atribuibles a la implementación física.

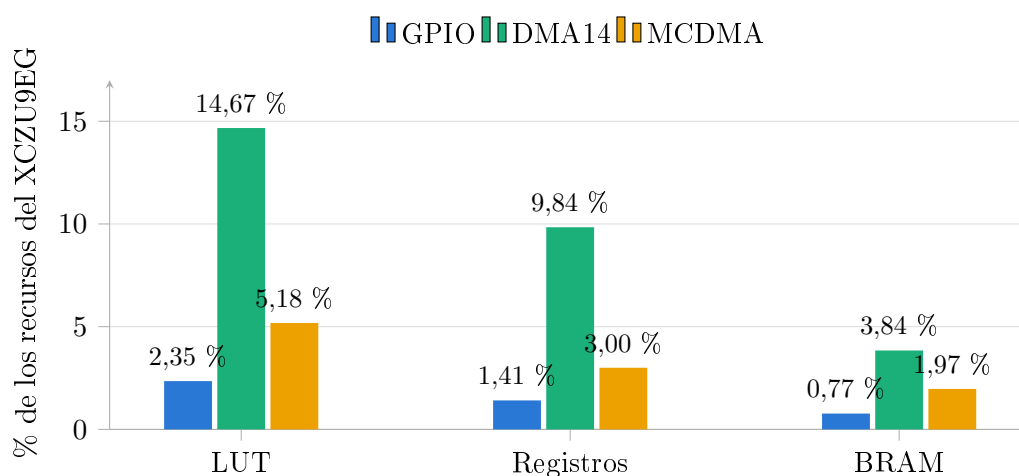


Figura 4.10: Ocupación de la FPGA por variante.

4.4.3. Coste de software

La tabla 4.6 recoge lo que cuesta cada variante del lado del PS: el tamaño del código del driver y los recursos que reserva en ejecución.

Variante	Código	RAM del driver	Líneas IRQ	Tareas	Semáforos
GPIO	3 099 B	114 688 B	1	14	28
DMA14	3 036 B	115 584 B	2	1	15
MCDMA	3 884 B	122 434 B	3	1	17

Tabla 4.6: Huella de software del driver de cada variante.

La memoria total reservada es prácticamente la misma en las tres, en torno a 112–120 KB, dominada por los *buffers* circulares de 4 KB por canal, aunque su origen difiere: la variante GPIO los pide con `malloc` y las de DMA los reservan estáticamente, porque el motor necesita direcciones fijas y alineadas.

La diferencia significativa está en las **primitivas de RTEMS**. La variante GPIO necesita **una tarea de recepción por canal** (catorce tareas y veintiocho semáforos) porque cada canal drena su propio FIFO hardware de forma independiente. Las dos variantes con DMA operan con **una sola tarea de despacho**, ya que el motor entrega paquetes completos y la ISR solo tiene que señalar quién ha terminado. Son trece tareas menos, con sus trece pilas y sus correspondientes cambios de contexto.

4.4.4. Coste en interrupciones

La métrica cuenta cuántas veces se expropia la CPU por cada kilobyte movido, acumulado sobre la sesión completa de pruebas. La tabla 4.7 recoge el desglose y la figura 4.11 lo representa a escala, en la que las barras de DMA14 y MCDMA quedan por debajo de la resolución del gráfico. La cifra de DMA14 recoge solo la transmisión.

Variante	IRQ/KB	IRQ TX	IRQ RX	Bytes TX	Bytes RX
GPIO	1 250	22 161	129 382	22 161	101 890
DMA14	3 *	85	—	23 132	—
MCDMA	3	194	192	23 132	101 472

* Solo transmisión: sin lazo de recepción, la cifra no es comparable.

Tabla 4.7: Interrupciones por kilobyte transferido.

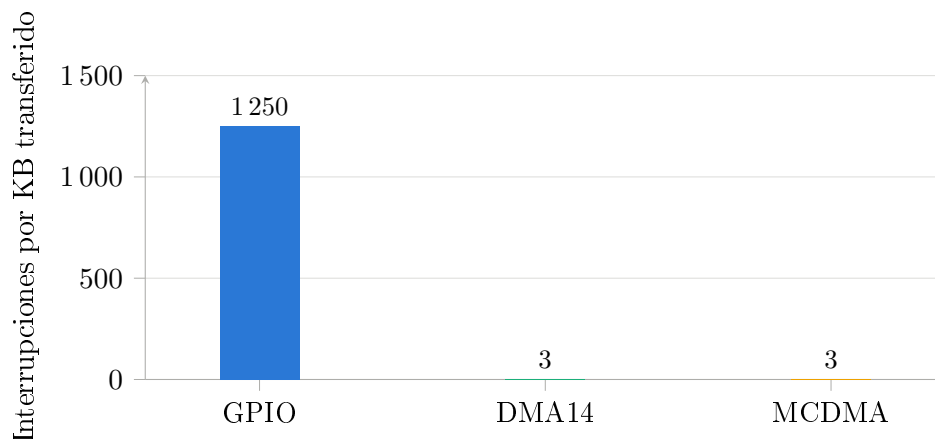


Figura 4.11: Interrupciones por kilobyte transferido, por variante.

En la variante GPIO, el número de interrupciones de transmisión **coincide exactamente con el número de bytes transmitidos** (22 161 y 22 161): es la confirmación empírica de que el procesador toca cada byte. En recepción ocurre lo mismo, con un sobrecoste añadido por los eventos de estado. El resultado son **1 250 interrupciones por kilobyte**.

El MCDMA mueve la misma cantidad de datos con **194 interrupciones de transmisión y 192 de recepción**: tres por kilobyte, unas **cuatrocientas veces menos**. La CPU pasa de estar atendiendo el bus a estar disponible para la aplicación.

4.4.5. Latencia

Tiempo desde justo antes de la llamada de envío hasta que el receptor tiene el *frame* de 11 bytes completo, sobre 50 muestras. Los estadísticos están en la tabla 4.8 y la comparación en la figura 4.12, que excluye la variante DMA14 por carecer de lazo de recepción.

Variante	Media (μ s)	p50	p95	Máx.	Timeouts
GPIO	4 606	4 608	4 611	4 611	0
DMA14	—	—	—	—	50
MCDMA	1 530	1 529	1 535	1 535	0

Tabla 4.8: Latencia de entrega de un *frame* de 11 bytes a 115 200 8N1.

El MCDMA es **tres veces más rápido** que el GPIO, y con una dispersión muy baja: la diferencia entre la mediana y el máximo es de 6 μ s sobre 50 muestras, lo que indica un

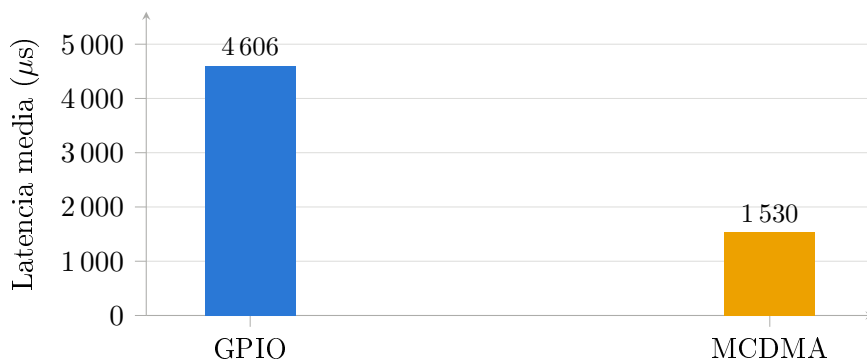


Figura 4.12: Latencia media de entrega de un *frame* de 11 bytes.

comportamiento notablemente determinista, una propiedad especialmente valiosa en un sistema de tiempo real embarcado.

4.4.6. *Throughput*

A 115 200 baudios en formato 8N1, cada byte ocupa 10 bits de línea, por lo que el techo físico de un canal es de **11 520 B/s**. Esa es la referencia contra la que hay que leer la primera columna de la tabla 4.9, sobre la que está calculado su porcentaje, y la que aparece como línea de trazos en la figura 4.13.

Variante	1 canal	3 maestros	6 receptores	Reps. sin pérdidas
GPIO	3 471 (30 %)	10 411	20 838	5 / 5
DMA14	—	—	—	—
MCDMA	11 461 (99 %)	34 303	68 504	5 / 5

Tabla 4.9: *Throughput* en B/s por variante.

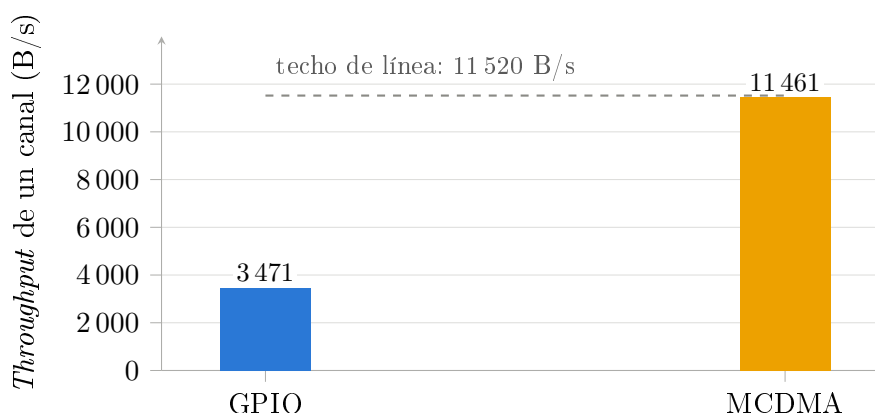


Figura 4.13: *Throughput* de un canal frente al techo físico de la línea.

El MCDMA alcanza el **99 % del techo físico de la línea**: el transporte deja de ser el cuello de botella y el único límite pasa a ser la propia línea serie, que es exactamente el objetivo de diseño. La variante GPIO se queda en el 30 %: dos de cada tres bytes que la línea podría llevar se pierden atendiendo interrupciones.

Con carga concurrente la diferencia se amplía en lugar de reducirse: con seis receptores simultáneos en el bus multipunto, el MCDMA sostiene **68 504 B/s** frente a los 20 838 del GPIO, sin perder ningún byte.

4.4.7. Barrido de velocidad

Se reprograman los catorce transceptores a cada velocidad y se mide un bloque de 1 KB en cada punto. La tabla 4.10 recoge los siete puntos del barrido, omitiendo la variante DMA14 por carecer de lazo de recepción, y la figura 4.14 los representa en escala logarítmica, donde la diferencia de comportamiento entre ambas arquitecturas es inmediata.

Variante	115 200	230 400	460 800	921 600	1 000 000	2 000 000	4 000 000
GPIO	3 471	4 087	4 485	4 714	4 733	4 848	4 907
MCDMA	11 458	22 789	45 104	88 298	96 186	183 053	344 781

Tabla 4.10: *Throughput* (B/s) frente a la velocidad de línea (baudios).

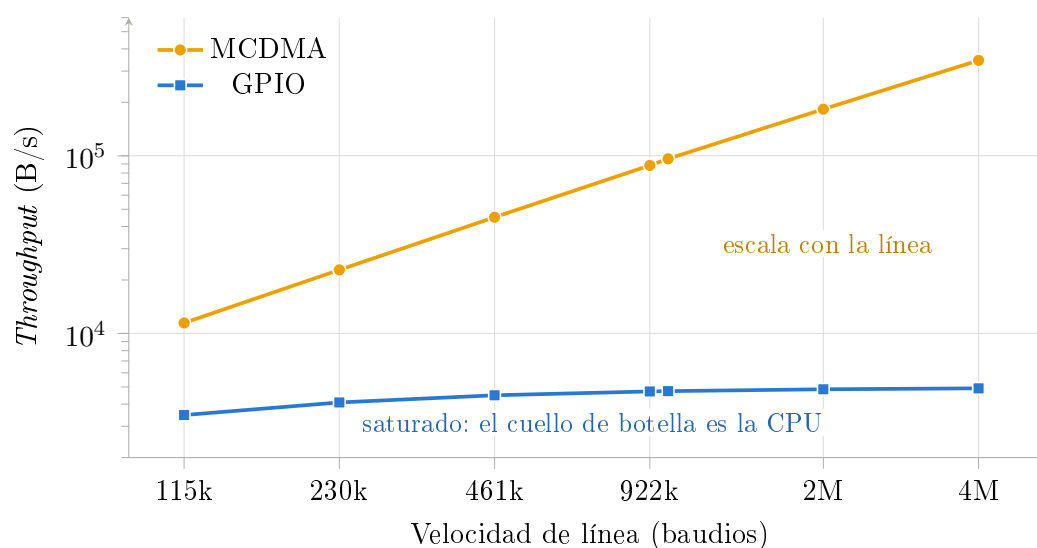


Figura 4.14: *Throughput* frente a la velocidad de línea, en escala logarítmica.

El barrido delimita la naturaleza del límite de cada arquitectura:

- El **GPIO se satura**. Multiplicar la velocidad de línea por 35 (de 115 200 a 4 Mbaudios) mejora el *throughput* un 41 %, de 3 471 a 4 907 B/s. La curva es plana porque **el cuello de botella no es la línea, es la CPU**: el coste de una interrupción por byte no depende de lo rápido que vaya el bus. Por rápido que se haga el hardware serie, el sistema no aprovechará esa velocidad.
- El **MCDMA escala linealmente**. El *throughput* crece de forma prácticamente proporcional a la velocidad de línea, hasta **344 781 B/s** a 4 Mbaudios: **70 veces** lo que consigue el GPIO en el mismo punto. La curva sigue a la línea porque el transporte ya no interviene.

En los siete puntos, y en ambas variantes, el número de errores de transporte es **cero**: la saturación del GPIO no produce corrupción, solo pérdida de rendimiento.

4.4.8. Robustez

Dos pruebas adicionales verifican que la ganancia de rendimiento no se paga con fragilidad: el envío de tres *frames* corruptos seguidos de uno válido (T7) y el envío de 8 KB sin que la aplicación vacíe un *ring* de 4 KB (T5), diseñado explícitamente para desbordar. La tabla 4.11 recoge el resultado de ambas.

Variante	<i>Frames</i> corruptos	Se recupera	Bytes aceptados	Bytes descartados
GPIO	3 / 3	sí	24 576	16 246
DMA14	3 / 3	—	—	—
MCDMA	3 / 3	sí	24 576	24 576

Tabla 4.11: Rechazo de *frames* corruptos (T7) y desbordamiento del *ring* (T5).

Ambas variantes rechazan los tres *frames* corruptos y siguen aceptando el *frame* válido posterior: el error no deja el driver en un estado inconsistente.

Los bytes descartados de la última columna **no son errores de transporte**, que son cero en las tres variantes: cuentan un byte cada vez que un dato llega correctamente pero **no cabe en el *ring* de software porque nadie lo ha leído**. La mayor parte procede de la prueba T5, cuyo propósito es justamente ese, y el resto de los canales que nadie lee en las pruebas de un solo maestro: al ser un bus multipunto, los seis receptores reciben cada bloque.

El número del MCDMA es *mayor* que el del GPIO por la velocidad de entrega, no por un defecto. En la variante GPIO el cuello de botella de interrupciones frena la entrega, con lo que el *ring* se llena más despacio y desborda menos; el MCDMA entrega a la velocidad de la línea todo lo que ésta produce y, en consecuencia, sobra más en los *buffers* que la aplicación de prueba deliberadamente no vacía. **Un contador de descartes alto corresponde aquí a un transporte que satura los *rings*, no a uno defectuoso.** Estas cifras son las de la campaña final, con las treinta y seis comprobaciones en verde; las corridas anteriores a las correcciones de contrapresión y de tamaño de paquete descritas en la sección 3.7.5 entregaban menos datos a los *rings* y no son comparables.

4.4.9. Selección de la arquitectura de transporte

La tabla 4.12 reúne en una sola vista las magnitudes de los apartados anteriores.

	GPIO	DMA14	MCDMA
Lógica (LUT)	6 434	40 217	14 191
IRQ/KB	1 250	3 (solo TX)	3
<i>Throughput</i> de 1 canal	3 471 B/s (30 %)	—	11 461 B/s (99 %)
Latencia	4 606 μ s	—	1 530 μs
Escala con el baudrate	no	—	sí
Líneas IRQ necesarias	1	28 (reducidas a 2)	3
Complejidad de diseño	baja	media	alta

Tabla 4.12: Síntesis de la comparativa.

Los datos sostienen que **el MCDMA con puente VHDL es la arquitectura adecuada para este sistema**. Su ventaja procede de una causa concreta y medible: **mueve paquetes en lugar de bytes**. Las 1 250 interrupciones por kilobyte de la variante GPIO se convierten en 3, y todo lo demás (la latencia tres veces menor, el 99 % del techo de línea, la escalabilidad con la velocidad) es consecuencia de eso.

La variante DMA14 resuelve el problema correcto, quitar a la CPU de en medio, por un camino que no escala. Replicar catorce motores cuesta 2,8 veces más lógica y, sobre todo, exige veintiocho líneas de interrupción cuando el silicio solo ofrece ocho: el límite no es del diseño, sino del dispositivo. A ello se suma un argumento de futuro: un 14,7 % del dispositivo es incompatible con la triplicación TMR que exigiría endurecer el diseño frente a la radiación, mientras que el 5,2 % del MCDMA deja ese margen abierto. Ese conjunto de razones cerró la inversión en esta variante cuando su lazo de recepción no llegó a funcionar, y justifica el diseño del puente VHDL de la variante C: de las tres arquitecturas es la única que satisface a la vez el presupuesto de área y el de líneas de interrupción del dispositivo.

El GPIO es una opción razonable **para pocos canales a baja velocidad**: ocupa menos de la mitad de lógica que el MCDMA (6 434 LUT frente a 14 191, tabla 4.5), no necesita mapeo de MMU ni gestión de coherencia de caché, y su driver es sensiblemente más simple. Los datos acotan su uso: **no sirve para catorce canales**, y su limitación no se cura subiendo la velocidad de la línea, porque el cuello de botella está en el otro extremo.

5. Conclusiones y líneas futuras

5.1. Conclusiones

El objetivo principal de este trabajo era desarrollar una plataforma funcional y validada de comunicación con periféricos serie sobre el MPSoC Zynq UltraScale+ ZCU102, que sirviera de base para el ordenador de a bordo del proyecto LINCE. Ese objetivo se desglosó en la sección 1.2 en seis objetivos específicos, y las conclusiones se ordenan aquí siguiéndolos uno a uno, para poder declarar en cada caso qué se ha conseguido y con qué evidencia.

El primer objetivo, diseñar un transceptor serie configurable en VHDL, se ha cumplido. El bloque desarrollado opera sobre los estándares RS422 y RS485 y admite la configuración completa del formato de trama en tiempo de ejecución: velocidad entre 50 y 4 000 000 de baudios sobre una tabla de 54 entradas, de 5 a 9 bits de datos, cinco modos de paridad, tres campos de parada (incluido el de 1,5 bits) y los dos órdenes de bit. Su base de tiempos, un NCO de 32 bits validado velocidad a velocidad con un *testbench* dedicado, presenta un error nulo o inferior a 1 ppm en la mayoría de las velocidades estándar y de 120 ppm en el peor punto del rango, dos órdenes de magnitud por debajo de la tolerancia habitual de un receptor UART. El canal completo supera además el barrido exhaustivo de las 150 configuraciones posibles, 450 tramas comparadas dato a dato sin ningún error (sección 3.2.7), y catorce instancias del bloque funcionan de forma simultánea sobre la placa.

El segundo objetivo, diseñar la arquitectura de transporte entre la lógica programable y el sistema de procesamiento, es el de mayor recorrido del trabajo y se ha cumplido con la arquitectura MCDMA que gobierna los catorce canales del sistema final. No se seleccionó una alternativa sobre el papel: se implementaron y midieron **tres arquitecturas completas** (AXI GPIO, catorce AXI DMA y un AXI MCDMA con puente VHDL propio) sobre el mismo transceptor y con el mismo programa de pruebas. La comparación es concluyente. La arquitectura de partida, basada en AXI GPIO, **interrumpe al procesador una vez por cada byte** (1 250 interrupciones por kilobyte), lo que la deja en el 30 % del techo físico de la línea y, sobre todo, la vuelve insensible a la velocidad del enlace: de 115 200 a 4 Mbaudios su rendimiento apenas mejora un 41 %, porque el cuello de botella no está en la línea sino en la CPU. La solución final, un MCDMA multicanal con un puente en VHDL que multiplexa los catorce canales, **baja a 3 interrupciones por kilobyte**, alcanza el **99 % del techo físico** de la línea, reduce la latencia a un tercio y, a diferencia de la anterior, escala linealmente con la velocidad hasta los 345 kB/s a 4 Mbaudios. El camino intermedio, replicar catorce controladores DMA, se hizo funcionar en placa pero resultó inviable no por rendimiento sino por recursos: cuesta 2,8 veces más lógica y exige veintiocho líneas de interrupción cuando el MPSoC solo ofrece ocho, un consumo de área incompatible, además, con una futura triplicación TMR de la

lógica para tolerancia a la radiación. Ese límite del silicio es lo que justificó el diseño del puente propio.

El tercer objetivo, un driver completo en C sobre RTEMS 7, se ha cumplido en sus tres versiones. El driver abstrae por completo el acceso al hardware tras una API pública de seis funciones, descubre en arranque cuántos transceptores hay instanciados en el *bitstream* (de modo que el mismo binario sirve para cualquier configuración, de un canal a catorce) y gestiona la comunicación orientada a interrupciones sin bloquear al procesador. La separación entre la ISR maestra y las tareas *worker* mantiene mínimo el tiempo en contexto de interrupción y preserva el determinismo propio de un sistema de tiempo real. Se escribió una versión del driver para cada arquitectura de transporte, lo que permitió que la comparativa del segundo objetivo midiera transportes y no implementaciones desiguales.

El cuarto objetivo, diseñar y fabricar una placa que ejercitara los catorce transceptores sobre topologías realistas, se ha cumplido y la placa se ha caracterizado eléctricamente. Sus catorce canales se reparten en un bus RS485 multipunto de siete nodos y dos buses RS422 *master/slave*, configurables por *jumper* sin recablear. La batería de nueve tests recorre la polarización de reposo, el mapa de conectividad real, la integridad del bus multipunto, la velocidad máxima de cada bus, la diafonía, la colisión y la estabilidad sostenida, y la placa los supera todos: cero bytes espurios en reposo, cero fugas entre buses, recuperación limpia tras una colisión y tasa de error nula en los dos buses RS422 tras diez segundos de tráfico continuo. La campaña acotó además el efecto del **limitador de *slew rate*** de los transceptores. El pin *SLO* del LTC2865 es un habilitador de modo lento **activo a nivel bajo**: con él a 0, que es el valor por defecto del software, el driver queda limitado a los 250 kbps nominales del chip en ese modo, pese a que la macro correspondiente se llama «*SLO* desactivado». Con el limitador desactivado (*SLO* = 1), los tres buses pasan de 460 kbaudios–1 Mbaudios a **4 Mbaudios** con tasa de error nula: cuatro veces más velocidad sin modificar el cobre ni resintetizar la lógica. Queda pendiente invertir ese valor por defecto y renombrar la macro.

El quinto objetivo, diseñar y fabricar las dos placas de expansión bajo especificación de Indra, se ha cumplido. La placa CDHS expone CAN redundante, tres canales RS422/RS485, cuatro líneas de PWM para calentadores y un ADC de termistores sobre seis conectores D-Sub-9; la placa AOCS, los canales serie, el control de motores por PWM y dos enlaces SpaceWire sobre ocho. Las tres tarjetas del trabajo se fabricaron en el propio laboratorio mediante soldadura por reflujo con *stencil*, y las tres se montaron sin retrabajos por haberse diseñado directamente contra las capacidades de proceso del fabricante. La restricción de 1,8 V del carril VADJ del FMC condicionó la selección de componentes y obligó a descartar las piezas con herencia de vuelo, una decisión que se declara explícitamente: las tarjetas son hardware de validación funcional, no hardware de vuelo. La placa AOCS presenta además un error de conexión en el eje Z del bloque de motores, corregido en la revisión V2 del esquemático.

El sexto objetivo, integrar y validar el sistema completo, se ha cumplido salvo en una interfaz. La validación experimental confirma el funcionamiento de las interfaces RS422 y RS485 de las placas CDHS y AOCS, del bus CAN de la CDHS (26 de 26 pruebas superadas, incluida la comprobación del efecto de las resistencias de terminación sobre la integridad de la señal diferencial), del ADC por SPI, cuyas lecturas siguen linealmente la tensión aplicada, y de las señales de PWM de ambas placas. La única interfaz que quedó sin validar es **SpaceWire**, por una razón declarada desde el inicio: la placa expone la capa

física LVDS, pero ejercitarla exige implementar en la lógica programable la codificación *Data-Strobe*, la máquina de estados del enlace y los niveles superiores del estándar, un desarrollo que por sí solo excede el alcance de este trabajo. Con menor peso, tampoco se completó la medida de recepción de la variante DMA14, descartada por recursos antes de depurar su lazo de prueba, ni se llegó a fabricar la revisión V2 de la placa AOCS.

En términos de volumen, el trabajo ha producido **20 188 líneas de código** de autoría propia (excluyendo todo lo generado automáticamente por las herramientas y el código de terceros), repartidas entre el RTL del transceptor y el puente, sus *testbenches*, el driver de RTEMS, las aplicaciones de prueba y las herramientas de automatización del flujo. El desglose completo se recoge en la tabla C.5 del Anexo C, y su distribución refleja las dos decisiones metodológicas que vertebraron el desarrollo: el código de verificación pesa prácticamente lo mismo que el hardware que verifica, y la automatización del flujo, con un 40 % del total, es la mayor partida individual del trabajo.

En conjunto, los seis objetivos específicos quedan cubiertos y el objetivo principal, con ellos: el resultado es una plataforma funcional, medida y documentada, con una arquitectura de transporte escogida sobre medidas tomadas en la propia plataforma, y no sobre estimaciones de catálogo. Esa arquitectura reduce la carga de interrupciones a 3 por kilobyte y ocupa menos lógica que la alternativa de catorce motores DMA, lo que deja margen de CPU y de área para el software de vuelo. El trabajo constituye así una contribución directa al proyecto LINCE y deja al laboratorio B105 una base sobre la que construir el subsistema de comunicaciones del ordenador de a bordo del satélite.

5.2. Líneas futuras

El trabajo deja abiertas varias tareas que continúan directamente lo desarrollado aquí, sobre la misma plataforma y sin cambiar sus decisiones de diseño.

Cerrar los pendientes detectados durante la validación. Son correcciones acotadas y ya identificadas: invertir el valor por defecto de la señal *SL0* en el driver y renombrar la macro correspondiente, de modo que los tres buses arranquen a 4 Mbaudios en lugar de quedar limitados a los 250 kbps del modo lento; fabricar la revisión V2 de la placa AOCS, cuyo esquemático ya corrige el error de conexión del eje Z del bloque de motores, y repetir sobre ella la batería de pruebas eléctricas; y completar la medida de recepción de la variante DMA14 para cerrar la tabla comparativa, aunque la arquitectura ya se haya descartado por recursos.

Poner en servicio la interfaz SpaceWire sobre la capa física ya disponible. Es la única interfaz de las placas que no llegó a validarse, y el hardware necesario está fabricado y medido: la tarjeta AOCS expone los dos enlaces con su señalización LVDS, sus repetidores y sus pares igualados en longitud. El trabajo pendiente queda íntegramente en la lógica programable y el software, y consiste en incorporar un controlador SpaceWire, propio o de terceros, y colgarlo del transporte MCDMA ya desarrollado, que admite canales adicionales sin rediseño. La verificación puede reutilizar el mismo esquema de este trabajo: simulación bloque a bloque y una campaña de pruebas en placa contra el enlace del otro conector.

Ampliar el driver y su campaña de pruebas a las condiciones de operación del satélite. El driver actual se ha validado con tráfico continuo de diez segundos sobre buses conocidos. Los pasos siguientes son medir el comportamiento en ejecuciones largas,

añadir el tratamiento y el recuento de errores de trama y de colisión que el bus RS485 multipunto puede provocar en vuelo, y exponer esas estadísticas a la capa de software superior. Todo ello se apoya en la API de cinco funciones ya definida y no exige tocar el RTL.

Endurecer la variante MCDMA mediante triplicación modular (TMR). La lógica programable comercial es susceptible a los *single event upsets* provocados por la radiación en órbita, y la mitigación habitual es triplicar la lógica e insertar módulos votadores. La tarea concreta es replicar el proyecto de la variante MCDMA triplicando su lógica propia, el puente VHDL y los transceptores serie, y comprobar que sigue superando la misma campaña de treinta y seis verificaciones. El presupuesto de área lo permite, y esa fue una de las razones de escoger esta arquitectura: triplicar el 5,2 % de ocupación del MCDMA mantiene el diseño en torno al 16 % del dispositivo, mientras que la misma operación sobre la variante DMA14 rondaría el 44 % (sección 4.4).

Más allá de estas tareas, la calificación para vuelo del subsistema es un trabajo en sí mismo. La protección de la memoria de configuración mediante *scrubbing*, la comparación sistemática de mecanismos de tolerancia a radiación o la migración a componentes con herencia de vuelo afectan al diseño de la placa entera, y no solo a la lógica desarrollada aquí.

Bibliografía

- [1] AMD/Xilinx. *ZCU102 Evaluation Board User Guide*. AMD/Xilinx, 2023. <https://docs.amd.com/v/u/en-US/ug1182-zcu102-eval-bd>. Copia disponible en tfm/00_docs/ug1182-zcu102-eval-bd.pdf del repositorio del proyecto.
- [2] Indra. Proyecto LINCE: Línea de industrialización de cargas de pago y plataformas espaciales. Indra Sistemas S.A., <https://www.ptelince.es/>, 2026. Consultado en junio de 2026.
- [3] Atlassian. Jira: Herramienta de gestión de proyectos ágiles. <https://www.atlassian.com/software/jira>, 2025. Consultado en 2025.
- [4] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [5] Jorge Alejandro Estefanía Hidalgo. Repositorio de código del TFM: transceptores serie configurables y transporte de datos sobre Zynq UltraScale+. <https://github.com/jaestefaniah27/tfm>, 2026. Consultado en agosto de 2026.
- [6] VITA Standards Organization. VITA 57.1: FPGA Mezzanine Card (FMC) standard. VITA Standards Organization, 2010.
- [7] Texas Instruments. The RS-485 design guide. Application Report SLLA272C, Texas Instruments, 2014.
- [8] Electronic Industries Alliance (EIA). Electrical characteristics of generators and receivers for use in balanced digital multipoint systems. TIA/EIA-485-A, 1998.
- [9] Texas Instruments. RS-422 and RS-485 standards overview and system configurations. Application Report SLLA070D, Texas Instruments, 2018.
- [10] Telecommunications Industry Association. Electrical characteristics of balanced voltage digital interface circuits. TIA/EIA-422-B, 1994.
- [11] European Cooperation for Space Standardization (ECSS). Space engineering – SpaceWire – links, nodes, routers and networks, 2008.
- [12] S. M. Parkes. *SpaceWire User's Guide*. STAR-Dundee Ltd., 2012.
- [13] International Organization for Standardization (ISO). Road vehicles — controller area network (CAN) — part 1: Data link layer and physical signalling. ISO 11898-1:2015, 2015.
- [14] Agencia Espacial Europea (ESA). CAN in space: Implementation guide and application notes. Technical report, TEC-ED & TEC-SW, ESA, 2015.
- [15] ARM Ltd. *AMBA AXI and ACE Protocol Specification (AXI3, AXI4, AXI4-Lite, ACE, ACE-Lite)*. ARM Ltd., 2011.
- [16] AMD/Xilinx. *AXI DMA v7.1 — LogiCORE IP Product Guide*. AMD/Xilinx, 2022. https://docs.xilinx.com/r/en-US/pg021_axi_dma.

- [17] AMD/Xilinx. *AXI Multichannel DMA (MCDMA) v1.1 — LogiCORE IP Product Guide*. AMD/Xilinx, 2022. <https://docs.xilinx.com/r/en-US/pg288-axi-mcdma>.
- [18] Jeffrey C. Mogul and K. K. Ramakrishnan. Eliminating receive livelock in an interrupt-driven kernel. *ACM Transactions on Computer Systems*, 15(3):217–252, 1997.
- [19] RTEMS Project. *RTEMS User Manual (RTEMS 7) — Cap. 2: Quick Start*. RTEMS Project, 2026. Guía paso a paso de instalación del *toolchain* y compilación de los BSP mediante el RTEMS Source Builder. Consultado en 2026. <https://docs.rtems.org/docs/main/user/start/index.html>.
- [20] RTEMS Project. *RTEMS User Manual (RTEMS 7) — Cap. 15: Source Builder (RSB)*. RTEMS Project, 2026. Consultado en 2026. <https://docs.rtems.org/docs/main/user/rsb/index.html>.
- [21] Reinhold P. Weicker. Dhrystone: A synthetic systems programming benchmark. *Communications of the ACM*, 27(10):1013–1030, 1984.
- [22] PCBWay. PCB capabilities: Manufacturing tolerances and design limits. <https://www.pcbway.com/capabilities.html>, 2025. Consultado en 2025.
- [23] Analog Devices. *LTC2862/LTC2863/LTC2864/LTC2865 — ±60V Fault Protected 3V to 5.5V RS485/RS422 Transceivers*. Analog Devices (Linear Technology), 2013. Documento 2862345fc. <https://www.analog.com/media/en/technical-documentation/data-sheets/2862345fc.pdf>.
- [24] Texas Instruments. ADS7950/51/52/53 — 12/10/8-bit, 1-MSPS, 4-/8-channel, serial interface, MicroPower sampling analog-to-digital converter. Technical Report SLAS605C, Texas Instruments, 2018.
- [25] AMD/Xilinx. *Zynq UltraScale+ MPSoC Technical Reference Manual*. AMD/Xilinx, 2023.
- [26] Parlamento Europeo y Consejo de la Unión Europea. Directiva 2012/19/UE sobre residuos de aparatos eléctricos y electrónicos (RAEE). Diario Oficial de la Unión Europea, L 197/38, 2012.
- [27] Parlamento Europeo y Consejo de la Unión Europea. Directiva 2011/65/UE sobre restricciones a la utilización de determinadas sustancias peligrosas en aparatos eléctricos y electrónicos (RoHS). Diario Oficial de la Unión Europea, L 174/88, 2011.
- [28] ESA Space Debris Office. ESA’s annual space environment report. Technical Report GEN-DB-LOG-00288-OPS-SD, European Space Agency, 2025. https://www.esa.int/Space_Safety/Space_Debris.

A. Aspectos éticos, económicos, sociales y ambientales

Este anexo recoge los aspectos éticos, económicos, sociales y ambientales del trabajo desarrollado, agrupados en los apartados siguientes.

B.1 INTRODUCCIÓN

Este trabajo se enmarca en el proyecto LINCE, una iniciativa del PERTE Aeroespacial para el desarrollo de microsátélites LEO liderada por Indra. El objetivo es desarrollar el subsistema de comunicaciones serie del ordenador de a bordo (OBC) del satélite, cubriendo tanto el diseño de hardware electrónico (PCBs de prueba) como el firmware embebido sobre un sistema operativo de tiempo real. Este tipo de tecnología tiene implicaciones relevantes en ámbitos sociales, económicos, ambientales y éticos que se analizan a continuación.

B.2 DESCRIPCIÓN DE IMPACTOS RELEVANTES RELACIONADOS CON EL PROYECTO

Los impactos relevantes del trabajo se agrupan en cuatro ámbitos.

Autonomía estratégica y tejido industrial (impacto económico y social). LINCE se financia a través del PERTE Aeroespacial con el objetivo declarado de que España disponga de capacidad propia para producir en serie plataformas de microsátélite y sus cargas de pago. El subsistema de comunicaciones del ordenador de a bordo es uno de los bloques que, de no desarrollarse aquí, habría que adquirir a proveedores extracomunitarios sujetos a controles de exportación. Un transceptor serie escrito en VHDL propio, con su código fuente disponible y su comportamiento medido y documentado, puede auditarse, modificarse y volver a sintetizarse sin depender de licencias ni de autorizaciones externas, a diferencia de un componente cerrado de terceros.

El desarrollo propio deja además conocimiento en el país. Un ingeniero que diseña y verifica un transceptor aporta más valor por hora que quien integra un producto ya definido por otro proveedor, y ese conocimiento (cómo se sintetiza un puente VHDL, cómo se caracteriza un driver serie a alta velocidad) permanece aunque cambie el contratista. Comprar el mismo subsistema fuera no lo deja.

Formar ese personal cualificado lleva años y exige experiencia en programas reales, que solo se adquiere participando en misiones. Retenerlo tampoco es sencillo: el perfil es internacionalmente móvil y compite con agencias y fabricantes de países con industria espacial

consolidada y con el sector tecnológico civil. Sin continuidad de proyectos en España, el resultado habitual es la fuga de titulados. El PERTE Aeroespacial no pretende sostener esa plantilla de forma indefinida con fondos públicos, sino alcanzar una masa crítica de empresas y personal que permita a la industria sostenerse después por contratos comerciales. Un trabajo de fin de máster dentro de un programa así forma parte de ese mecanismo: aplica conocimiento académico a un problema real, con las restricciones y los plazos de una misión concreta. Su resultado es alguien capacitado para seguir trabajando en el sector.

Doble uso civil y militar. Una plataforma de microsátélites LEO con capacidad de observación y de comunicaciones es tecnología de doble uso por definición: los mismos enlaces serie que transportan telemetría de un sensor térmico sirven a una carga de pago de vigilancia. El trabajo aquí desarrollado se sitúa en la capa más genérica posible del sistema (transporte de datos entre periféricos y procesador) y no incorpora ninguna funcionalidad específica de aplicación, pero sería deshonesto presentarlo como neutral: forma parte de un ordenador de a bordo cuyo uso final lo determina la carga de pago que se le conecte y el operador que la explote. La responsabilidad sobre ese uso recae en el marco regulatorio de exportación y en las decisiones del consorcio, no en el componente, pero queda declarada.

Residuos electrónicos y ciclo de vida del hardware. El trabajo ha producido seis placas de circuito impreso físicas (dos unidades de cada uno de los tres diseños), con más de doscientos componentes montados y conectores metálicos, además de las revisiones descartadas durante el desarrollo. Todo ello es residuo de aparatos eléctricos y electrónicos en el sentido de la Directiva 2012/19/UE [26]. Los componentes seleccionados cumplen la Directiva 2011/65/UE de restricción de sustancias peligrosas [27], y por eso la lista de materiales específica encapsulados y acabados libres de plomo.

El proyecto adopta un enfoque NewSpace: componentes comerciales (COTS) en lugar de piezas cualificadas para espacio. Esto reduce el coste y el plazo, pero acorta la vida útil esperada del hardware y aumenta la frecuencia de sustitución, con el consiguiente aumento del volumen de residuo por unidad de servicio prestado.

Sostenibilidad del entorno orbital. LINCE produce microsátélites de órbita baja destinados a operar en constelación. La densidad de objetos en LEO es el principal problema de sostenibilidad del sector, documentado anualmente por la Oficina de Basura Espacial de la ESA [28], y toda decisión que acorte la vida operativa de un satélite (por ejemplo, un fallo de aviónica no recuperable) adelanta su sustitución y, con ella, un lanzamiento adicional y un objeto más en órbita.

Consumo energético y presupuesto de potencia en órbita. La lógica programable es **el único elemento del MPSoC cuyo consumo depende directamente de las decisiones de diseño tomadas en este trabajo**: la estimación de Vivado da 3,52 W para la variante GPIO, 3,86 W para la de catorce controladores DMA y 3,62 W para la solución MCDMA finalmente seleccionada (tabla 4.5). En un microsátélite de 100–200 kg, alimentado por paneles solares y con un presupuesto de potencia cerrado, cada vatio de la aviónica es un vatio que no está disponible para la carga de pago. Este impacto se analiza en detalle en el apartado siguiente.

B.3 ANÁLISIS DETALLADO DE ALGUNO DE LOS IMPACTOS

Se analiza en detalle el **impacto energético**, por ser el único de los anteriores sobre el que este trabajo aporta medidas propias y sobre el que las decisiones de diseño tomadas

tienen efecto directo y cuantificable.

Planteamiento. La magnitud relevante en un satélite no es la potencia instantánea que consume la aviónica, sino la **energía que cuesta entregar un byte útil**. Dos arquitecturas que consumen lo mismo no son equivalentes si una transporta el triple de datos con esa potencia. La comparativa del capítulo 4 permite calcular esa magnitud, porque mide sobre el mismo dispositivo, el mismo transceptor y el mismo programa de pruebas tanto la potencia estimada por la herramienta de implementación como el caudal realmente sostenido.

Datos. Con seis receptores simultáneos sobre el bus multipunto, la variante GPIO sostiene 20 838 B/s con una potencia de 3,52 W y la variante MCDMA 68 504 B/s con 3,62 W (sección 4.4). La energía por byte transportado resulta ser:

Variante	Potencia	Caudal sostenido	Energía por byte
GPIO	3,52 W	20 838 B/s	169 μ J
MCDMA	3,62 W	68 504 B/s	52,8 μ J

La solución seleccionada consume 0,10 W más (un 2,8 %) y es **3,2 veces más eficiente por byte transportado**. La variante intermedia de catorce controladores DMA, con 3,86 W, es además la que más consume de las tres en términos absolutos.

Interpretación. El resultado no es una casualidad de la implementación: es la consecuencia directa de haber quitado a la CPU de la ruta de datos. La arquitectura GPIO interrumpe al procesador 1 250 veces por kilobyte recibido, y cada una de esas interrupciones es un ciclo de guardado y restauración de contexto que consume energía sin transportar información. El MCDMA baja a 3 interrupciones por kilobyte. La lógica programable añadida (el puente VHDL y el controlador multicanal) cuesta 0,10 W adicionales, pero libera a un procesador de cuatro núcleos que consume bastante más que eso cuando está ocupado atendiendo interrupciones en lugar de en espera de evento.

Consecuencias sobre el sistema completo. El margen liberado es doble y ambos efectos tienen lectura ambiental. El primero es el margen de CPU: un procesador que no está atendiendo interrupciones puede permanecer en espera de evento, lo que reduce el consumo medio del sistema de procesamiento y, en un satélite alimentado por paneles solares, deja potencia disponible para la carga de pago o permite dimensionar a la baja el generador fotovoltaico y las baterías, con la consiguiente reducción de masa lanzada. El segundo es el margen de área: la solución seleccionada ocupa el 5,18 % de la lógica del dispositivo frente al 14,67 % de la alternativa replicada, de modo que su triplicación modular redundante para tolerancia a radiación mantiene el diseño en torno al 16 % del dispositivo, mientras que la misma operación sobre la variante de catorce DMA rondaría el 44 % y resultaría inviable. Ese margen es lo que hace posible endurecer el sistema frente a los *single event upsets* (SEU), y un satélite que sobrevive a un evento de radiación en lugar de perderse es un satélite que no hay que sustituir: se evitan un lanzamiento y un objeto adicional en una órbita ya congestionada [28].

Caudal de misión. El cálculo anterior supone el canal trabajando a caudal sostenido. El caudal real de la misión no está fijado todavía, y conviene acotar la conclusión sin adornar esa falta de dato. Con tráfico alto por las interfaces, el argumento se sostiene por la energía por byte y por el margen de CPU. Con tráfico bajo, si el canal no se satura, la ventaja en energía por byte se diluye, pero la variante GPIO tampoco era una opción: ni por número

de canales ni por coste de interrupción. El cambio es favorable en los dos escenarios. Por eso la plataforma se dimensiona sin fijar si las interfaces servirán a telemetría, a la carga de pago o a ambas, y el margen liberado, de CPU y de área, vale mientras por ellas circule tráfico sostenido.

Limitaciones del análisis. Las cifras de potencia son la estimación del implementador de Vivado, no una medida sobre el carril de alimentación de la placa, por lo que sirven para comparar variantes entre sí, que es su propósito aquí, pero no como valor absoluto de consumo. El cálculo tampoco incluye el consumo del sistema de procesamiento, que es donde el efecto descrito debería ser mayor.

B.4 CONCLUSIONES

Desde el punto de vista **económico**, el trabajo contribuye a un objetivo declarado de política industrial: sustituir por desarrollo propio un bloque del ordenador de a bordo que de otro modo se adquiriría fuera de la Unión Europea. El coste directo del desarrollo, detallado en el Anexo B, es de un orden de magnitud inferior al de la adquisición y cualificación de una solución comercial equivalente, y el resultado queda en el laboratorio como base reutilizable.

Desde el punto de vista **social**, el impacto es indirecto pero real: la plataforma forma parte de una constelación de observación y comunicaciones cuya explotación afecta a la capacidad de vigilancia y a los servicios de comunicación disponibles. El trabajo se sitúa en una capa deliberadamente genérica del sistema, pero se declara aquí su condición de tecnología de doble uso en lugar de presentarlo como un componente neutral.

Desde el punto de vista **medioambiental**, el balance tiene dos signos. En el lado negativo, el trabajo genera residuo electrónico (seis placas fabricadas más las revisiones descartadas) y se inscribe en un enfoque NewSpace basado en componentes comerciales que acorta la vida útil del hardware y aumenta la frecuencia de sustitución. En el lado positivo, la arquitectura de transporte seleccionada es 3,2 veces más eficiente por byte transportado que la de partida y deja margen de área suficiente para endurecer el diseño frente a radiación, lo que alarga la vida operativa esperada del satélite y reduce la necesidad de lanzamientos de reposición. La selección de componentes se ha ceñido a la Directiva RoHS [27], y las placas descartadas deben gestionarse como residuo de aparatos eléctricos y electrónicos a través del canal habilitado por el laboratorio, conforme a la Directiva RAEE [26].

Desde el punto de vista **ético y de gestión**, el criterio seguido a lo largo de la memoria ha sido declarar las limitaciones en lugar de omitirlas: las tarjetas se identifican explícitamente como hardware de validación funcional y no de vuelo, se documenta la interfaz que quedó sin validar (SpaceWire) y el motivo, se declaran las limitaciones de cada método de medida y se señalan los errores de diseño detectados y sus correcciones. Las decisiones de arquitectura se sostienen sobre medidas propias y reproducibles, con el código y los bancos de prueba publicados [5], y no sobre estimaciones. Ese es el compromiso con la responsabilidad profesional que el ejercicio de la ingeniería exige y que este anexo pretende acreditar.

B. Presupuesto económico

El presupuesto de este trabajo se recoge en la tabla B.1. Los criterios aplicados para elaborarlo son los siguientes:

- **Mano de obra.** El trabajo se desarrolló a lo largo de veinte *sprints* de dos semanas (de octubre a julio), diez meses en total. El coste imputado es el real: la beca de colaboración con la que se retribuyó al autor, de 500 € mensuales.
- **Recursos materiales amortizables.** Equipamiento e instrumentación del laboratorio y licencias de las herramientas de diseño, imputados en proporción a los meses de uso efectivo sobre su período de amortización. Las licencias de Vivado ML Enterprise y Altium Designer se amortizan a un año por tratarse de suscripciones anuales.
- **Material fungible.** Componentes electrónicos y fabricación de las placas de circuito impreso. Las cifras de componentes proceden directamente de las listas de materiales de los tres diseños, para una tirada de dos unidades de cada placa, tomando el precio del proveedor primario. Los costes de fabricación (placa de cuatro capas más *stencil* en PCBWay [22]) son estimados.
- **Gastos generales y beneficio industrial.** Se aplican los porcentajes habituales del 15 % sobre el coste directo y del 6 % sobre la suma de costes directos e indirectos. El IVA aplicable es del 21 %.

COSTE DE MANO DE OBRA (coste directo)		Meses	Importe mensual	Total
Becario de colaboración (desarrollo VHDL, firmware y PCB)		10	500,00 €	5.000,00 €
COSTE TOTAL DE MANO DE OBRA				5.000,00 €

COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total
Placa de desarrollo AMD ZCU102	3.500,00 €	10	5	583,33 €
Estación de trabajo (síntesis y diseño de PCB)	1.800,00 €	10	5	300,00 €
Osciloscopio digital de 4 canales	2.500,00 €	4	8	104,17 €
Analizador lógico USB	400,00 €	4	5	26,67 €
Fuente de alimentación de laboratorio	600,00 €	4	8	25,00 €
Estación de soldadura y horno de reflujo	1.200,00 €	3	8	37,50 €
Multímetro de mesa	300,00 €	3	8	9,38 €
Licencia Vivado ML Enterprise (anual)	2.000,00 €	10	1	1.666,67 €
Licencia Altium Designer (anual)	3.000,00 €	5	1	1.250,00 €
COSTE TOTAL DE RECURSOS MATERIALES				4.002,72 €

COSTE DIRECTO (CD) = mano de obra + recursos materiales				9.002,72 €
--	--	--	--	-------------------

GASTOS GENERALES (costes indirectos)	15 %	sobre CD	1.350,41 €
BENEFICIO INDUSTRIAL	6 %	sobre CD + CI	621,19 €

MATERIAL FUNGIBLE			
Componentes de la placa de comunicación serie (2 unidades)	424,13 €		
Componentes de la placa CDHS (2 unidades)	116,77 €		
Componentes de la placa AOCS (2 unidades)	302,02 €		
Fabricación de las tres placas con <i>stencil</i> (2 unidades de cada una)	420,00 €		
Cableado, conectores, tarjeta microSD y consumibles de soldadura	180,00 €		
COSTE TOTAL DE MATERIAL FUNGIBLE	1.442,92 €		

SUBTOTAL PRESUPUESTO	12.417,24 €		
IVA APLICABLE	21 %	2.607,62 €	

TOTAL PRESUPUESTO	15.024,86 €		
--------------------------	--------------------	--	--

Tabla B.1: Presupuesto económico.

El coste total asciende a **quince mil veinticuatro euros con ochenta y seis céntimos** (15.024,86 €), IVA incluido. La mano de obra es la mayor partida individual, con un 40 % del subtotal, seguida de los recursos materiales amortizados con un 32 %, dentro de los cuales las licencias de las herramientas de diseño pesan más que el propio equipamiento; el hardware físicamente fabricado representa un 10 % del presupuesto.

El coste de mano de obra imputado es el realmente incurrido, correspondiente a una beca de colaboración, y no el que tendría el mismo trabajo contratado a precio de mercado. Con una tarifa de ingeniero junior de 30 €/hora sobre una dedicación equivalente de 800 horas, la partida de mano de obra pasaría de 5.000 € a 24.000 € y el total del presupuesto se situaría en torno a los 43.000 €. Esa segunda cifra es la referencia adecuada para estimar lo que costaría reproducir este desarrollo en un entorno industrial.

C. Mapas de señales, tabla del NCO, volumen de código y detalle RTL

Este anexo se organiza en tres bloques. El primero (C.1–C.5) reúne el material de consulta: el mapa de señales completo entre los conectores FMC de la ZCU102 y las placas LINCE Comunicación Serial, CDHS y AOCS, la tabla de frecuencias soportadas por el NCO del transceptor serie junto con su error medido, y el recuento del código desarrollado a lo largo del trabajo. El segundo (C.6–C.8) recoge el detalle de implementación que el Capítulo 3 deja fuera para no interrumpir el hilo: la descripción RTL de los bloques auxiliares del transceptor, la aplicación de testing del driver serie y el procedimiento completo del ejemplo con el que se validó el flujo de desarrollo. El tercero (C.9–C.10) contiene los volcados completos del terminal de las pruebas de validación hardware del Capítulo 4.

C.1 Mapa de señales LINCE Comunicación Serial

La tabla C.1 recoge el mapeado de los 14 drivers RS485/RS422 de la placa LINCE Comunicación Serial con el conector FMC HPC0 (P2) de la ZCU102. Cada driver expone las señales TX, RX, SLO y DE (esta última ausente en los drivers 7 y 11, que solo disponen de control de dirección hardware).

Driver	Señal	Red Interna	Red (FMC HPC0)	Pin FMC (P2)	Pin Zynq
Driver0	TX	TX0	FMC_HPC0_LA32_N	H38	T11
Driver0	RX	RX0	FMC_HPC0_LA26_N	D27	K15
Driver0	SLO	SLO0	FMC_HPC0_LA27_N	C27	L10
Driver0	DE	DE0	FMC_HPC0_LA33_N	G37	V11
Driver1	TX	TX1	FMC_HPC0_LA33_P	G36	V12
Driver1	RX	RX1	FMC_HPC0_LA30_N	H35	U6
Driver1	SLO	SLO1	FMC_HPC0_LA32_P	H37	U11
Driver1	DE	DE1	FMC_HPC0_LA27_P	C26	M10
Driver2	TX	TX2	FMC_HPC0_LA30_P	H34	V6
Driver2	RX	RX2	FMC_HPC0_LA26_P	D26	L15
Driver2	SLO	SLO2	FMC_HPC0_LA31_N	G34	V7
Driver2	DE	DE2	FMC_HPC0_LA31_P	G33	V8
Driver3	TX	TX3	FMC_HPC0_LA29_N	G31	U8
Driver3	RX	RX3	FMC_HPC0_LA29_P	G30	U9

Driver	Señal	Red Interna	Red (FMC HPC0)	Pin FMC (P2)	Pin Zynq
Driver3	SLO	SLO3	FMC_HPC0_LA28_N	H32	T6
Driver3	DE	DE3	FMC_HPC0_LA28_P	H31	T7
Driver4	TX	TX4	FMC_HPC0_LA14_P	C18	AC7
Driver4	RX	RX4	FMC_HPC0_LA13_N	D18	AC8
Driver4	SLO	SLO4	FMC_HPC0_LA22_P	G24	M15
Driver4	DE	DE4	FMC_HPC0_LA19_N	H23	K13
Driver5	TX	TX5	FMC_HPC0_LA20_N	G22	M13
Driver5	RX	RX5	FMC_HPC0_LA15_N	H20	Y9
Driver5	SLO	SLO5	FMC_HPC0_LA19_P	H22	L13
Driver5	DE	DE5	FMC_HPC0_LA20_P	G21	N13
Driver6	TX	TX6	FMC_HPC0_LA15_P	H19	Y10
Driver6	RX	RX6	FMC_HPC0_LA13_P	D17	AB8
Driver6	SLO	SLO6	FMC_HPC0_LA16_N	G19	AA12
Driver6	DE	DE6	FMC_HPC0_LA16_P	G18	Y12
Driver7	TX	TX7	FMC_HPC0_LA06_P	C10	AC2
Driver7	RX	RX7	FMC_HPC0_LA01_CC_N	D9	AC4
Driver7	SLO	SLO7	FMC_HPC0_LA01_CC_P	D8	AB4
Driver8	TX	TX8	FMC_HPC0_LA05_P	D11	AB3
Driver8	RX	RX8	FMC_HPC0_LA05_N	D12	AC3
Driver8	SLO	SLO8	FMC_HPC0_LA06_N	C11	AC1
Driver8	DE	DE8	FMC_HPC0_LA10_P	C14	W5
Driver9	TX	TX9	FMC_HPC0_LA02_P	H7	V2
Driver9	RX	RX9	FMC_HPC0_LA00_CC_N	G7	Y3
Driver9	SLO	SLO9	FMC_HPC0_LA00_CC_P	G6	Y4
Driver9	DE	DE9	FMC_HPC0_LA02_N	H8	V1
Driver10	TX	TX10	FMC_HPC0_LA09_P	D14	W2
Driver10	RX	RX10	FMC_HPC0_LA03_N	G10	Y1
Driver10	SLO	SLO10	FMC_HPC0_LA03_P	G9	Y2
Driver10	DE	DE10	FMC_HPC0_LA04_P	H10	AA2
Driver11	TX	TX11	FMC_HPC0_LA08_N	G13	V3
Driver11	RX	RX11	FMC_HPC0_LA08_P	G12	V4
Driver11	SLO	SLO11	FMC_HPC0_LA04_N	H11	AA1
Driver12	TX	TX12	FMC_HPC0_LA10_N	C15	W4
Driver12	RX	RX12	FMC_HPC0_LA07_N	H14	U4
Driver12	SLO	SLO12	FMC_HPC0_LA07_P	H13	U5
Driver12	DE	DE12	FMC_HPC0_LA09_N	D15	W1
Driver13	TX	TX13	FMC_HPC0_LA11_N	H17	AB5
Driver13	RX	RX13	FMC_HPC0_LA11_P	H16	AB6
Driver13	SLO	SLO13	FMC_HPC0_LA12_P	G15	W7

Driver	Señal	Red Interna	Red (FMC HPC0)	Pin FMC (P2)	Pin Zynq
Driver13	DE	DE13	FMC_HPC0_LA12_N	G16	W6

Tabla C.1: Mapa de señales entre la placa LINCE Comunicación Serial y el conector FMC HPC0 (P2) de la ZCU102.

C.2 Mapa de señales CDHS

La tabla C.2 recoge el mapeado de los cuatro subsistemas de la placa CDHS (SPI del ADC, CAN nominal y redundante, los tres canales serie y las cuatro líneas de PWM) con el conector FMC HPC0 de la ZCU102.

Subsistema	Señal	Red (FMC HPC0)	Pin FMC	Pin Zynq
SPI (ADC)	SDO	FMC_HPC0_LA02_N	H8	V1
SPI (ADC)	SDI	FMC_HPC0_LA00_CC_N	G7	Y3
SPI (ADC)	SCLK	FMC_HPC0_LA02_P	H7	V2
SPI (ADC)	CS_N	FMC_HPC0_LA00_CC_P	G6	Y4
CAN (Nominal)	TX_N	FMC_HPC0_LA21_P	H25	P12
CAN (Nominal)	RX_N	FMC_HPC0_LA22_P	G24	M15
CAN (Redundante)	TX_R	FMC_HPC0_LA21_N	H26	N12
CAN (Redundante)	RX_R	FMC_HPC0_LA22_N	G25	M14
RS1 (Serial 1)	TX	FMC_HPC0_LA15_P	H19	Y10
RS1 (Serial 1)	RX	FMC_HPC0_LA11_N	H17	AB5
RS1 (Serial 1)	DE	FMC_HPC0_LA16_P	G18	Y12
RS2 (Serial 2)	TX	FMC_HPC0_LA12_N	G16	W6
RS2 (Serial 2)	RX	FMC_HPC0_LA12_P	G15	W7
RS2 (Serial 2)	DE	FMC_HPC0_LA11_P	H16	AB6
RS3 (Serial 3)	TX	FMC_HPC0_LA07_N	H14	U4
RS3 (Serial 3)	RX	FMC_HPC0_LA07_P	H13	U5
RS3 (Serial 3)	DE	FMC_HPC0_LA08_N	G13	V3
PWM (Heaters)	IN1	FMC_HPC0_LA04_N	H11	AA1
PWM (Heaters)	IN2	FMC_HPC0_LA04_P	H10	AA2
PWM (Heaters)	IN3	FMC_HPC0_LA03_P	G9	Y2
PWM (Heaters)	IN4	FMC_HPC0_LA03_N	G10	Y1

Tabla C.2: Mapa de señales entre la placa CDHS y el conector FMC HPC0 de la ZCU102.

C.3 Mapa de señales AOCS

El mapa de la tabla C.3 corresponde a la revisión V2 de la placa, la versión cerrada del esquemático (sección 3.5.5). Respecto a la revisión fabricada, cada canal serie consume cuatro señales del FMC en lugar de tres, porque las líneas DE y RE dejan de estar cortocircuitadas, y el conector J4 alberga dos canales (RS41 y RS42).

Tabla C.3: Mapa de señales entre la revisión V2 de la placa AOCS y el conector FMC HPC0 de la ZCU102.

Subsistema	Señal	Red (FMC HPC0)	Pin FMC	Pin Zynq
RS1 (Serial J1)	TX	FMC_HPC0_LA19_P	H22	L13
RS1 (Serial J1)	RX	FMC_HPC0_LA17_CC_N	D21	N11
RS1 (Serial J1)	DE	FMC_HPC0_LA18_CC_P	C22	N9
RS1 (Serial J1)	RE	FMC_HPC0_LA20_P	G21	N13
MOT-PWM (J2)	PWM_X_1	FMC_HPC0_LA17_CC_P	D20	P11
MOT-PWM (J2)	PWM_X_2	FMC_HPC0_LA15_N	H20	Y9
MOT-PWM (J2)	PWM_Y_1	FMC_HPC0_LA16_N	G19	AA12
MOT-PWM (J2)	PWM_Y_2	FMC_HPC0_LA15_P	H19	Y10
MOT-PWM (J2)	PWM_Z_1	FMC_HPC0_LA16_P	G18	Y12
MOT-PWM (J2)	PWM_Z_2	FMC_HPC0_LA13_N	D18	AC8
RS3 (Serial J3)	TX	FMC_HPC0_LA11_N	H17	AB5
RS3 (Serial J3)	RX	FMC_HPC0_LA11_P	H16	AB6
RS3 (Serial J3)	DE	FMC_HPC0_LA14_P	C18	AC7
RS3 (Serial J3)	RE	FMC_HPC0_LA13_P	D17	AB8
RS41 (Serial J4)	TX	FMC_HPC0_LA12_N	G16	W6
RS41 (Serial J4)	RX	FMC_HPC0_LA09_N	D15	W1
RS41 (Serial J4)	DE	FMC_HPC0_LA10_N	C15	W4
RS41 (Serial J4)	RE	FMC_HPC0_LA12_P	G15	W7
RS42 (Serial J4)	TX	FMC_HPC0_LA07_N	H14	U4
RS42 (Serial J4)	RX	FMC_HPC0_LA07_P	H13	U5
RS42 (Serial J4)	DE	FMC_HPC0_LA09_P	D14	W2
RS42 (Serial J4)	RE	FMC_HPC0_LA08_N	G13	V3
RS5 (Serial J5)	TX	FMC_HPC0_LA05_N	D12	AC3
RS5 (Serial J5)	RX	FMC_HPC0_LA05_P	D11	AB3
RS5 (Serial J5)	DE	FMC_HPC0_LA08_P	G12	V4
RS5 (Serial J5)	RE	FMC_HPC0_LA06_N	C11	AC1
SPW1 (J6)	DIN_P	FMC_HPC0_LA02_P	H7	V2
SPW1 (J6)	DIN_N	FMC_HPC0_LA02_N	H8	V1
SPW1 (J6)	SIN_P	FMC_HPC0_LA04_P	H10	AA2
SPW1 (J6)	SIN_N	FMC_HPC0_LA04_N	H11	AA1
SPW1 (J6)	SOUT_P	FMC_HPC0_LA01_CC_P	D8	AB4
SPW1 (J6)	SOUT_N	FMC_HPC0_LA01_CC_N	D9	AC4
SPW1 (J6)	DOUT_P	FMC_HPC0_LA00_CC_P	G6	Y4
SPW1 (J6)	DOUT_N	FMC_HPC0_LA00_CC_N	G7	Y3
SPW2 (J7)	DIN_P	FMC_HPC0_LA24_P	H28	L12
SPW2 (J7)	DIN_N	FMC_HPC0_LA24_N	H29	K12

Subsistema	Señal	Red (FMC HPC0)	Pin FMC	Pin Zynq
SPW2 (J7)	SIN_P	FMC_HPC0_LA21_P	H25	P12
SPW2 (J7)	SIN_N	FMC_HPC0_LA21_N	H26	N12
SPW2 (J7)	SOUT_P	FMC_HPC0_LA28_P	H31	T7
SPW2 (J7)	SOUT_N	FMC_HPC0_LA28_N	H32	T6
SPW2 (J7)	DOUT_P	FMC_HPC0_LA30_P	H34	V6
SPW2 (J7)	DOUT_N	FMC_HPC0_LA30_N	H35	U6
RS8 (Serial J8)	TX	FMC_HPC0_LA22_P	G24	M15
RS8 (Serial J8)	RX	FMC_HPC0_LA20_N	G22	M13
RS8 (Serial J8)	DE	FMC_HPC0_LA19_N	H23	K13
RS8 (Serial J8)	RE	FMC_HPC0_LA23_P	D23	L16

C.4 Tabla de frecuencias del NCO

La tabla del NCO del transceptor serie contiene **54 entradas**, de 50 a 4 000 000 de baudios. La tabla C.4 recoge las **29 que ejerce el *testbench* `tb_NCO.vhd`**, que son las del rango de uso previsto (de 9 600 a 3,6864 Mbaudios): para cada una, los dos valores de incremento que la función de síntesis calcula (el nominal y el de doble frecuencia) y la frecuencia medida en simulación, con su error en ppm respecto a la nominal. Las entradas por debajo de 9 600 baudios quedan fuera del uso previsto y la de 4 Mbaudios exige un tiempo de simulación desproporcionado, de modo que ninguna de ellas se barre.

Baud	Half	INC_ROM	INC_HALF_ROM	Medido (Hz)	Error (ppm)
9600	0	412316	824633	9600,002	0,2
14400	0	618475	1236950	14399,988	-0,8
19200	0	824633	1649267	19200,012	0,6
28800	0	1236950	2473901	28799,977	-0,8
31250	0	1342177	2684354	31250,000	0,0
38400	0	1649267	3298534	38400,025	0,6
56000	0	2405181	4810363	55999,978	-0,4
57600	0	2473901	4947802	57600,037	0,6
74400	0	3195455	6390911	74400,057	0,8
115200	0	4947802	9895604	115200,074	0,6
128000	0	5497558	10995116	128000,000	0,0
153600	0	6597069	13194139	153600,393	2,6
230400	0	9895604	19791209	230398,820	-5,1
256000	0	10995116	21990232	256000,000	0,0
312500	0	13421772	26843545	312500,000	0,0
460800	0	19791209	39582418	460808,258	17,9
500000	0	21474836	42949672	500000,000	0,0
576000	0	24739011	49478023	576003,686	6,4
614400	0	26388279	52776558	614401,573	2,6
750000	0	32212254	64424509	749990,625	-12,5
921600	0	39582418	79164837	921616,515	17,9
1000000	0	42949672	85899345	1000000,000	0,0
1152000	0	49478023	98956046	1152007,373	6,4
1500000	0	64424509	128849018	1499925,004	-50,0
1843200	0	79164837	158329674	1843148,097	-28,2
2000000	0	85899345	171798691	2000000,000	0,0
2500000	0	107374182	214748364	2500000,000	0,0
3000000	0	128849018	257698037	3000300,030	100,0
3686400	0	158329674	316659348	3685956,506	-120,3

Tabla C.4: Velocidades del NCO verificadas en simulación.

C.5 Volumen y distribución del código desarrollado

Se recoge aquí el recuento de líneas de código de autoría propia escritas en el trabajo, desglosado por lenguaje y por función. El criterio de recuento es restrictivo: se excluye todo el código generado automáticamente por las herramientas (los IP de AMD, los ficheros que Vivado produce al empaquetar un IP o al sintetizar un diseño de bloques), el código de terceros incorporado al proyecto (el sistema de compilación `waf` de RTEMS, el árbol de dispositivos de Xilinx) y las copias de un mismo fichero replicadas entre las distintas variantes del proyecto, de las que se contabiliza únicamente la versión final.

Lenguaje	Función	Líneas	Peso
TCL	Automatización del flujo de Vivado	5 338	26,4 %
VHDL	RTL sintetizable (transceptor, puente)	4 710	23,3 %
VHDL	<i>Testbenches</i> de simulación	4 551	22,5 %
C	Driver RTEMS y aplicaciones de prueba	2 617	13,0 %
Python	Utilidades de instrumentación y GUI	1 525	7,6 %
Bash	Compilación y despliegue de imágenes	1 231	6,1 %
XDC	<i>Constraints</i> de la FPGA	216	1,1 %
Total		20 188	100 %

Tabla C.5: Distribución de las líneas de código desarrolladas en el trabajo.

La distribución es en sí misma un resultado, y admite dos lecturas que respaldan los principios metodológicos enunciados en la sección de metodología.

La primera es que el **código de verificación pesa prácticamente lo mismo que el código que verifica**: 4 551 líneas de *testbench* frente a 4 710 de VHDL sintetizable, una relación de 0,97:1. Por cada línea de hardware descrita se escribió casi una línea destinada exclusivamente a comprobarla en simulación, lo que cuantifica la política de no integrar ningún bloque sin verificación previa aislada.

La segunda es que la **automatización es la mayor partida del trabajo**: TCL, Bash y Python suman 8 094 líneas, el 40,1 % del total, por encima de cualquier otra categoría y muy por encima del código de aplicación. Es la traducción cuantitativa de una decisión deliberada, la de invertir por adelantado en herramientas que hicieran el flujo reproducible y barato de iterar, cuyo retorno se manifestó en las campañas de depuración, donde regenerar el diseño completo desde cero pasó a ser un comando en lugar de una tarde de trabajo manual.

C.6 Descripción RTL del registro de desplazamiento y la FIFO

Este apartado detalla los dos bloques auxiliares situados entre el receptor y el PS, cuyo comportamiento se resume en la sección 3.2.4 del capítulo de desarrollo.

C.6.1 Registro de desplazamiento

El registro de desplazamiento convierte a paralelo los bits que va entregando el receptor. Sus entradas son el bit muestreado (D), un pulso de habilitación por bit recibido (**Enable**, conectado al **Valid_out** del receptor) y las dos señales de configuración que determinan el formato de la palabra (**data_bits** y **bit_order**); su salida Q es la palabra de 9 bits. El registro solo evoluciona en los ciclos en que **Enable** está activo, de modo que la FSM del receptor controla exactamente qué muestras entran en la palabra.

Está diseñado para poder almacenar datos tanto si la configuración es *LSB-first* como *MSB-first*, de modo que una vez recibido el mensaje la palabra en la salida siempre es

la correcta independientemente del modo configurado. Para ello dispone de dos caminos de desplazamiento combinacionales, seleccionados por `bit_order`: en *LSB first* el bit entrante se inserta en la posición más significativa de la palabra activa y el contenido se desplaza hacia la derecha; en *MSB first* se inserta en la posición cero y el contenido se desplaza hacia la izquierda. Cada camino tiene a su vez cinco variantes, una por anchura de palabra, que fijan dónde se inserta el bit y rellenan a cero las posiciones no utilizadas.

El efecto de esta **normalización** es que, completada la trama, el primer bit de datos recibido ocupa siempre `Q(0)` y la palabra está alineada al LSB con independencia de la configuración. Los dos consumidores de la salida, la FIFO y el cálculo de paridad esperada del receptor, pueden así ignorar por completo el orden de bits configurado, y el software del PS recibe la palabra en el mismo formato en los cinco anchos y en los dos órdenes posibles.

C.6.2 FIFO

La FIFO es un bloque IP con 9 bits de datos (para poder almacenar mensajes de 9 bits) y 512 de profundidad, que permite almacenar temporalmente varios mensajes y habilitarlos para su lectura posterior por el PS. Está configurada en modo *first word fall through*, de modo que la palabra más antigua está disponible en la salida sin necesidad de una lectura previa, y su reset síncrono se deriva del reset global del transceptor. La escritura la ordena directamente el pulso `Store_out` del receptor, que solo se emite cuando la trama ha superado las comprobaciones de paridad y bit de parada.

La habilitación de lectura se construye como la conjunción de FIFO no vacía y la señal `Data_read` procedente del PS. Esta última se toma a **nivel**: en la versión inicial se empleaba el flanco detectado de `Data_read`, lo que resulta adecuado cuando el PS lee byte a byte, pero deja de serlo con el transporte por DMA descrito en la sección 3.7, donde `Data_read` lo conduce la señal *tready* del canal AXI-Stream. La latencia del detector de flanco, de dos etapas de registro, hacía que el receptor de DMA capturase varias veces la misma palabra de la FIFO. Con la lectura a nivel, la FIFO avanza exactamente en el ciclo en que el consumidor acepta el dato.

C.7 Aplicación de testing del driver serie en RTEMS

Este apartado detalla la aplicación de pruebas con la que se ejercitó el driver serie sobre la ZCU102, resumida en la sección 3.6.1 del capítulo de desarrollo. Está repartida en tres ficheros: `init.c`, con la configuración estática del sistema; el driver (`transceiver.c` y `transceiver.h`); y `main.c`, con la aplicación propiamente dicha.

C.7.1 Configuración del sistema — `init.c`

`init.c` no contiene lógica de aplicación. Define el entorno de ejecución mediante macros que `<rtems/confdefs.h>` convierte en estructuras de datos internas del núcleo.

Listado C.1: Configuración estática de RTEMS en `init.c`

```
CONFIGURE_APPLICATION_NEEDS_CLOCK_DRIVER    // rtems_task_wake_after()
CONFIGURE_APPLICATION_NEEDS_CONSOLE_DRIVER  // stdin/stdout (printf, fgets)
CONFIGURE_MICROSECONDS_PER_TICK 10000       // Tick = 10 ms (100 Hz)
CONFIGURE_UNLIMITED_OBJECTS                 // Sin limite de tareas/semaforos
CONFIGURE_UNIFIED_WORK_AREAS                 // Un unico pool de memoria
```

```
CONFIGURE_INIT_TASK_STACK_SIZE (64 KB)    // Stack grande para Init
```

C.7.2 Aplicación — main.c

La aplicación tiene tres secciones claramente separadas.

Callback de recepción `on_rx_data()`. Se registra en el driver y se invoca desde la tarea *worker* cuando llegan datos. Implementa un ensamblador de líneas por canal: por cada byte recibido acumula hasta encontrar un carácter de fin de línea (`\n` o `\r`), momento en el que imprime la línea completa con el formato `[RX UART 02]: mensaje`. Hay un `AppLineBuffer` de 1024 bytes por cada uno de los 14 canales, declarado estáticamente en el array `rx_lines[MAX_TRANSCEIVERS]`. Esto evita mezclar caracteres de UART distintas en la misma línea de consola.

Tarea de consola de transmisión `Tx_Console_Task()`. Es una tarea de RTEMS independiente, de prioridad 100, que bloquea en `fgets(stdin)` esperando comandos del usuario por la consola USB. El protocolo de comandos es el siguiente.

Listado C.2: Protocolo de comandos de la aplicación de testing

1	<ID> <MENSAJE>	-> Envía MENSAJE por la UART numero ID
2	ALL <MENSAJE>	-> Envía MENSAJE por TODAS las UARTs
3	<ID> SLO ON	-> Reconfigura UART ID con Slew Rate limitado
4	<ID> SLO OFF	-> Reconfigura UART ID con Slew Rate normal
5	ALL SLO ON/OFF	-> Aplica configuracion SLO a todas las UARTs

Los comandos SLO reejecutan `Transceiver_Init()` con una configuración distinta, lo que permite reconfigurar el hardware en caliente sin reiniciar el sistema.

Función `Init()`. RTEMS la llama al arrancar en lugar de `main()`. La secuencia de inicialización es:

1. `mmu_map_pl_axi_early()`: mapea en la MMU la región de la PL para acceso *bare-metal*.
2. `Transceiver_Global_INIT()`: descubre el hardware e instala la ISR maestra.
3. Bucle de inicialización: para cada UART se invoca `Transceiver_Init()`, se registra su *callback* de recepción y se envía el mensaje de disponibilidad del canal.
4. Creación y lanzamiento de `Tx_Console_Task`.
5. `rtems_task_delete(RTEMS_SELF)`: la tarea `Init` se elimina a sí misma; el núcleo queda con las tareas *worker* y la tarea de consola.

C.8 Flujo de desarrollo de extremo a extremo: ejemplo de la puerta AND

Este apartado recoge el procedimiento completo del ejemplo de validación descrito en el apartado 3.1.3: la puerta AND descrita en VHDL y sintetizada en la PL, la imagen de arranque que la lleva a la placa y la traza que confirma su funcionamiento. El proyecto completo, con el diseño y el código de las dos mitades, está en `tfm/00_docs/AND_example` dentro del repositorio del proyecto [5].

C.8.1 Descripción VHDL y *Block Design*

El proyecto de Vivado, llamado `and_gate_ps_p`, parte de la descripción de la puerta:

Listado C.3: AND_GATE.vhd

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity AND_GATE is
5     Port ( A_B_IN : in  STD_LOGIC_VECTOR (1 downto 0);
6           Z_OUT  : out STD_LOGIC);
7 end AND_GATE;
8
9 architecture Behavioral of AND_GATE is
10 begin
11     Z_OUT <= A_B_IN(1) and A_B_IN(0);
12 end Behavioral;
```

Sobre un *Block Design* nuevo se añadió el bloque Zynq UltraScale+ MPSoC IP, configurado con la *Block Automation* de Vivado, el fichero VHDL anterior y dos AXI GPIO IP para comunicar el PS con la PL. Las conexiones restantes se resolvieron con *Automate Connection*. El resultado es la figura C.1, y en ella se aprecia la estructura que se repite en todos los diseños del trabajo: el bloque del PS, los periféricos de la PL y el interconector AXI que los une, con el bloque VHDL propio colgando de uno de los dos AXI GPIO.

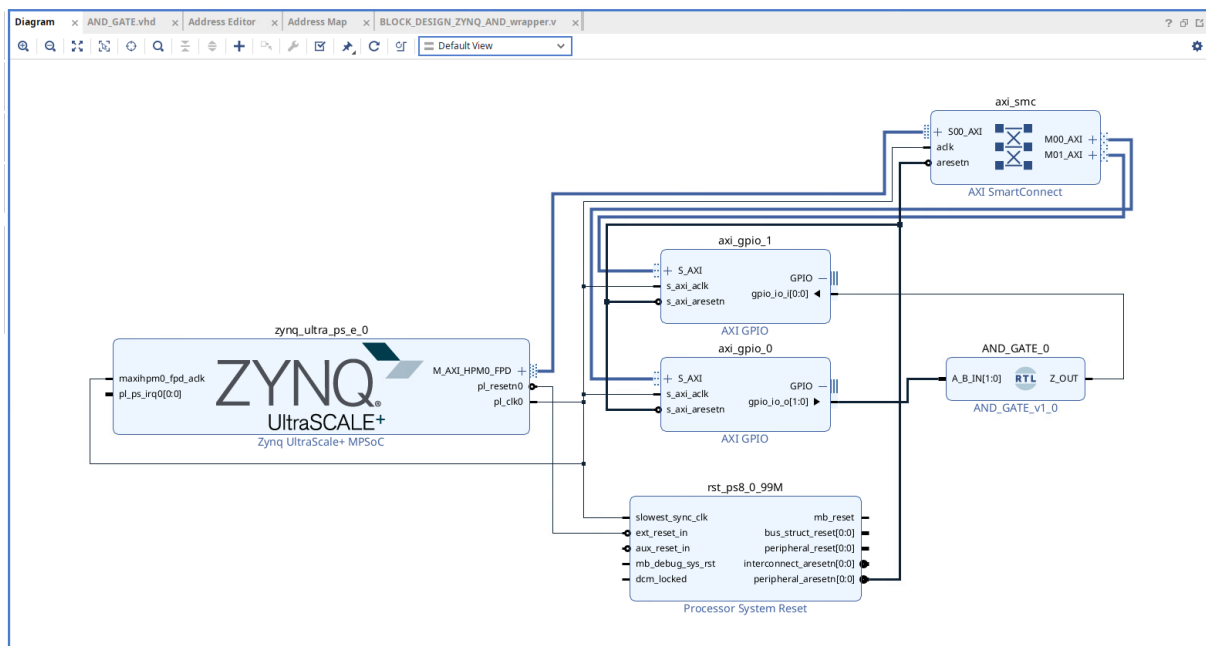


Figura C.1: *Block Design* del ejemplo de la puerta AND.

Después se creó un *Wrapper* del *Block Design* para que Vivado pudiera sintetizarlo. La pestaña *Address Editor* lista las direcciones base con las que el PS accede a los registros de cada periférico de la PL. Por último se generó el *BitStream* y se exportó el hardware como `.xsa`.

C.8.2 Generación de la imagen de arranque en Vitis

Este paso empaqueta el hardware generado en Vivado, junto con el software de arranque, en el único binario que el MPSoC ejecuta desde la tarjeta SD:

1. Se creó un componente de tipo *Platform* en Vitis, tomando como base el .xsa exportado desde Vivado, y se compiló.
2. Sobre esa plataforma se creó una aplicación a partir del ejemplo *Zynq MP FSBL*, el cargador de arranque de primera etapa (*First Stage Boot Loader*), que es el primer programa que ejecuta el PS y el que carga el resto de componentes de la imagen. Al compilarla se obtiene el `zynqmp_fsbl_project.elf`.
3. Se lanzó el generador de imágenes de Vitis (*Create Zynq UltraScale+ Boot Image*) y se añadieron los componentes en el orden de la figura C.2, ya que el orden dentro del fichero BIF (*Boot Image Format*) determina la secuencia de arranque.

El modo y el orden de los componentes dentro del BIF son la parte del procedimiento que la documentación de AMD no reúne en un solo sitio, y un error en cualquiera de los dos impide el arranque sin dar un mensaje útil. De ahí el detalle de la lista siguiente:

- `zynqmp_fsbl_project.elf`, en modo *bootloader*, destino a53-0 y nivel de excepción EL3. Se encuentra en la carpeta `build` de la aplicación anterior.
- `pmufw.elf`, en modo *pmufw_image* y destino a53-0. Es el firmware de la unidad de gestión de potencia y se descargó del repositorio de firmware precompilado de AMD para la ZCU102¹.
- `BLOCK_DESIGN_ZYNQ_AND_wrapper.bit`, en modo *datafile* y con la PL como dispositivo de destino. Es el *bitstream* generado por Vivado.
- `bl31.elf`, en modo *datafile*, destino a53-0 y nivel de excepción EL3. Es el firmware de ARM Trusted Firmware y se descarga del mismo sitio que el PMUFW.
- `u-boot.elf`, en modo *datafile*, destino a53-0 y nivel de excepción EL2. U-Boot es un gestor de arranque de segunda etapa. Se ejecuta después del FSBL, inicializa los periféricos necesarios (tarjeta SD, red, consola serie) y carga en memoria la imagen del sistema operativo (en este caso `rtems.img`), a la que cede el control. Ofrece además una consola interactiva desde la que se puede inspeccionar la memoria o elegir manualmente qué imagen arrancar, lo que resultó útil durante la depuración.
- `system.dtb`, en modo *datafile*, destino a53-0 y dirección de carga `0x100000`. Este *device tree* es el que permite a BL31 saber dónde está enlazado el BL33 (U-Boot) y arrancarlo.

C.8.3 Aplicación de RTEMS y arranque desde la tarjeta SD

La aplicación se reparte en dos fuentes. `and_demo.c` contiene la tarea `Init`: escribe la pareja de entradas (A,B) en el registro de datos del `axi_gpio_0` (mapeado en `0xA0000000`), espera un instante a que la lógica de la PL se estabilice, lee la salida Z del `axi_gpio_1` (`0xA0010000`) e imprime cada resultado por la consola serie. `mmu_pl_map.c` declara como memoria de periférico el rango `0xA0000000-0xA01FFFFF`, donde Vivado ha situado los

¹https://github.com/Xilinx/soc-prebuilt-firmware/tree/xilinx_v2025.1/zcu102-zynqmp

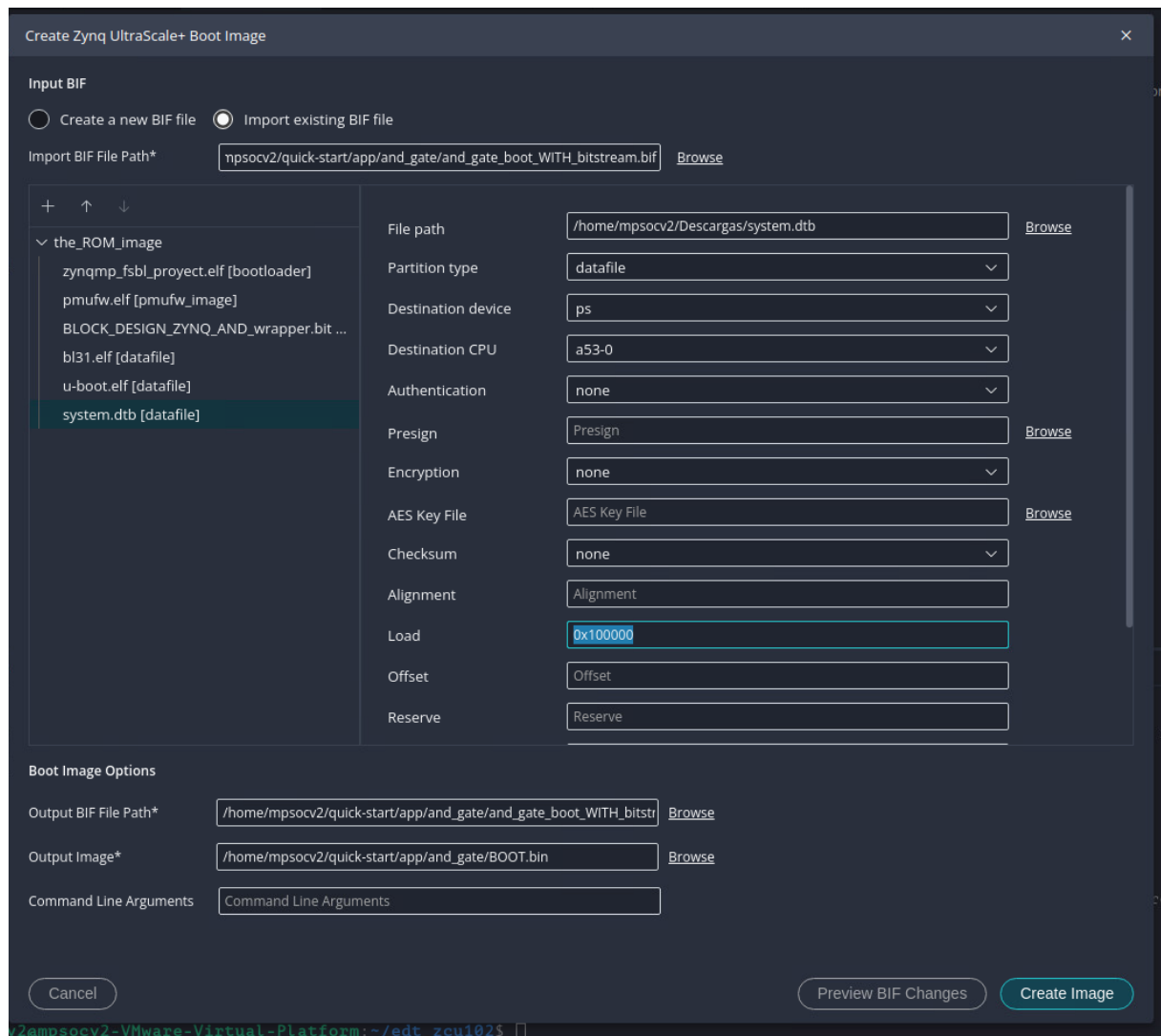


Figura C.2: Generador de imágenes de arranque de Vitis.

AXI GPIO; el mapeado se realiza nada más arrancar `Init`, porque RTEMS desconoce a priori las direcciones de la PL. La compilación necesita además un `wscript`.

Los dos ficheros, `BOOT.BIN` y `rtems.img`, se copian en la partición FAT de una tarjeta SD, que se inserta en el zócalo J100 de la placa [1]. El `BOOT.BIN` es el que el MPSoC ejecuta nada más alimentarse: inicializa el PS, carga el *bitstream* en la PL y encadena el resto de etapas hasta U-Boot, que a su vez carga el `rtems.img` en memoria y le cede el control. Si el ensayo solo ejerce la PL, el `BOOT.BIN` basta por sí solo, ya que lleva dentro el *bitstream*.

Antes de encender la placa hay que colocar el selector de modo de arranque en la posición de tarjeta SD. Es el conmutador DIP `SW6`, la referencia 44 del plano de componentes de la ZCU102 [1], cuyas posiciones recoge la tabla C.6. Conviene tenerlo presente porque la configuración de fábrica es `QSPI32`: sin cambiar el conmutador, la placa no arranca desde la tarjeta aunque los ficheros estén correctamente copiados.

Modo de arranque	PS_MODE[3:0]	SW6 [4:1]
JTAG	0000	ON, ON, ON, ON
QSPI32 (por defecto)	0010	ON, ON, OFF, ON
Tarjeta SD	1110	OFF, OFF, OFF, ON

Tabla C.6: Posiciones del selector de modo de arranque `SW6` de la ZCU102 [1].

C.8.4 Barrido de la tabla de verdad

Con la tarjeta insertada y el conmutador en su sitio, el barrido completo sale por la consola serie:

Listado C.4: Salida de terminal — barrido de la puerta AND

```

1 ## Transferring control to RTEMS (at address 000100000) ...
2 Barrido AND via PL
3 A=0, B=0, Z=0 (raw=0x00000000)
4 A=0, B=1, Z=0 (raw=0x00000000)
5 A=1, B=0, Z=0 (raw=0x00000000)
6 A=1, B=1, Z=1 (raw=0x00000001)
7
8 [ RTEMS shutdown ]

```

La primera línea la emite U-Boot al ceder el control a la imagen de RTEMS. Las cuatro siguientes son las escritas por `and_demo.c`, con la salida Z a uno únicamente cuando ambas entradas valen uno. La salida se conserva en `tfm/00_docs/logs_terminal/and_gate_output.txt`.

C.9 Volcados completos de terminal de las pruebas serie RS422/RS485

Este apartado recoge la salida íntegra del terminal de las dos sesiones de prueba de comunicaciones serie RS422/RS485 descritas en las secciones 4.1.2 y 4.2 del capítulo de validación hardware.

C.9.1 Placa CDHS (3 transceptores)

La sesión recoge el descubrimiento dinámico de los tres transceptores instanciados en el *bitstream* y los intercambios de *loopback* entre UART 0, UART 1 y UART 2.

Listado C.5: Salida completa del terminal — prueba RS422/RS485 con la placa CDHS (3 transceptores)

```
MMU_MAP: return from mmu_map_pl_axi_early()
[TRANSCIEVER DEBUG] Detectados: 3 | Base: 0xA0000000 | INT: 0xA0003000
[TRANSCIEVER DEBUG] - Transceptor 0: 0xA0000000
[TRANSCIEVER DEBUG] - Transceptor 1: 0xA0001000
[TRANSCIEVER DEBUG] - Transceptor 2: 0xA0002000

=== ARRANQUE SISTEMA ZCU102 (5 UARTs) ===
UART 00 [OK] Base: 0xA0000000
UART 01 [OK] Base: 0xA0001000
UART 02 [OK] Base: 0xA0002000
[RX UART 00]: UART 2 Lista.

=====
CONSOLA DE CONTROL MULTI-TRANSCIEPTOR
=====
Formato: <ID> <MENSAJE> | <ID> SLO ON | <ID> SLO OFF
Ejemplos:
'0 Hola' -> Envía 'Hola' por UART 0
'ALL Reset' -> Envía 'Reset' por TODAS las UARTs
'2 SLO ON' -> Activa el modo Slow Rate en UART 2
'ALL SLO OFF' -> Desactiva el modo Slow en TODAS
=====

CMD>
---- Sent utf8 encoded message: "0 hola" ----
0 hola
Tx -> UART 0: OK
CMD> [RX UART 02]: hola
---- Sent utf8 encoded message: "2 hola" ----
2 hola
Tx -> UART 2: OK
CMD> [RX UART 00]: hola
---- Sent utf8 encoded message: "0 hola" ----
0 hola
Tx -> UART 0: OK
CMD> [RX UART 02]: hola
---- Sent utf8 encoded message: "2 hola" ----
2 hola
Tx -> UART 2: OK
CMD> [RX UART 00]: hola
---- Sent utf8 encoded message: "0 hola" ----
0 hola
Tx -> UART 0: OK
CMD> [RX UART 02]: ?hola
---- Sent utf8 encoded message: "2 hola" ----
2 hola
Tx -> UART 2: OK
CMD> [RX UART 00]: ?hola
---- Sent utf8 encoded message: "0 hola" ----
0 hola
Tx -> UART 0: OK
CMD> [RX UART 02]: ?hola
---- Sent utf8 encoded message: "2 hola" ----
2 hola
Tx -> UART 2: OK
CMD> [RX UART 00]: ?hola
---- Sent utf8 encoded message: "0 hola" ----
0 hola
Tx -> UART 0: OK
CMD> [RX UART 02]: hola
---- Sent utf8 encoded message: "2 hola" ----
2 hola
Tx -> UART 2: OK
CMD> [RX UART 00]: hola
---- Sent utf8 encoded message: "0 HOLA MUNDO" ----
0 HOLA MUNDO
```

```

Tx -> UART 0: OK
CMD> [RX UART 02]: HOLA MUNDO
---- Sent utf8 encoded message: "1 HOLA MUNDO" ----
1 HOLA MUNDO
Tx -> UART 1: OK
CMD>
---- Sent utf8 encoded message: "2 HOLA MUNDO" ----
2 HOLA MUNDO
Tx -> UART 2: OK
CMD> [RX UART 00]: HOLA MUNDO

```

C.9.2 Placa AOCS (5 transceptores)

La sesión recoge el descubrimiento dinámico de los cinco transceptores instanciados en el *bitstream* y los cuatro pares de *loopback* ejercitados entre UART 0 y el resto de canales.

Listado C.6: Salida completa del terminal — prueba RS422/RS485 con la placa AOCS (5 transceptores)

```

MMU_MAP: return from mmu_map_pl_axi_early()
[TRANSCIEVER DEBUG] Detectados: 5 | Base: 0xA0000000 | INT: 0xA0005000
[TRANSCIEVER DEBUG] - Transceptor 0: 0xA0000000
[TRANSCIEVER DEBUG] - Transceptor 1: 0xA0001000
[TRANSCIEVER DEBUG] - Transceptor 2: 0xA0002000
[TRANSCIEVER DEBUG] - Transceptor 3: 0xA0003000
[TRANSCIEVER DEBUG] - Transceptor 4: 0xA0004000

=== ARRANQUE SISTEMA ZCU102 (5 UARTs) ===
UART 00 [OK] Base: 0xA0000000
UART 01 [OK] Base: 0xA0001000
UART 02 [OK] Base: 0xA0002000
UART 03 [OK] Base: 0xA0003000
UART 04 [OK] Base: 0xA0004000
[RX UART 00]: UART 4 Lista.

=====
CONSOLA DE CONTROL MULTI-TRANSCIEPTOR
=====
Formato: <ID> <MENSAJE> | <ID> SLO ON | <ID> SLO OFF
Ejemplos:
'0 Hola' -> Envía 'Hola' por UART 0
'ALL Reset' -> Envía 'Reset' por TODAS las UARTs
'2 SLO ON' -> Activa el modo Slow Rate en UART 2
'ALL SLO OFF' -> Desactiva el modo Slow en TODAS
=====

CMD>
---- Sent utf8 encoded message: "0 HOLA" ----
0 HOLA
Tx -> UART 0: OK
CMD> [RX UART 04]: HOLA
---- Sent utf8 encoded message: "4 HOLA" ----
4 HOLA
Tx -> UART 4: OK
CMD> [RX UART 00]: HOLA
---- Sent utf8 encoded message: "0 HOLA" ----
0 HOLA
Tx -> UART 0: OK
CMD> [RX UART 03]: HOLA
---- Sent utf8 encoded message: "3 HOLA" ----
3 HOLA
Tx -> UART 3: OK
CMD> [RX UART 00]: HOLA
---- Sent utf8 encoded message: "0 HOLA" ----
0 HOLA
Tx -> UART 0: OK
CMD> [RX UART 02]: HOLA
---- Sent utf8 encoded message: "2 HOLA" ----
2 HOLA
Tx -> UART 2: OK
CMD> [RX UART 00]: HOLA
---- Sent utf8 encoded message: "0 HOLA" ----

```

```
0 HOLA
Tx -> UART 0: OK
CMD> [RX UART 01]: HOLA
---- Sent utf8 encoded message: "1 HOLA" ----
1 HOLA
Tx -> UART 1: OK
CMD> [RX UART 00]: HOLA
```

C.10 Volcado completo del barrido del ADC

Este apartado recoge la salida íntegra del terminal del barrido de tensión aplicado al canal CH2 del ADC ADS7950 de la placa CDHS, descrito en la sección 4.1 del capítulo de validación hardware. Se aprecian los dos tramos de subida y bajada de la rampa aplicada a CH2, los dos escalones intermedios en los que la fuente se dejó fija y la estabilidad de los otros tres canales durante todo el ensayo.

Listado C.7: Salida completa del terminal — barrido de tensión sobre CH2 del ADS7950 (placa CDHS)

```
ADS7950: CH0= 244 CH1= 43 CH2= 0 CH3= 19
ADS7950: CH0= 243 CH1= 43 CH2= 0 CH3= 19
ADS7950: CH0= 244 CH1= 43 CH2= 0 CH3= 19
ADS7950: CH0= 245 CH1= 43 CH2= 0 CH3= 19
ADS7950: CH0= 244 CH1= 43 CH2= 0 CH3= 19
ADS7950: CH0= 245 CH1= 44 CH2= 0 CH3= 19
ADS7950: CH0= 244 CH1= 43 CH2=1577 CH3= 19
ADS7950: CH0= 245 CH1= 43 CH2=1577 CH3= 19
ADS7950: CH0= 244 CH1= 43 CH2=1579 CH3= 19
ADS7950: CH0= 245 CH1= 43 CH2=1582 CH3= 19
ADS7950: CH0= 245 CH1= 43 CH2=1580 CH3= 20
ADS7950: CH0= 245 CH1= 43 CH2=1827 CH3= 19
ADS7950: CH0= 245 CH1= 43 CH2=2156 CH3= 20
ADS7950: CH0= 245 CH1= 44 CH2=2501 CH3= 20
ADS7950: CH0= 245 CH1= 43 CH2=2814 CH3= 20
ADS7950: CH0= 246 CH1= 43 CH2=2934 CH3= 21
ADS7950: CH0= 245 CH1= 43 CH2=3419 CH3= 21
ADS7950: CH0= 245 CH1= 43 CH2=3565 CH3= 21
ADS7950: CH0= 245 CH1= 43 CH2=3535 CH3= 22
ADS7950: CH0= 245 CH1= 43 CH2=3536 CH3= 22
ADS7950: CH0= 246 CH1= 43 CH2=3069 CH3= 22
ADS7950: CH0= 246 CH1= 43 CH2=2770 CH3= 22
ADS7950: CH0= 246 CH1= 43 CH2=2436 CH3= 22
ADS7950: CH0= 247 CH1= 43 CH2=1896 CH3= 22
ADS7950: CH0= 246 CH1= 43 CH2=1509 CH3= 22
ADS7950: CH0= 246 CH1= 43 CH2=1276 CH3= 22
ADS7950: CH0= 247 CH1= 43 CH2= 928 CH3= 22
ADS7950: CH0= 246 CH1= 43 CH2= 433 CH3= 22
ADS7950: CH0= 247 CH1= 43 CH2= 400 CH3= 22
ADS7950: CH0= 247 CH1= 43 CH2= 583 CH3= 22
ADS7950: CH0= 246 CH1= 43 CH2=1238 CH3= 22
ADS7950: CH0= 247 CH1= 43 CH2=1808 CH3= 23
ADS7950: CH0= 247 CH1= 43 CH2=1827 CH3= 23
ADS7950: CH0= 247 CH1= 43 CH2=1826 CH3= 23
ADS7950: CH0= 248 CH1= 43 CH2= 647 CH3= 23
ADS7950: CH0= 248 CH1= 44 CH2= 245 CH3= 23
ADS7950: CH0= 248 CH1= 43 CH2= 189 CH3= 23
ADS7950: CH0= 248 CH1= 43 CH2= 166 CH3= 22
ADS7950: CH0= 247 CH1= 43 CH2= 148 CH3= 23
ADS7950: CH0= 248 CH1= 43 CH2= 130 CH3= 22
ADS7950: CH0= 248 CH1= 43 CH2= 116 CH3= 22
ADS7950: CH0= 248 CH1= 43 CH2= 102 CH3= 22
ADS7950: CH0= 248 CH1= 43 CH2=1823 CH3= 23
ADS7950: CH0= 247 CH1= 43 CH2=1821 CH3= 23
ADS7950: CH0= 248 CH1= 43 CH2=1820 CH3= 23
ADS7950: CH0= 248 CH1= 43 CH2=1825 CH3= 23
ADS7950: CH0= 248 CH1= 43 CH2=1826 CH3= 23
ADS7950: CH0= 250 CH1= 43 CH2=1826 CH3= 23
ADS7950: CH0= 249 CH1= 43 CH2=1826 CH3= 23
ADS7950: CH0= 249 CH1= 44 CH2=1827 CH3= 23
```

ADS7950:	CH0=	249	CH1=	43	CH2=	1825	CH3=	23
ADS7950:	CH0=	249	CH1=	43	CH2=	1826	CH3=	24
ADS7950:	CH0=	248	CH1=	43	CH2=	1820	CH3=	24
ADS7950:	CH0=	249	CH1=	43	CH2=	390	CH3=	23
ADS7950:	CH0=	249	CH1=	43	CH2=	230	CH3=	23
ADS7950:	CH0=	250	CH1=	43	CH2=	189	CH3=	23